•

Contents

```python
from flask import Flask, render_template, request, jsonify, session, redirect, url_for
import pyodbc
import json
import random
from datetime import datetime, timedelta
import config

app = Flask(__name__)
app.secret_key = 'your_secret_key_here'

# 数据库配置 - Windows身份验证
DB_CONFIG = {
    'server': config.DB_SERVER,
    'database': config.DB_NAME,
    'trusted_connection': 'yes',
    'driver': '{ODBC Driver 17 for SQL Server}'
}

def get_db_connection():
    """获取数据库连接 - Windows身份验证"""
    conn_str = f"DRIVER={DB_CONFIG['driver']};SERVER={DB_CONFIG['server']};DATABASE=
{DB_CONFIG['database']};Trusted_Connection=yes"
    try:
        return pyodbc.connect(conn_str)
    except Exception as e:
        print(f"数据库连接失败: {str(e)}")
        raise

@app.route('/')
def index():
    """首页"""
    return render_template('index.html')

@app.route('/problems')
def problems():
    """题目列表页面"""
    return render_template('problems.html')

@app.route('/contests')
def contests():
    """比赛列表页面"""
    return render_template('contests.html')

@app.route('/submissions')
def submissions():
    """提交记录页面"""
    return render_template('submissions.html')

@app.route('/users')
def users():
    """用户列表页面"""
    return render_template('users.html')
```

```python
@app.route('/profile')
def profile():
    """用户个人资料页面"""
    return render_template('profile.html')

# API 路由
@app.route('/api/register', methods=['POST'])
def register():
    """用户注册"""
    data = request.json
    handle = data.get('handle')
    email = data.get('email')
    password = data.get('password')
    name = data.get('name', '')

    if not handle or not email or not password:
        return jsonify({'success': False, 'message': '请填写完整信息'})

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 检查用户名和邮箱是否已存在
        cursor.execute("SELECT user_id FROM Users WHERE handle = ? OR email = ?",
(handle, email))
        if cursor.fetchone():
            return jsonify({'success': False, 'message': '用户名或邮箱已存在'})

        # 插入新用户
        cursor.execute("""
            INSERT INTO Users (handle, email, password, name, registration_time,
last_online_time)
            VALUES (?, ?, ?, ?, GETDATE(), GETDATE())
        """, (handle, email, password, name))

        conn.commit()
        return jsonify({'success': True, 'message': '注册成功'})

    except Exception as e:
        return jsonify({'success': False, 'message': f'注册失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()

@app.route('/api/login', methods=['POST'])
def login():
    """用户登录"""
    data = request.json
    handle = data.get('handle')
    password = data.get('password')
```

```
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        cursor.execute("""
            SELECT user_id, handle, name, rating, user_rank, is_admin
            FROM Users
            WHERE handle = ? AND password = ? AND is_active = 1
        """, (handle, password))

        user = cursor.fetchone()
        if user:
            session['user_id'] = user[0]
            session['handle'] = user[1]
            session['name'] = user[2]
            session['rating'] = user[3]
            session['rank'] = user[4]
            session['is_admin'] = user[5]

            # 更新最后在线时间
            cursor.execute("UPDATE Users SET last_online_time = GETDATE() WHERE user_id
= ?", user[0])
            conn.commit()

            return jsonify({
                'success': True,
                'message': '登录成功',
                'user': {
                    'user_id': user[0],
                    'handle': user[1],
                    'name': user[2],
                    'rating': user[3],
                    'rank': user[4],
                    'is_admin': user[5]
                }
            })
        else:
            return jsonify({'success': False, 'message': '用户名或密码错误'})

    except Exception as e:
        return jsonify({'success': False, 'message': f'登录失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()

@app.route('/api/logout')
def logout():
    """用户登出"""
    session.clear()
    return jsonify({'success': True, 'message': '登出成功'})
```

```python
@app.route('/problem/<problem_id>')
def problem_detail(problem_id):
    """题目详情页面"""
    # 传递一些基本变量给模板，避免模板渲染错误
    return render_template('problem_detail.html',
                           problem={'title': '加载中...', 'difficulty': '简单'},
                           difficulty_color='secondary')

@app.route('/api/problems')
def get_problems():
    """获取题目列表"""
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        cursor.execute("""
            SELECT p.problem_id, p.title, p.difficulty, p.accepted_count,
p.submission_count,
                   p.time_limit_ms, p.memory_limit_kb, c.name as contest_name
            FROM PROBLEM p
            LEFT JOIN CONTEST c ON p.contest_id = c.contest_id
            WHERE p.is_visible = 1
            ORDER BY p.problem_id
        """)

        problems = []
        for row in cursor.fetchall():
            problems.append({
                'problem_id': row[0],
                'title': row[1],
                'difficulty': row[2],
                'accepted_count': row[3],
                'submission_count': row[4],
                'time_limit': row[5],
                'memory_limit': row[6],
                'contest_name': row[7]
            })

        return jsonify({'success': True, 'problems': problems})

    except Exception as e:
        return jsonify({'success': False, 'message': f'获取题目失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()


@app.route('/api/problem/<problem_id>')
def get_problem_detail(problem_id):
    """获取题目详情"""
```

```
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 获取题目基本信息
        cursor.execute("""
            SELECT p.problem_id, p.title, p.statement, p.input_specification,
                   p.output_specification, p.sample_tests, p.time_limit_ms,
                   p.memory_limit_kb, p.difficulty, p.accepted_count,
p.submission_count,
                   c.name as contest_name
            FROM PROBLEM p
            LEFT JOIN CONTEST c ON p.contest_id = c.contest_id
            WHERE p.problem_id = ? AND p.is_visible = 1
        """, (problem_id,))

        problem = cursor.fetchone()
        if not problem:
            return jsonify({'success': False, 'message': '题目不存在'})

        # 获取题目标签
        cursor.execute("""
            SELECT t.name
            FROM PROBLEM_TAG t
            INNER JOIN PROBLEM_TAG_RELATION r ON t.tag_id = r.tag_id
            WHERE r.problem_id = ?
        """, (problem_id,))

        tags = [row[0] for row in cursor.fetchall()]

        # 解析样例数据
        sample_tests = []
        if problem[5]:  # sample_tests 字段
            try:
                sample_tests = json.loads(problem[5])
            except:
                # 如果解析失败，使用默认样例
                sample_tests = [{"input": "1 2", "output": "3"}]

        problem_data = {
            'problem_id': problem[0],
            'title': problem[1],
            'statement': problem[2],
            'input_specification': problem[3],
            'output_specification': problem[4],
            'sample_tests': sample_tests,
            'time_limit': problem[6],
            'memory_limit': problem[7],
            'difficulty': problem[8],
            'accepted_count': problem[9],
            'submission_count': problem[10],
            'contest_name': problem[11],
            'tags': tags
```

```
        }

        return jsonify({'success': True, 'problem': problem_data})

    except Exception as e:
        return jsonify({'success': False, 'message': f'获取题目详情失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()

@app.route('/api/submit', methods=['POST'])
@app.route('/api/submit', methods=['POST'])
def submit_solution():
    """提交代码"""
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': '请先登录'})

    data = request.json
    problem_id = data.get('problem_id')
    source_code = data.get('source_code')
    language = data.get('language', 'C++')

    if not problem_id or not source_code:
        return jsonify({'success': False, 'message': '请填写完整信息'})

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 检查题目是否存在
        cursor.execute("SELECT problem_id FROM PROBLEM WHERE problem_id = ? AND
is_visible = 1", (problem_id,))
        if not cursor.fetchone():
            return jsonify({'success': False, 'message': '题目不存在'})

        # 插入提交记录
        cursor.execute("""
            INSERT INTO SUBMISSION (user_id, problem_id, programming_language,
source_code, source_length, submission_time)
            VALUES (?, ?, ?, ?, ?, GETDATE())
        """, (session['user_id'], problem_id, language, source_code, len(source_code)))

        submission_id = cursor.execute("SELECT @@IDENTITY").fetchone()[0]

        # 评测结果和权重
        verdicts = ['Accepted', 'Wrong Answer', 'Time Limit Exceeded', 'Memory Limit
Exceeded', 'Runtime Error',
                    'Compilation Error', 'Idleness Limit Exceeded', 'Presentation
Error', 'Partial Solution']
        weights = [35, 25, 8, 8, 8, 6, 4, 4, 2]  # 不同结果的权重
```

```
        verdict = random.choices(verdicts, weights=weights)[0]
        time_consumed = random.randint(0, 2000)
        memory_consumed = random.randint(1000, 256000)
        passed_tests = random.randint(0, 10)

        cursor.execute("""
            UPDATE SUBMISSION
            SET verdict = ?, time_consumed_ms = ?, memory_consumed_kb = ?,
passed_test_count = ?
            WHERE submission_id = ?
        """, (verdict, time_consumed, memory_consumed, passed_tests, submission_id))

        # 更新题目统计信息
        cursor.execute("UPDATE PROBLEM SET submission_count = submission_count + 1
WHERE problem_id = ?", (problem_id,))
        if verdict == 'Accepted':
            cursor.execute("UPDATE PROBLEM SET accepted_count = accepted_count + 1
WHERE problem_id = ?", (problem_id,))

        conn.commit()

        return jsonify({
            'success': True,
            'message': '提交成功',
            'submission_id': submission_id,
            'verdict': verdict
        })

    except Exception as e:
        return jsonify({'success': False, 'message': f'提交失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()

@app.route('/api/submissions')
def get_submissions():
    """获取提交记录"""
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        user_handle = request.args.get('user_handle')
        problem_title = request.args.get('problem_title')
        verdict = request.args.get('verdict')

        query = """
            SELECT s.submission_id, u.handle, p.title, s.programming_language,
                   s.verdict, s.time_consumed_ms, s.memory_consumed_kb,
                   s.passed_test_count, s.submission_time
            FROM SUBMISSION s
            INNER JOIN Users u ON s.user_id = u.user_id
```

```
            INNER JOIN PROBLEM p ON s.problem_id = p.problem_id
            WHERE 1=1
        """
        params = []

        if user_handle:
            query += " AND u.handle LIKE ?"
            params.append(f'%{user_handle}%')

        if problem_title:
            query += " AND p.title LIKE ?"
            params.append(f'%{problem_title}%')

        if verdict:
            query += " AND s.verdict = ?"
            params.append(verdict)

        query += " ORDER BY s.submission_time DESC"

        cursor.execute(query, params)

        submissions = []
        for row in cursor.fetchall():
            submissions.append({
                'submission_id': row[0],
                'handle': row[1],
                'problem_title': row[2],
                'language': row[3],
                'verdict': row[4],
                'time_consumed': row[5],
                'memory_consumed': row[6],
                'passed_tests': row[7],
                'submission_time': row[8].strftime('%Y-%m-%d %H:%M:%S')
            })

        return jsonify({'success': True, 'submissions': submissions})

    except Exception as e:
        return jsonify({'success': False, 'message': f'获取提交记录失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()

@app.route('/api/contests')
def get_contests():
    """获取比赛列表"""
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        cursor.execute("""
```

```
            SELECT contest_id, name, type, phase, start_time, duration_seconds,
                   difficulty, kind, icpc_region
            FROM CONTEST
            ORDER BY start_time DESC
        """)

        contests = []
        for row in cursor.fetchall():
            end_time = row[4] + timedelta(seconds=row[5])
            contests.append({
                'contest_id': row[0],
                'name': row[1],
                'type': row[2],
                'phase': row[3],
                'start_time': row[4].strftime('%Y-%m-%d %H:%M:%S'),
                'end_time': end_time.strftime('%Y-%m-%d %H:%M:%S'),
                'duration': row[5],
                'difficulty': row[6],
                'kind': row[7],
                'region': row[8]
            })

        return jsonify({'success': True, 'contests': contests})

    except Exception as e:
        return jsonify({'success': False, 'message': f'获取比赛列表失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()

@app.route('/api/users')
def get_users():
    """获取用户列表"""
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        cursor.execute("""
            SELECT user_id, handle, name, rating, max_rating, user_rank, max_rank,
                   country, city, organization, registration_time, is_admin
            FROM Users
            WHERE is_active = 1
            ORDER BY rating DESC
        """)

        users = []
        for row in cursor.fetchall():
            users.append({
                'user_id': row[0],
                'handle': row[1],
                'name': row[2],
```

```
                'rating': row[3],
                'max_rating': row[4],
                'rank': row[5],
                'max_rank': row[6],
                'country': row[7],
                'city': row[8],
                'organization': row[9],
                'registration_time': row[10].strftime('%Y-%m-%d %H:%M:%S'),
                'is_admin': row[11]  # 添加管理员信息
            })

        return jsonify({'success': True, 'users': users})

    except Exception as e:
        return jsonify({'success': False, 'message': f'获取用户列表失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()


@app.route('/api/contest/<contest_id>/standings')
def get_contest_standings(contest_id):
    """获取比赛排名"""
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 首先验证contest_id是否为数字
        if not contest_id.isdigit():
            return jsonify({'success': False, 'message': '比赛ID格式错误'})

        contest_id_int = int(contest_id)

        # 获取比赛信息
        cursor.execute("SELECT name, phase FROM CONTEST WHERE contest_id = ?",
(contest_id_int,))
        contest = cursor.fetchone()
        if not contest:
            return jsonify({'success': False, 'message': '比赛不存在'})

        contest_name, contest_phase = contest

        # 获取参赛用户及其排名
        cursor.execute("""
            SELECT
                u.handle,
                u.rating,
                cu.contest_rank,
                cu.solved_count,
                cu.total_penalty,
                cu.scores,
```

```
                cu.rating_before,
                cu.rating_after
            FROM CONTEST_USER cu
            INNER JOIN Users u ON cu.user_id = u.user_id
            WHERE cu.contest_id = ? AND cu.role = 'contestant'
            ORDER BY
                CASE
                    WHEN cu.contest_rank IS NOT NULL THEN cu.contest_rank
                    ELSE 999999
                END,
                cu.solved_count DESC,
                cu.total_penalty ASC
        """, (contest_id_int,))

        standings = []
        rank = 1
        for row in cursor.fetchall():
            handle, rating, contest_rank, solved_count, total_penalty, scores,
rating_before, rating_after = row

            # 计算Rating变化
            rating_change = None
            if rating_after is not None and rating_before is not None:
                rating_change = rating_after - rating_before

            standings.append({
                'handle': handle,
                'rating': rating,
                'contest_rank': contest_rank or rank,
                'solved_count': solved_count or 0,
                'penalty': total_penalty or 0,
                'score': scores or 0,
                'rating_change': rating_change
            })
            rank += 1

        return jsonify({
            'success': True,
            'standings': standings,
            'contest_name': contest_name,
            'contest_phase': contest_phase
        })

    except Exception as e:
        return jsonify({'success': False, 'message': f'获取比赛排名失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()

@app.route('/api/user/current')
def get_current_user():
```

```
    """获取当前登录用户信息"""
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': '用户未登录'})

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        cursor.execute("""
            SELECT user_id, handle, email, name, rating, max_rating, user_rank,
max_rank,
                   country, city, organization, avatar, registration_time,
last_online_time,
                   contribution, is_admin
            FROM Users
            WHERE user_id = ? AND is_active = 1
        """, (session['user_id'],))

        user = cursor.fetchone()
        if not user:
            return jsonify({'success': False, 'message': '用户不存在'})

        user_data = {
            'user_id': user[0],
            'handle': user[1],
            'email': user[2],
            'name': user[3],
            'rating': user[4],
            'max_rating': user[5],
            'rank': user[6],
            'max_rank': user[7],
            'country': user[8],
            'city': user[9],
            'organization': user[10],
            'avatar': user[11],
            'registration_time': user[12].strftime('%Y-%m-%d %H:%M:%S'),
            'last_online_time': user[13].strftime('%Y-%m-%d %H:%M:%S'),
            'contribution': user[14],
            'is_admin': user[15]
        }

        return jsonify({'success': True, 'user': user_data})

    except Exception as e:
        return jsonify({'success': False, 'message': f'获取用户详情失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()

@app.route('/api/user/<user_id>')
def get_user_detail(user_id):
```

```
    """获取用户详情"""
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        cursor.execute("""
            SELECT user_id, handle, email, name, rating, max_rating, user_rank,
max_rank,
                   country, city, organization, avatar, registration_time,
last_online_time,
                   contribution, is_admin
            FROM Users
            WHERE user_id = ? AND is_active = 1
        """, (user_id,))

        user = cursor.fetchone()
        if not user:
            return jsonify({'success': False, 'message': '用户不存在'})

        user_data = {
            'user_id': user[0],
            'handle': user[1],
            'email': user[2],
            'name': user[3],
            'rating': user[4],
            'max_rating': user[5],
            'rank': user[6],
            'max_rank': user[7],
            'country': user[8],
            'city': user[9],
            'organization': user[10],
            'avatar': user[11],
            'registration_time': user[12].strftime('%Y-%m-%d %H:%M:%S'),
            'last_online_time': user[13].strftime('%Y-%m-%d %H:%M:%S'),
            'contribution': user[14],
            'is_admin': user[15]
        }

        return jsonify({'success': True, 'user': user_data})

    except Exception as e:
        return jsonify({'success': False, 'message': f'获取用户详情失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()


@app.route('/api/user/update', methods=['POST'])
def update_user_profile():
    """更新用户个人信息"""
    if 'user_id' not in session:
```

```
        return jsonify({'success': False, 'message': '请先登录'})

    data = request.json
    user_id = session['user_id']

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 构建更新字段和参数
        update_fields = []
        params = []

        allowed_fields = ['name', 'country', 'city', 'organization', 'avatar']
        for field in allowed_fields:
            if field in data:
                update_fields.append(f"{field} = ?")
                params.append(data[field])

        # 如果没有要更新的字段
        if not update_fields:
            return jsonify({'success': False, 'message': '没有要更新的信息'})

        # 添加用户ID参数
        params.append(user_id)

        # 执行更新
        update_query = f"UPDATE Users SET {', '.join(update_fields)} WHERE user_id = ?"
        cursor.execute(update_query, params)

        # 更新session中的用户信息
        if 'name' in data:
            session['name'] = data['name']

        conn.commit()

        return jsonify({'success': True, 'message': '个人信息更新成功'})

    except Exception as e:
        return jsonify({'success': False, 'message': f'更新失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()


@app.route('/api/user/change_password', methods=['POST'])
def change_password():
    """修改密码"""
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': '请先登录'})
```

```
    data = request.json
    old_password = data.get('old_password')
    new_password = data.get('new_password')
    user_id = session['user_id']

    if not old_password or not new_password:
        return jsonify({'success': False, 'message': '请填写完整信息'})

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 验证旧密码
        cursor.execute("SELECT user_id FROM Users WHERE user_id = ? AND password = ?",
(user_id, old_password))
        if not cursor.fetchone():
            return jsonify({'success': False, 'message': '原密码错误'})

        # 更新密码
        cursor.execute("UPDATE Users SET password = ? WHERE user_id = ?",
(new_password, user_id))
        conn.commit()

        return jsonify({'success': True, 'message': '密码修改成功'})

    except Exception as e:
        return jsonify({'success': False, 'message': f'密码修改失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()


@app.route('/create-problem')
def create_problem_page():
    """题目创建页面"""
    return render_template('create_problem.html')


@app.route('/api/problems/create', methods=['POST'])
def create_problem():
    """创建题目"""
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': '请先登录'})

    # 检查是否是管理员
    if not session.get('is_admin'):
        return jsonify({'success': False, 'message': '只有管理员可以创建题目'})

    data = request.json

    required_fields = ['problem_id', 'title', 'statement', 'time_limit',
```

```
'memory_limit']
    for field in required_fields:
        if not data.get(field):
            return jsonify({'success': False, 'message': f'字段 {field} 不能为空'})

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 检查题目ID是否已存在
        cursor.execute("SELECT problem_id FROM PROBLEM WHERE problem_id = ?",
(data['problem_id'],))
        if cursor.fetchone():
            return jsonify({'success': False, 'message': '题目ID已存在'})

        # 插入题目基本信息
        cursor.execute("""
            INSERT INTO PROBLEM (
                problem_id, title, statement, input_specification,
output_specification,
                time_limit_ms, memory_limit_kb, difficulty, notes
            ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            data['problem_id'],
            data['title'],
            data.get('statement', ''),
            data.get('input_specification', ''),
            data.get('output_specification', ''),
            data['time_limit'],
            data['memory_limit'],
            data.get('difficulty', '简单'),
            data.get('notes', '')
        ))

        # 处理标签
        tags = data.get('tags', [])
        for tag_name in tags:
            # 检查标签是否存在
            cursor.execute("SELECT tag_id FROM PROBLEM_TAG WHERE name = ?",
(tag_name,))
            tag_row = cursor.fetchone()

            if tag_row:
                tag_id = tag_row[0]
            else:
                # 创建新标签
                cursor.execute("INSERT INTO PROBLEM_TAG (name) VALUES (?)",
(tag_name,))
                tag_id = cursor.execute("SELECT @@IDENTITY").fetchone()[0]

            # 建立题目标签关系
            cursor.execute(
                "INSERT INTO PROBLEM_TAG_RELATION (problem_id, tag_id) VALUES (?, ?)",
```

```
                (data['problem_id'], tag_id)
            )

        # 处理测试用例
        test_cases = data.get('test_cases', [])
        sample_tests = []

        for i, test_case in enumerate(test_cases):
            cursor.execute("""
                INSERT INTO TEST_CASE (problem_id, input_data, expected_output,
is_sample, test_order)
                VALUES (?, ?, ?, ?, ?)
            """, (
                data['problem_id'],
                test_case['input'],
                test_case['output'],
                test_case.get('is_sample', False),
                test_case.get('test_order', i + 1)
            ))

            # 收集样例测试用于存储到problem表
            if test_case.get('is_sample', False):
                sample_tests.append({
                    'input': test_case['input'],
                    'output': test_case['output']
                })

        # 更新题目的sample_tests字段
        if sample_tests:
            cursor.execute(
                "UPDATE PROBLEM SET sample_tests = ? WHERE problem_id = ?",
                (json.dumps(sample_tests), data['problem_id'])
            )

        conn.commit()
        return jsonify({'success': True, 'message': '题目创建成功'})

    except Exception as e:
        return jsonify({'success': False, 'message': f'创建题目失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()


@app.route('/api/user/set_admin', methods=['POST'])
def set_admin():
    """设置用户管理员权限"""
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': '请先登录'})

    # 检查当前用户是否是管理员
```

```
    if not session.get('is_admin'):
        return jsonify({'success': False, 'message': '只有管理员可以设置管理员权限'})

    data = request.json
    target_user_id = data.get('user_id')
    is_admin = data.get('is_admin', False)

    if not target_user_id:
        return jsonify({'success': False, 'message': '用户ID不能为空'})

    # 不能修改自己的管理员权限
    if target_user_id == session['user_id']:
        return jsonify({'success': False, 'message': '不能修改自己的管理员权限'})

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 检查目标用户是否存在
        cursor.execute("SELECT user_id FROM Users WHERE user_id = ? AND is_active = 1",
(target_user_id,))
        if not cursor.fetchone():
            return jsonify({'success': False, 'message': '用户不存在'})

        # 更新管理员权限
        cursor.execute("UPDATE Users SET is_admin = ? WHERE user_id = ?", (is_admin,
target_user_id))
        conn.commit()

        action = "设为" if is_admin else "取消"
        return jsonify({'success': True, 'message': f'用户已{action}管理员'})

    except Exception as e:
        return jsonify({'success': False, 'message': f'设置管理员权限失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()


@app.route('/create-contest')
def create_contest_page():
    """比赛创建页面"""
    return render_template('create_contest.html')


@app.route('/api/contests/create', methods=['POST'])
def create_contest():
    """创建比赛"""
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': '请先登录'})
```

```
    # 检查是否是管理员
    if not session.get('is_admin'):
        return jsonify({'success': False, 'message': '只有管理员可以创建比赛'})

    data = request.json

    required_fields = ['name', 'start_time', 'duration_seconds']
    for field in required_fields:
        if not data.get(field):
            return jsonify({'success': False, 'message': f'字段 {field} 不能为空'})

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 处理日期时间格式
        start_time_str = data['start_time']
        try:
            # 将前端传来的 datetime-local 格式转换为 SQL Server 可识别的格式
            # 前端格式: "YYYY-MM-DDTHH:MM"
            # 转换为: "YYYY-MM-DD HH:MM:SS"
            if 'T' in start_time_str:
                start_time_str = start_time_str.replace('T', ' ') + ':00'
        except Exception as e:
            return jsonify({'success': False, 'message': f'日期时间格式错误: {str(e)}'})

        # 插入比赛信息
        cursor.execute("""
            INSERT INTO CONTEST (
                name, type, phase, start_time, duration_seconds,
                description, difficulty, kind, icpc_region, country, city, season,
created_by
            ) VALUES (?, ?, ?, CONVERT(DATETIME2, ?), ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            data['name'],
            data.get('type', 'CF'),
            data.get('phase', 'BEFORE'),
            start_time_str,  # 使用转换后的日期时间字符串
            data['duration_seconds'],
            data.get('description', ''),
            data.get('difficulty', 0),
            data.get('kind', ''),
            data.get('icpc_region', ''),
            data.get('country', ''),
            data.get('city', ''),
            data.get('season', ''),
            session['user_id']
        ))

        contest_id = cursor.execute("SELECT @@IDENTITY").fetchone()[0]

        conn.commit()
        return jsonify({
```

```
            'success': True,
            'message': '比赛创建成功',
            'contest_id': contest_id
        })

    except Exception as e:
        return jsonify({'success': False, 'message': f'创建比赛失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()


@app.route('/contest/<contest_id>')
def contest_detail(contest_id):
    """比赛详情页面"""
    return render_template('contest_detail.html', contest_id=contest_id)


@app.route('/contest/<contest_id>/standings')
def contest_standings_page(contest_id):
    """比赛排名页面"""
    return render_template('contest_standings.html', contest_id=contest_id)


@app.route('/api/contest/<contest_id>')
def get_contest_detail(contest_id):
    """获取比赛详情"""
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        cursor.execute("""
            SELECT contest_id, name, type, phase, start_time, duration_seconds,
                   description, difficulty, kind, icpc_region, country, city, season
            FROM CONTEST
            WHERE contest_id = ?
        """, (contest_id,))

        contest = cursor.fetchone()
        if not contest:
            return jsonify({'success': False, 'message': '比赛不存在'})

        # 计算结束时间
        start_time = contest[4]
        duration = contest[5]
        end_time = start_time + timedelta(seconds=duration)

        contest_data = {
            'contest_id': contest[0],
            'name': contest[1],
            'type': contest[2],
```

```
            'phase': contest[3],
            'start_time': contest[4].strftime('%Y-%m-%d %H:%M:%S'),
            'end_time': end_time.strftime('%Y-%m-%d %H:%M:%S'),
            'duration': contest[5],
            'description': contest[6],
            'difficulty': contest[7],
            'kind': contest[8],
            'icpc_region': contest[9],
            'country': contest[10],
            'city': contest[11],
            'season': contest[12]
        }

        return jsonify({'success': True, 'contest': contest_data})

    except Exception as e:
        return jsonify({'success': False, 'message': f'获取比赛详情失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()


@app.route('/api/contest/<contest_id>/problems')
def get_contest_problems(contest_id):
    """获取比赛题目列表"""
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 验证contest_id
        if not contest_id.isdigit():
            return jsonify({'success': False, 'message': '比赛ID格式错误'})

        contest_id_int = int(contest_id)

        cursor.execute("""
            SELECT problem_id, title, difficulty, time_limit_ms, memory_limit_kb
            FROM PROBLEM
            WHERE contest_id = ? AND is_visible = 1
            ORDER BY problem_index
        """, (contest_id_int,))

        problems = []
        for row in cursor.fetchall():
            problems.append({
                'problem_id': row[0],
                'title': row[1],
                'difficulty': row[2],
                'time_limit': row[3],
                'memory_limit': row[4]
            })
```

```
        return jsonify({'success': True, 'problems': problems})

    except Exception as e:
        return jsonify({'success': False, 'message': f'获取比赛题目失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()

@app.route('/api/contest/<contest_id>/add_problem', methods=['POST'])
def add_problem_to_contest(contest_id):
    """添加题目到比赛"""
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': '请先登录'})

    # 检查是否是管理员
    if not session.get('is_admin'):
        return jsonify({'success': False, 'message': '只有管理员可以管理比赛题目'})

    data = request.json
    problem_id = data.get('problem_id')
    problem_index = data.get('problem_index', 'A')  # 默认题目索引为A

    if not problem_id:
        return jsonify({'success': False, 'message': '题目ID不能为空'})

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 检查比赛是否存在
        cursor.execute("SELECT contest_id FROM CONTEST WHERE contest_id = ?",
(contest_id,))
        if not cursor.fetchone():
            return jsonify({'success': False, 'message': '比赛不存在'})

        # 检查题目是否存在
        cursor.execute("SELECT problem_id, title FROM PROBLEM WHERE problem_id = ? AND
is_visible = 1", (problem_id,))
        problem = cursor.fetchone()
        if not problem:
            return jsonify({'success': False, 'message': '题目不存在'})

        # 检查题目是否已经在比赛中
        cursor.execute("""
            SELECT problem_id FROM PROBLEM
            WHERE contest_id = ? AND problem_id = ?
        """, (contest_id, problem_id))
        if cursor.fetchone():
            return jsonify({'success': False, 'message': '题目已在此比赛中'})
```

```
        # 更新题目的比赛信息
        cursor.execute("""
            UPDATE PROBLEM
            SET contest_id = ?, problem_index = ?
            WHERE problem_id = ?
        """, (contest_id, problem_index, problem_id))

        conn.commit()
        return jsonify({
            'success': True,
            'message': '题目添加成功',
            'problem': {
                'problem_id': problem_id,
                'title': problem[1],
                'index': problem_index
            }
        })

    except Exception as e:
        return jsonify({'success': False, 'message': f'添加题目失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()

@app.route('/api/contest/<contest_id>/remove_problem', methods=['POST'])
def remove_problem_from_contest(contest_id):
    """从比赛中移除题目"""
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': '请先登录'})

    # 检查是否是管理员
    if not session.get('is_admin'):
        return jsonify({'success': False, 'message': '只有管理员可以管理比赛题目'})

    data = request.json
    problem_id = data.get('problem_id')

    if not problem_id:
        return jsonify({'success': False, 'message': '题目ID不能为空'})

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 检查题目是否在比赛中
        cursor.execute("""
            SELECT problem_id FROM PROBLEM
            WHERE contest_id = ? AND problem_id = ?
        """, (contest_id, problem_id))
        if not cursor.fetchone():
            return jsonify({'success': False, 'message': '题目不在此比赛中'})
```

```
        # 移除题目的比赛关联
        cursor.execute("""
            UPDATE PROBLEM
            SET contest_id = NULL, problem_index = NULL
            WHERE problem_id = ?
        """, (problem_id,))

        conn.commit()
        return jsonify({'success': True, 'message': '题目移除成功'})

    except Exception as e:
        return jsonify({'success': False, 'message': f'移除题目失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()

@app.route('/api/contest/<contest_id>/available_problems')
def get_available_problems(contest_id):
    """获取可添加到比赛的题目列表（未在任何比赛中或仅在此比赛中的题目）"""
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': '请先登录'})

    # 检查是否是管理员
    if not session.get('is_admin'):
        return jsonify({'success': False, 'message': '只有管理员可以管理比赛题目'})

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 获取所有可见的题目，包括不在任何比赛中的和在此比赛中的
        cursor.execute("""
            SELECT problem_id, title, difficulty, time_limit_ms, memory_limit_kb,
                   contest_id, problem_index
            FROM PROBLEM
            WHERE is_visible = 1 AND (contest_id IS NULL OR contest_id = ?)
            ORDER BY
                CASE WHEN contest_id = ? THEN 0 ELSE 1 END,  -- 当前比赛中的题目排在前面
                problem_id
        """, (contest_id, contest_id))

        problems = []
        for row in cursor.fetchall():
            problems.append({
                'problem_id': row[0],
                'title': row[1],
                'difficulty': row[2],
                'time_limit': row[3],
                'memory_limit': row[4],
                'in_contest': row[5] == int(contest_id) if row[5] else False,
```

```
                'problem_index': row[6]
            })

        return jsonify({'success': True, 'problems': problems})

    except Exception as e:
        return jsonify({'success': False, 'message': f'获取题目列表失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()

@app.route('/api/contest/<contest_id>/register', methods=['POST'])
def register_contest(contest_id):
    """用户报名参加比赛"""
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': '请先登录'})

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 验证contest_id
        if not contest_id.isdigit():
            return jsonify({'success': False, 'message': '比赛ID格式错误'})

        contest_id_int = int(contest_id)

        # 检查比赛是否存在
        cursor.execute("SELECT contest_id FROM CONTEST WHERE contest_id = ?",
(contest_id_int,))
        if not cursor.fetchone():
            return jsonify({'success': False, 'message': '比赛不存在'})

        # 检查是否已经报名
        cursor.execute("""
            SELECT contest_id FROM CONTEST_USER
            WHERE contest_id = ? AND user_id = ?
        """, (contest_id_int, session['user_id']))

        if cursor.fetchone():
            return jsonify({'success': False, 'message': '已经报名参加此比赛'})

        # 获取用户当前rating
        cursor.execute("SELECT rating FROM Users WHERE user_id = ?",
(session['user_id'],))
        user_rating = cursor.fetchone()[0]

        # 插入报名记录
        cursor.execute("""
            INSERT INTO CONTEST_USER (contest_id, user_id, registration_time, role,
rating_before)
```

```
            VALUES (?, ?, GETDATE(), 'contestant', ?)
        """, (contest_id_int, session['user_id'], user_rating))

        conn.commit()
        return jsonify({'success': True, 'message': '报名成功'})

    except Exception as e:
        return jsonify({'success': False, 'message': f'报名失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()

@app.route('/api/contest/<contest_id>/generate_standings', methods=['POST'])
def generate_contest_standings(contest_id):
    """生成比赛排名（管理员功能）"""
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': '请先登录'})

    if not session.get('is_admin'):
        return jsonify({'success': False, 'message': '只有管理员可以生成排名'})

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # 验证contest_id
        if not contest_id.isdigit():
            return jsonify({'success': False, 'message': '比赛ID格式错误'})

        contest_id_int = int(contest_id)

        # 检查比赛是否存在
        cursor.execute("SELECT contest_id, start_time FROM CONTEST WHERE contest_id =
?", (contest_id_int,))
        contest = cursor.fetchone()
        if not contest:
            return jsonify({'success': False, 'message': '比赛不存在'})

        contest_id_db, start_time = contest

        # 获取比赛中的提交记录
        cursor.execute("""
            SELECT
                s.user_id,
                s.problem_id,
                s.verdict,
                s.submission_time,
                s.points
            FROM SUBMISSION s
            WHERE s.contest_id = ?
            ORDER BY s.user_id, s.problem_id, s.submission_time
```

```
        """, (contest_id_int,))

        submissions = cursor.fetchall()

        # 获取参赛用户
        cursor.execute("""
            SELECT user_id FROM CONTEST_USER
            WHERE contest_id = ? AND role = 'contestant'
        """, (contest_id_int,))

        contestants = [row[0] for row in cursor.fetchall()]

        if not contestants:
            return jsonify({'success': False, 'message': '没有参赛用户'})

        # 计算每个用户的解题情况
        user_stats = {}
        for user_id in contestants:
            user_stats[user_id] = {
                'solved_count': 0,
                'total_penalty': 0,
                'total_score': 0,
                'problems': {}
            }

        # 处理提交记录
        for submission in submissions:
            user_id, problem_id, verdict, submission_time, points = submission

            if user_id not in user_stats:
                continue

            if problem_id not in user_stats[user_id]['problems']:
                user_stats[user_id]['problems'][problem_id] = {
                    'solved': False,
                    'penalty': 0,
                    'submissions': 0,
                    'score': 0,
                    'solve_time': None
                }

            problem_stats = user_stats[user_id]['problems'][problem_id]

            if not problem_stats['solved']:
                problem_stats['submissions'] += 1

                if verdict == 'Accepted':
                    problem_stats['solved'] = True
                    # 计算解题时间（从比赛开始算起的分钟数）
                    solve_minutes = (submission_time - start_time).total_seconds() / 60
                    problem_stats['solve_time'] = solve_minutes
                    problem_stats['penalty'] = solve_minutes +
(problem_stats['submissions'] - 1) * 20  # 每错误提交加20分钟罚时
```

```
                    user_stats[user_id]['solved_count'] += 1
                    user_stats[user_id]['total_penalty'] += problem_stats['penalty']

                # 处理得分制比赛
                if points and points > problem_stats['score']:
                    problem_stats['score'] = points

        # 计算总分（对于得分制比赛）
        for user_id in user_stats:
            total_score = sum(problem['score'] for problem in user_stats[user_id]
['problems'].values())
            user_stats[user_id]['total_score'] = total_score

        # 排序用户（按解题数降序，罚时升序）
        sorted_users = sorted(contestants, key=lambda uid: (
            -user_stats[uid]['solved_count'],
            user_stats[uid]['total_penalty']
        ))

        # 更新数据库中的排名
        current_rank = 1
        for user_id in sorted_users:
            stats = user_stats[user_id]
            cursor.execute("""
                UPDATE CONTEST_USER
                SET contest_rank = ?, solved_count = ?, total_penalty = ?, scores = ?
                WHERE contest_id = ? AND user_id = ?
            """, (current_rank, stats['solved_count'], int(stats['total_penalty']),
                  stats['total_score'], contest_id_int, user_id))
            current_rank += 1

        conn.commit()
        return jsonify({'success': True, 'message': '排名生成成功'})

    except Exception as e:
        return jsonify({'success': False, 'message': f'生成排名失败: {str(e)}'})
    finally:
        if 'cursor' in locals():
            cursor.close()
        if 'conn' in locals():
            conn.close()

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

codeforces_crawlerpy

to top

```python
# codeforces_crawler_fixed.py
import pyodbc
import requests
import json
import random
import string
import time
from datetime import datetime, timedelta


class CodeforcesCrawler:
    def __init__(self):
        self.base_url = "https://codeforces.com/api/"

    def get_users(self, count=100):
        """获取用户数据"""
        try:
            print("正在获取用户数据...")
            response = requests.get(f"{self.base_url}user.ratedList?activeOnly=true")
            if response.status_code == 200:
                data = response.json()
                if data['status'] == 'OK':
                    users = data['result'][:count]
                    print(f"成功获取 {len(users)} 个用户")
                    return users
            print("获取用户数据失败")
            return []
        except Exception as e:
            print(f"获取用户数据错误: {e}")
            return []

    def get_contests(self):
        """获取比赛数据"""
        try:
            print("正在获取比赛数据...")
            response = requests.get(f"{self.base_url}contest.list")
            if response.status_code == 200:
                data = response.json()
                if data['status'] == 'OK':
                    contests = data['result']
                    print(f"成功获取 {len(contests)} 个比赛")
                    return contests
            print("获取比赛数据失败")
            return []
        except Exception as e:
            print(f"获取比赛数据错误: {e}")
            return []

    def get_problems(self):
        """获取题目数据"""
        try:
            print("正在获取题目数据...")
```

```
            response = requests.get(f"{self.base_url}problemset.problems")
            if response.status_code == 200:
                data = response.json()
                if data['status'] == 'OK':
                    problems = data['result']['problems']
                    stats = data['result']['problemStatistics']
                    print(f"成功获取 {len(problems)} 个题目")
                    return problems, stats
            print("获取题目数据失败")
            return [], []
        except Exception as e:
            print(f"获取题目数据错误: {e}")
            return [], []

    def get_user_submissions(self, handle, count=50):
        """获取用户提交记录"""
        try:
            print(f"正在获取用户 {handle} 的提交记录...")
            response = requests.get(f"{self.base_url}user.status?handle={handle}&count=
{count}")
            if response.status_code == 200:
                data = response.json()
                if data['status'] == 'OK':
                    submissions = data['result']
                    print(f"成功获取用户 {handle} 的 {len(submissions)} 条提交记录")
                    return submissions
            print(f"获取用户 {handle} 提交记录失败")
            return []
        except Exception as e:
            print(f"获取用户 {handle} 提交记录错误: {e}")
            return []


class DatabaseManager:
    def __init__(self):
        self.conn = self.get_db_connection()
        self.cursor = self.conn.cursor()
        self.existing_tags_cache = {}  # 缓存已存在的标签

    def get_db_connection(self):
        """使用 Windows 身份验证连接 SQL Server"""
        try:
            # Windows 身份验证连接字符串
            connection_string = (
                'DRIVER={ODBC Driver 17 for SQL Server};'
                'SERVER=localhost;'  # 修改为您的服务器地址
                'DATABASE=OJ;'
                'Trusted_Connection=yes;'
            )
            conn = pyodbc.connect(connection_string)
            print("成功连接到数据库")
            return conn
        except Exception as e:
```

```
            print(f"数据库连接错误: {e}")
            raise

    def generate_random_password(self, length=12):
        """生成随机密码"""
        characters = string.ascii_letters + string.digits + string.punctuation
        return ''.join(random.choice(characters) for _ in range(length))

    def calculate_rank(self, rating):
        """根据评分计算等级"""
        if rating is None or rating < 1200:
            return "Newbie"
        elif rating < 1400:
            return "Pupil"
        elif rating < 1600:
            return "Specialist"
        elif rating < 1900:
            return "Expert"
        elif rating < 2100:
            return "Candidate Master"
        elif rating < 2300:
            return "Master"
        elif rating < 2400:
            return "International Master"
        elif rating < 2600:
            return "Grandmaster"
        elif rating < 3000:
            return "International Grandmaster"
        else:
            return "Legendary Grandmaster"

    def insert_users(self, users_data):
        """插入用户数据"""
        inserted_count = 0
        print("开始插入用户数据...")

        for user in users_data:
            try:
                # 检查用户是否已存在
                self.cursor.execute("SELECT user_id FROM Users WHERE handle = ?",
user['handle'])
                if self.cursor.fetchone():
                    continue

                password = self.generate_random_password()
                rating = user.get('rating', 0)
                max_rating = user.get('maxRating', 0)

                # 计算用户等级
                user_rank = self.calculate_rank(rating)
                max_rank = self.calculate_rank(max_rating)

                # 处理可能为空的字段
```

```
                first_name = user.get('firstName', '')
                last_name = user.get('lastName', '')
                full_name = f"{first_name} {last_name}".strip()
                if not full_name:
                    full_name = user['handle']

                registration_time =
datetime.fromtimestamp(user.get('registrationTimeSeconds', time.time()))
                last_online_time =
datetime.fromtimestamp(user.get('lastOnlineTimeSeconds', time.time()))

                self.cursor.execute("""
                    INSERT INTO Users (handle, email, password, name, rating,
max_rating,
                                       user_rank, max_rank, country, city, organization,
                                       registration_time, last_online_time, contribution)
                    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                """,
                                       user['handle'],
                                       f"{user['handle']}@codeforces.com",
                                       password,
                                       full_name,
                                       rating,
                                       max_rating,
                                       user_rank,
                                       max_rank,
                                       user.get('country', ''),
                                       user.get('city', ''),
                                       user.get('organization', ''),
                                       registration_time,
                                       last_online_time,
                                       user.get('contribution', 0)
                                       )
                inserted_count += 1

            except Exception as e:
                print(f"插入用户 {user['handle']} 错误: {e}")
                continue

        self.conn.commit()
        print(f"成功插入 {inserted_count} 个用户")
        return inserted_count

    def insert_contests(self, contests_data):
        """插入比赛数据"""
        inserted_count = 0
        print("开始插入比赛数据...")

        # 先获取一个管理员用户ID作为创建者
        self.cursor.execute("SELECT TOP 1 user_id FROM Users WHERE is_admin = 1 OR
user_id = 1")
        admin_user = self.cursor.fetchone()
        created_by = admin_user[0] if admin_user else 1
```

```
        for contest in contests_data:
            try:
                # 检查比赛是否已存在
                self.cursor.execute("SELECT contest_id FROM CONTEST WHERE name = ?",
contest['name'])
                if self.cursor.fetchone():
                    continue

                # 处理比赛阶段
                phase = contest.get('phase', 'FINISHED')
                if phase == 'BEFORE':
                    phase = 'BEFORE'
                elif phase == 'FINISHED':
                    phase = 'FINISHED'
                else:
                    phase = 'CODING'

                start_time = datetime.fromtimestamp(contest.get('startTimeSeconds',
time.time()))

                self.cursor.execute("""
                    INSERT INTO CONTEST (name, type, phase, frozen, start_time,
                                         duration_seconds, website_url, description,
created_by)
                    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                """,
                                    contest['name'],
                                    contest.get('type', 'CF'),
                                    phase,
                                    contest.get('frozen', False),
                                    start_time,
                                    contest.get('durationSeconds', 7200),  # 默认2小时
                                    f"https://codeforces.com/contest/{contest['id']}",
                                    contest.get('description', f"Codeforces Contest
{contest['id']}"),
                                    created_by
                                    )
                inserted_count += 1

            except Exception as e:
                print(f"插入比赛 {contest['name']} 错误: {e}")
                continue

        self.conn.commit()
        print(f"成功插入 {inserted_count} 个比赛")
        return inserted_count

    def load_existing_tags(self):
        """加载已存在的标签到缓存"""
        try:
            self.cursor.execute("SELECT tag_id, name FROM PROBLEM_TAG")
            for row in self.cursor.fetchall():
```

```
                self.existing_tags_cache[row[1]] = row[0]
            print(f"已加载 {len(self.existing_tags_cache)} 个标签到缓存")
        except Exception as e:
            print(f"加载标签缓存错误: {e}")

    def insert_problems(self, problems_data, problem_stats):
        """插入题目数据"""
        inserted_count = 0
        print("开始插入题目数据...")

        # 加载已存在的标签
        self.load_existing_tags()

        # 首先获取所有比赛ID映射
        contest_map = {}
        try:
            self.cursor.execute("SELECT contest_id, name FROM CONTEST")
            for row in self.cursor.fetchall():
                contest_name = row[1]
                # 尝试从比赛名称中提取数字ID
                import re
                numbers = re.findall(r'\d+', contest_name)
                if numbers:
                    contest_map[int(numbers[-1])] = row[0]
        except Exception as e:
            print(f"获取比赛映射错误: {e}")

        # 用于记录已处理的标签关系，避免重复
        processed_tag_relations = set()

        for i, problem in enumerate(problems_data):
            try:
                problem_id = f"{problem['contestId']}{problem['index']}"

                # 检查题目是否已存在
                self.cursor.execute("SELECT problem_id FROM PROBLEM WHERE problem_id =
?", problem_id)
                if self.cursor.fetchone():
                    continue

                # 获取对应的统计信息
                stats = next((s for s in problem_stats if
                              s['contestId'] == problem['contestId'] and s['index'] ==
problem['index']), {})

                contest_id = contest_map.get(problem.get('contestId'), None)

                self.cursor.execute("""
                    INSERT INTO PROBLEM (problem_id, contest_id, problem_index, title,
                                         time_limit_ms, memory_limit_kb, difficulty,
                                         accepted_count, submission_count)
                    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                """,
```

```
                                    problem_id,
                                    contest_id,
                                    problem['index'],
                                    problem['name'],
                                    problem.get('timeLimitMillis', 2000),
                                    problem.get('memoryLimitBytes', 256000) // 1024,
                                    problem.get('rating', 0),
                                    stats.get('solvedCount', 0),
                                    stats.get('solvedCount', 0) + random.randint(0,
100)
                                    )
                inserted_count += 1

                # 插入标签（使用缓存和去重）
                for tag in problem.get('tags', []):
                    relation_key = (problem_id, tag)
                    if relation_key not in processed_tag_relations:
                        self.insert_problem_tag_safe(problem_id, tag)
                        processed_tag_relations.add(relation_key)

            except Exception as e:
                print(f"插入题目 {problem.get('name', 'Unknown')} 错误: {e}")
                continue

        self.conn.commit()
        print(f"成功插入 {inserted_count} 个题目")
        return inserted_count

    def insert_problem_tag_safe(self, problem_id, tag_name):
        """安全插入题目标签（避免重复）"""
        try:
            # 检查标签是否在缓存中
            if tag_name in self.existing_tags_cache:
                tag_id = self.existing_tags_cache[tag_name]
            else:
                # 插入新标签
                self.cursor.execute("INSERT INTO PROBLEM_TAG (name) VALUES (?)",
tag_name)
                self.cursor.execute("SELECT @@IDENTITY")
                tag_id = self.cursor.fetchone()[0]
                self.existing_tags_cache[tag_name] = tag_id

            # 检查标签关系是否已存在
            self.cursor.execute("""
                SELECT 1 FROM PROBLEM_TAG_RELATION
                WHERE problem_id = ? AND tag_id = ?
            """, problem_id, tag_id)

            if not self.cursor.fetchone():
                # 插入标签关系
                self.cursor.execute("""
                    INSERT INTO PROBLEM_TAG_RELATION (problem_id, tag_id)
                    VALUES (?, ?)
```

```
                """, problem_id, tag_id)

            except Exception as e:
                # 如果是重复键错误，忽略它
                if 'PRIMARY KEY' in str(e) or '2627' in str(e):
                    pass  # 忽略重复键错误
                else:
                    print(f"插入标签错误: {e}")

    def insert_submissions(self, crawler):
        """插入提交记录"""
        inserted_count = 0
        print("开始插入提交记录...")

        # 获取用户列表
        self.cursor.execute("SELECT user_id, handle FROM Users")
        users = self.cursor.fetchall()

        # 获取有效的比赛ID集合
        valid_contest_ids = set()
        self.cursor.execute("SELECT contest_id FROM CONTEST")
        for row in self.cursor.fetchall():
            valid_contest_ids.add(row[0])

        for user_id, handle in users:
            try:
                submissions = crawler.get_user_submissions(handle, 10)  # 每个用户获取10
条记录

                for submission in submissions:
                    try:
                        # 检查提交记录中是否包含必要的字段
                        if 'problem' not in submission or 'contestId' not in
submission['problem']:
                            continue

                        problem_id = f"{submission['problem']['contestId']}
{submission['problem']['index']}"

                        # 检查题目是否存在
                        self.cursor.execute("SELECT problem_id FROM PROBLEM WHERE
problem_id = ?", problem_id)
                        if not self.cursor.fetchone():
                            continue

                        # 处理比赛ID：如果不在有效比赛ID中，设为NULL
                        contest_id = submission.get('contestId')
                        if contest_id is None or contest_id not in valid_contest_ids:
                            contest_id = None

                        # 映射判决结果到新的格式
                        verdict_map = {
                            'OK': 'Accepted',
```

```
                            'WRONG_ANSWER': 'Wrong Answer',
                            'TIME_LIMIT_EXCEEDED': 'Time Limit Exceeded',
                            'MEMORY_LIMIT_EXCEEDED': 'Memory Limit Exceeded',
                            'RUNTIME_ERROR': 'Runtime Error',
                            'COMPILATION_ERROR': 'Compilation Error',
                            'PRESENTATION_ERROR': 'Presentation Error',
                            'FAILED': 'Failed',
                            'PARTIAL': 'Partial Solution',
                            'CHALLENGED': 'Challenged',
                            'SKIPPED': 'Skipped',
                            'REJECTED': 'Rejected',
                            'IDLENESS_LIMIT_EXCEEDED': 'Idleness Limit Exceeded'
                        }

                        verdict = verdict_map.get(submission.get('verdict', ''),
'Pending')

                        # 处理可能缺失的字段
                        time_consumed_ms = submission.get('timeConsumedMillis', 0)
                        memory_consumed_bytes = submission.get('memoryConsumedBytes',
0)
                        memory_consumed_kb = memory_consumed_bytes // 1024 if
memory_consumed_bytes else 0
                        passed_test_count = submission.get('passedTestCount', 0)
                        creation_time = submission.get('creationTimeSeconds',
time.time())
                        relative_time = submission.get('relativeTimeSeconds', 0)

                        self.cursor.execute("""
                            INSERT INTO SUBMISSION (user_id, problem_id, contest_id,
                                                    programming_language, source_code,
                                                    source_length, verdict,
time_consumed_ms,
                                                    memory_consumed_kb, passed_test_count,
                                                    submission_time, relative_time)
                            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                        """,
                                        user_id,
                                        problem_id,
                                        contest_id,  # 可能为NULL
                                        submission.get('programmingLanguage',
'Unknown'),
                                        f"// Source code for {problem_id} by
{handle}",
                                        len(f"// Source code for {problem_id} by
{handle}"),
                                        verdict,
                                        time_consumed_ms,
                                        memory_consumed_kb,
                                        passed_test_count,
                                        datetime.fromtimestamp(creation_time),
                                        relative_time
                                        )
```

```
                        inserted_count += 1

                    except Exception as e:
                        # 如果是外键约束错误，忽略它
                        if 'FOREIGN KEY' in str(e) or '547' in str(e):
                            continue
                        else:
                            print(f"插入提交记录错误: {e}")
                        continue

            except Exception as e:
                print(f"获取用户 {handle} 提交记录错误: {e}")
                continue

        self.conn.commit()
        print(f"成功插入 {inserted_count} 个提交记录")
        return inserted_count

    def insert_contest_users(self, crawler):
        """插入比赛用户关系数据"""
        inserted_count = 0
        print("开始插入比赛用户关系数据...")

        # 获取用户列表
        self.cursor.execute("SELECT user_id, handle FROM Users")
        users = self.cursor.fetchall()

        if not users:
            print("没有找到用户数据，跳过比赛用户关系插入")
            return 0

        # 获取比赛列表
        self.cursor.execute("SELECT contest_id, name FROM CONTEST")
        contests = self.cursor.fetchall()

        if not contests:
            print("没有找到比赛数据，跳过比赛用户关系插入")
            return 0

        # 为每个比赛随机分配一些参与者
        for contest_id, contest_name in contests:
            try:
                # 确保不会选择超过用户总数的参与者
                participant_count = min(random.randint(5, 20), len(users))
                participants = random.sample(users, participant_count)

                for user_id, handle in participants:
                    try:
                        # 检查是否已存在该关系
                        self.cursor.execute("""
                            SELECT 1 FROM CONTEST_USER
                            WHERE contest_id = ? AND user_id = ?
                        """, contest_id, user_id)
```

```
                        if self.cursor.fetchone():
                            continue

                        # 随机分配角色（大部分为参赛者）
                        roles = ['contestant'] * 8 + ['virtual'] * 1 +
['out_of_competition'] * 1
                        role = random.choice(roles)

                        # 如果是前几个比赛，添加一些作者和测试者
                        if contest_id <= 5 and participants.index((user_id, handle)) <
3:
                            if random.random() < 0.3:
                                role = 'author'
                            elif random.random() < 0.5:
                                role = 'tester'

                        # 获取用户当前评分
                        self.cursor.execute("SELECT rating FROM Users WHERE user_id =
?", user_id)
                        user_rating_result = self.cursor.fetchone()
                        rating_before = user_rating_result[0] if user_rating_result
else 0

                        # 模拟比赛后的评分变化（±0-200）
                        rating_change = random.randint(-50, 150)
                        rating_after = max(0, rating_before + rating_change)

                        # 随机生成比赛排名
                        contest_rank = random.randint(1, len(participants))

                        # 根据排名生成解题数量和罚时
                        max_solved = min(10, contest_rank // 2 + 1)
                        solved_count = random.randint(0, max_solved)

                        # 罚时与解题数量和排名相关
                        base_penalty = solved_count * 1200  # 每道题基础罚时
                        rank_penalty = contest_rank * 10    # 排名相关的额外罚时
                        total_penalty = base_penalty + random.randint(-300, 300) +
rank_penalty

                        # 计算得分（与解题数量和排名相关）
                        scores = solved_count * 100 + max(0, 500 - contest_rank * 10)

                        # 使用 timedelta（需要导入）
                        registration_time = datetime.now() -
timedelta(days=random.randint(1, 365))

                        self.cursor.execute("""
                            INSERT INTO CONTEST_USER (contest_id, user_id,
registration_time,
                                                      role, rating_before, rating_after,
                                                      contest_rank, solved_count,
```

```
total_penalty, scores)
                            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                        """,
                                        contest_id,
                                        user_id,
                                        registration_time,
                                        role,
                                        rating_before,
                                        rating_after,
                                        contest_rank,
                                        solved_count,
                                        int(total_penalty),
                                        scores
                                        )
                        inserted_count += 1

                        # 如果比赛已完成，更新用户的评分
                        if role == 'contestant' and rating_after != rating_before:
                            self.cursor.execute("""
                                UPDATE Users
                                SET rating = ?,
                                    max_rating = CASE WHEN ? > max_rating THEN ? ELSE
max_rating END,
                                    user_rank = ?,
                                    max_rank = CASE WHEN ? > max_rating THEN ? ELSE
max_rank END
                                WHERE user_id = ?
                            """,
                                                rating_after,
                                                rating_after, rating_after,
                                                self.calculate_rank(rating_after),
                                                rating_after,
self.calculate_rank(rating_after),
                                                user_id)

                    except Exception as e:
                        print(f"插入用户 {handle} 到比赛 {contest_name} 错误: {e}")
                        continue

            except Exception as e:
                print(f"处理比赛 {contest_name} 用户关系错误: {e}")
                continue

        self.conn.commit()
        print(f"成功插入 {inserted_count} 个比赛用户关系")
        return inserted_count



    def get_user_contest_history(self, user_id):
        """获取用户比赛历史"""
        try:
            self.cursor.execute("""
```

```
                SELECT c.name, c.start_time, cu.contest_rank, cu.solved_count,
                    cu.rating_before, cu.rating_after, cu.role
                FROM CONTEST_USER cu
                JOIN CONTEST c ON cu.contest_id = c.contest_id
                WHERE cu.user_id = ?
                ORDER BY c.start_time DESC
            """, user_id)

            history = self.cursor.fetchall()
            return history
        except Exception as e:
            print(f"获取用户比赛历史错误: {e}")
            return []

    def get_contest_standings(self, contest_id):
        """获取比赛排名"""
        try:
            self.cursor.execute("""
                SELECT
                    u.handle,
                    COALESCE(u.name, u.handle) as display_name,  -- 如果name为NULL，使用
handle
                    cu.contest_rank,
                    cu.solved_count,
                    cu.total_penalty,
                    cu.scores,
                    cu.rating_before,
                    cu.rating_after,
                    cu.role
                FROM CONTEST_USER cu
                JOIN Users u ON cu.user_id = u.user_id
                WHERE cu.contest_id = ?
                ORDER BY cu.contest_rank ASC
            """, contest_id)

            standings = self.cursor.fetchall()
            return standings
        except Exception as e:
            print(f"获取比赛排名错误: {e}")
            return []

    def display_contest_standings(self, contest_id):
        """显示比赛排名（更好的格式化输出）"""
        standings = self.get_contest_standings(contest_id)
        if not standings:
            print(f"比赛 {contest_id} 没有找到排名数据")
            return

        print(f"\n=== 比赛 {contest_id} 排名 ===")
        print(f"{'排名':<4} {'用户名':<20} {'显示名':<20} {'解题数':<6} {'罚时':<8} {'得
分':<6} {'角色':<15}")
        print("-" * 85)
```

```
        for standing in standings[:10]:  # 只显示前10名
            handle, display_name, rank, solved, penalty, score, rating_before,
rating_after, role = standing

            # 处理可能的None值
            handle = handle or "Unknown"
            display_name = display_name or handle
            solved = solved or 0
            penalty = penalty or 0
            score = score or 0
            role = role or "contestant"

            print(f"{rank:<4} {handle:<20} {display_name:<20} {solved:<6} {penalty:<8}
{score:<6} {role:<15}")


    def close(self):
        """关闭数据库连接"""
        if self.conn:
            self.conn.close()
            print("数据库连接已关闭")


def main():
    print("=== Codeforces 数据爬取程序开始 ===")

    crawler = CodeforcesCrawler()
    db_manager = DatabaseManager()

    try:
        # 1. 爬取并插入用户数据
        users = crawler.get_users(20000)
        if users:
            db_manager.insert_users(users)

        # 2. 爬取并插入比赛数据
        contests = crawler.get_contests()
        if contests:
            db_manager.insert_contests(contests[:5000])  # 减少比赛数量以便测试

        # 3. 爬取并插入题目数据
        problems, stats = crawler.get_problems()
        if problems:
            db_manager.insert_problems(problems[:10000], stats)  # 减少题目数量以便测试

        # 4. 插入比赛用户关系
        db_manager.insert_contest_users(crawler)

        # 5. 插入提交记录
        # db_manager.insert_submissions(crawler)

        print("=== 数据爬取和插入完成 ===")
```

```
        # 显示统计信息
        print("\n=== 统计信息 ===")

        # 检查数据是否成功插入
        db_manager.cursor.execute("SELECT COUNT(*) FROM Users")
        user_count = db_manager.cursor.fetchone()[0]
        print(f"总用户数: {user_count}")

        db_manager.cursor.execute("SELECT COUNT(*) FROM CONTEST")
        contest_count = db_manager.cursor.fetchall()[0]
        print(f"总比赛数: {contest_count}")

        db_manager.cursor.execute("SELECT COUNT(*) FROM CONTEST_USER")
        contest_user_count = db_manager.cursor.fetchone()[0]
        print(f"总比赛参与记录: {contest_user_count}")

        # 获取第一个比赛并显示排名
        db_manager.cursor.execute("SELECT TOP 1 contest_id, name FROM CONTEST")
        first_contest = db_manager.cursor.fetchone()
        if first_contest:
            contest_id, contest_name = first_contest
            print(f"\n显示比赛 '{contest_name}' (ID: {contest_id}) 的排名:")
            db_manager.display_contest_standings(contest_id)
        else:
            print("没有找到比赛数据")

    except Exception as e:
        print(f"程序执行过程中发生错误: {e}")
        import traceback
        traceback.print_exc()
    finally:
        db_manager.close()


if __name__ == "__main__":
    main()
```

configpy

to top

```
# config.py
# Windows 身份验证配置
DB_SERVER = 'localhost'  # 或您的SQL Server实例名，如 'localhost\\SQLEXPRESS'
DB_NAME = 'OJ'
# 不需要用户名和密码，使用Windows身份验证
```

docs\前端开发手册md

to top

前端开发手册（HTML + CSS + JS）

项目结构

```
Database-Principles-Major-Assignment/
├── templates/              # HTML 模板文件
│   ├── index.html          # 首页 - 系统概览和导航
│   ├── index_loger.html    # 登录页面
│   ├── problems.html       # 题目列表页面
│   ├── problem_detail.html # 题目详情页面
│   ├── contests.html       # 比赛列表页面
│   ├── contest_detail.html # 比赛详情页面
│   ├── contest_standings.html # 比赛排名页面
│   ├── submissions.html    # 提交记录页面
│   ├── users.html          # 用户列表页面
│   ├── profile.html        # 用户个人资料页面
│   ├── create_problem.html # 题目创建页面（管理员）
│   └── create_contest.html # 比赛创建页面（管理员）
└── static/                 # 静态资源文件
    ├── css/                # 样式文件（待扩展）
    ├── js/                 # JavaScript 文件（待扩展）
    ├── images/             # 图片资源（待扩展）
    └── fonts/              # 字体文件（待扩展）
```

技术架构

1. 前端技术栈

- **HTML5**: 语义化标签，响应式布局
- **CSS3**: Bootstrap 5 框架，自定义样式
- **JavaScript**: 原生 ES6+，Fetch API
- **模板引擎**: Flask Jinja2 模板

2. 设计原则

- 响应式设计，支持多设备访问
- 模块化组件，便于维护和扩展
- 渐进式增强，确保基础功能可用
- 无障碍访问，遵循 WCAG 标准

页面功能详解

1. 首页 (index.html)

- 系统概览和功能介绍
- 快速导航到主要功能模块
- 显示系统统计信息（题目数、用户数等）

2. 登录页面 (index_loger.html)

- 用户登录表单
- 注册链接
- 表单验证和错误提示

3. 题目相关页面

题目列表 (problems.html)

- 题目列表展示（分页）
- 搜索和筛选功能
- 难度等级颜色标识
- 题目统计信息显示

题目详情 (problem_detail.html)

- 完整题目内容展示
- 样例测试用例
- 代码编辑器（Monaco Editor）
- 提交代码表单

4. 比赛相关页面

比赛列表 (contests.html)

- 比赛列表和时间线
- 比赛状态标识（进行中/已结束/未开始）
- 快速参与比赛

比赛详情 (contest_detail.html)

- 比赛完整信息
- 比赛题目列表
- 比赛规则说明
- 参与比赛功能

比赛排名 (contest_standings.html)

- 实时排名更新
- 用户解题情况
- 分数和罚分统计

5. 用户相关页面

用户列表 (users.html)

- 用户排名展示
- 用户信息概览
- 评分变化趋势

个人资料 (profile.html)

- 个人信息展示和编辑
- 提交记录统计
- 比赛参与历史

6. 提交记录 (submissions.html)

- 提交历史列表
- 多维度筛选（用户、题目、结果）
- 详细评测信息

7. 管理页面

题目创建 (create_problem.html)

- 题目基本信息表单
- 题目内容编辑器
- 测试用例管理
- 标签设置

比赛创建 (create_contest.html)

- 比赛基本信息
- 时间设置
- 题目选择和管理

前端组件设计

1. 导航组件

- 顶部导航栏
- 面包屑导航
- 侧边栏菜单（响应式）

2. 数据表格组件

- 分页功能
- 排序功能
- 筛选功能
- 响应式适配

3. 表单组件

- 输入验证
- 实时反馈
- 错误提示
- 加载状态

4. 代码编辑器组件

- 语法高亮
- 代码补全
- 主题切换
- 多语言支持

API 交互设计

1. 请求封装

```
class ApiClient {
    constructor(baseURL = '') {
        this.baseURL = baseURL;
    }

    async request(endpoint, options = {}) {
        const url = `${this.baseURL}${endpoint}`;
        const config = {
            headers: {
                'Content-Type': 'application/json',
                ...options.headers
            },
            ...options
        };

        if (config.body && typeof config.body === 'object') {
            config.body = JSON.stringify(config.body);
        }

        try {
            const response = await fetch(url, config);
            const data = await response.json();

            if (!response.ok) {
                throw new Error(data.message || '请求失败');
            }

            return data;
        } catch (error) {
            console.error('API请求错误:', error);
            throw error;
        }
    }

    // 具体API方法
    async login(credentials) {
        return this.request('/api/login', {
            method: 'POST',
            body: credentials
        });
    }

    async getProblems(params = {}) {
        const queryString = new URLSearchParams(params).toString();
        return this.request(`/api/problems?${queryString}`);
    }

    // 更多API方法...
}
```

2. 错误处理

```
// 统一错误处理
function handleApiError(error) {
    const message = error.message || '网络错误，请稍后重试';

    // 显示错误提示
    showNotification(message, 'error');

    // 特定错误处理
    if (error.message.includes('未登录')) {
        redirectToLogin();
    }
}
```

3. 状态管理

```
// 简单的状态管理
class AppState {
    constructor() {
        this.user = null;
        this.problems = [];
        this.contests = [];
        this.submissions = [];
        this.listeners = [];
    }

    setUser(user) {
        this.user = user;
        this.notify('user');
    }

    subscribe(event, callback) {
        this.listeners.push({ event, callback });
    }

    notify(event) {
        this.listeners
            .filter(listener => listener.event === event)
            .forEach(listener => listener.callback());
    }
}
```

样式设计规范

1. 颜色系统

```css
:root {
    --primary-color: #007bff;
    --success-color: #28a745;
    --warning-color: #ffc107;
    --danger-color: #dc3545;
    --secondary-color: #6c757d;
    --light-color: #f8f9fa;
    --dark-color: #343a40;

    /* 难度等级颜色 */
    --difficulty-easy: #28a745;
    --difficulty-medium: #ffc107;
    --difficulty-hard: #dc3545;
}
```

2. 响应式断点

```css
/* 小屏幕（手机） */
@media (max-width: 576px) { }

/* 中等屏幕（平板） */
@media (min-width: 577px) and (max-width: 768px) { }

/* 大屏幕（桌面） */
@media (min-width: 769px) and (max-width: 992px) { }

/* 超大屏幕 */
@media (min-width: 993px) { }
```

性能优化

1. 资源优化

- 图片懒加载
- CSS/JS 文件压缩和合并
- 使用 CDN 加速静态资源

2. 代码优化

- 避免重复 DOM 操作
- 使用事件委托
- 合理使用缓存

3. 用户体验

- 加载状态指示
- 错误边界处理
- 离线功能支持

浏览器兼容性

支持浏览器

- Chrome 60+
- Firefox 55+
- Safari 12+
- Edge 79+

特性检测

```
// 检查 Fetch API 支持
if (!window.fetch) {
    // 使用 polyfill 或降级方案
}
```

扩展开发指南

添加新页面

1. 在 `templates/` 目录创建 HTML 文件
2. 在 `app.py` 中添加对应的路由
3. 实现前端交互逻辑
4. 添加样式和响应式适配

组件开发

1. 遵循模块化原则
2. 保持组件独立性
3. 提供清晰的接口文档
4. 编写单元测试

性能监控

- 页面加载时间
- API 响应时间
- 用户交互延迟
- 错误率统计

返回README (../README.md)

docs\功能实现md

to top

功能实现文档

一、数据查询功能

1.1 用户信息查询

支持查询所有用户的公开信息，包括：

- 基础身份信息：用户ID、用户名（handle）、邮箱
- 评分相关信息：当前评分、最高评分、当前等级、最高等级
- 个人资料信息：国家、城市、组织、头像
- 活跃度信息：注册时间、最后在线时间、贡献值
- 权限标识：是否为管理员、账号活跃状态

1.2 比赛信息查询

支持查询所有比赛的详细信息，包括：

- 基本信息：比赛ID、名称、类型（CF/IOI/ICPC）
- 时间信息：开始时间、持续时间（秒）
- 描述信息：官方网址、比赛描述、难度评级、比赛类型、ICPC区域等
- 归属信息：比赛创建者

1.3 题目信息查询

支持查询所有题目的完整信息，包括：

- 标识信息：题目ID（唯一）、所属比赛、题目索引（如A、B、C）
- 题目内容：标题、题目描述、输入输出规范、样例数据
- 限制条件：时间限制、内存限制
- 统计数据：通过数、总提交数、难度评级

1.4 提交记录查询

支持查询所有代码提交记录的详细信息，包括：

- 基本标识：提交ID、用户ID、题目ID、比赛ID
- 代码信息：编程语言、源代码内容、代码长度
- 评测结果：状态（AC/WRONG_ANSWER等）、运行耗时、内存使用量
- 测试详情：通过测试用例数、详细测试结果、提交时间
- Hack相关：是否被Hack

1.5 比赛排名查询

支持查询特定比赛的排名情况，包括：

- 用户角色：参赛者或虚拟参赛（VP）
- 评分变化：比赛前后的用户评分变化

- 排名详情：比赛排名、解题数量、罚分、最终得分

1.6 查询高级功能

所有查询功能均支持对数据列的筛选功能，可根据需要选择显示特定字段。

二、数据修改功能

2.1 用户注册

实现新用户账号注册功能：

- 在用户表中添加新用户的完整信息

2.2 题目与比赛创建

支持创建新题目和新比赛：

- 在题目表中添加新题目的详细信息
- 在比赛表中添加新比赛的完整信息

2.3 代码提交

支持用户提交代码进行评测：

- 在提交表中添加提交记录信息
- 评测结果使用随机生成方式

返回README (README.md)

docs\后端开发手册.md

to top

后端开发手册（Python + Flask）

项目结构

```
Database-Principles-Major-Assignment/
├── app.py                    # Flask 主应用 - 包含所有路由和业务逻辑
├── codeforces_crawler.py     # Codeforces 数据爬虫工具
├── config.py                 # 数据库配置 - Windows 身份验证设置
├── requirements.txt          # Python 依赖包列表
└── database/                 # 数据库相关文件
    ├── create.sql           # 数据库表结构创建脚本
    ├── import.sql           # 数据导入脚本
    ├── view.sql             # 视图创建脚本
    └── tmp.sql              # 临时 SQL 脚本
```

核心架构

1. 应用入口 (app.py)

- **Flask 应用初始化**：配置会话密钥、路由注册
- **数据库连接**：使用 pyodbc 连接 SQL Server
- **路由组织**：所有 API 路由和页面路由集中管理

2. 数据库配置 (config.py)

```
# Windows 身份验证配置
DB_SERVER = 'localhost'  # SQL Server 实例名
DB_NAME = 'OJ'           # 数据库名称
# 不需要用户名和密码，使用 Windows 身份验证
```

3. 数据库连接管理

```
def get_db_connection():
    """获取数据库连接 - Windows 身份验证"""
    conn_str = f"DRIVER={DB_CONFIG['driver']};SERVER={DB_CONFIG['server']};DATABASE=
{DB_CONFIG['database']};Trusted_Connection=yes"
    return pyodbc.connect(conn_str)
```

API 接口设计

用户管理接口

- `POST /api/register` - 用户注册
- `POST /api/login` - 用户登录
- `GET /api/logout` - 用户登出
- `GET /api/user/current` - 获取当前用户信息
- `POST /api/user/update` - 更新用户信息
- `POST /api/user/change_password` - 修改密码
- `POST /api/user/set_admin` - 设置管理员权限

题目管理接口

- `GET /api/problems` - 获取题目列表
- `GET /api/problem/<problem_id>` - 获取题目详情
- `POST /api/submit` - 提交代码
- `POST /api/problems/create` - 创建题目（管理员）
- `GET /api/contest/<contest_id>/available_problems` - 获取可添加到比赛的题目

比赛管理接口

- `GET /api/contests` - 获取比赛列表
- `GET /api/contest/<contest_id>` - 获取比赛详情

- `GET /api/contest/<contest_id>/standings` - 获取比赛排名
- `GET /api/contest/<contest_id>/problems` - 获取比赛题目列表
- `POST /api/contests/create` - 创建比赛（管理员）
- `POST /api/contest/<contest_id>/add_problem` - 添加题目到比赛
- `POST /api/contest/<contest_id>/remove_problem` - 从比赛中移除题目

提交记录接口

- `GET /api/submissions` - 获取提交记录
- 支持按用户、题目、评测结果筛选

数据库操作模式

1. 连接管理

```
try:
    conn = get_db_connection()
    cursor = conn.cursor()
    # 执行 SQL 操作
    cursor.execute(query, params)
    conn.commit()
except Exception as e:
    # 错误处理
    return jsonify({'success': False, 'message': f'操作失败: {str(e)}'})
finally:
    # 资源清理
    if 'cursor' in locals(): cursor.close()
    if 'conn' in locals(): conn.close()
```

2. 事务处理

- 使用显式事务控制
- 确保数据一致性
- 异常时回滚事务

会话管理

Flask Session 配置

```
app.secret_key = 'your_secret_key_here'
```

会话数据结构

```
session['user_id']    # 用户ID
session['handle']     # 用户名
session['name']       # 真实姓名
session['rating']     # 用户评分
session['rank']       # 用户等级
session['is_admin']   # 管理员标识
```

评测系统设计

模拟评测机制

```
# 评测结果和权重
verdicts = ['Accepted', 'Wrong Answer', 'Time Limit Exceeded', 'Memory Limit Exceeded',
            'Runtime Error', 'Compilation Error', 'Idleness Limit Exceeded',
            'Presentation Error', 'Partial Solution']
weights = [35, 25, 8, 8, 8, 6, 4, 4, 2]  # 不同结果的权重

verdict = random.choices(verdicts, weights=weights)[0]
```

评测流程

1. 接收代码提交
2. 验证用户权限和题目存在性
3. 插入提交记录
4. 执行模拟评测
5. 更新题目统计信息
6. 返回评测结果

错误处理

统一错误响应格式

```
{
    'success': False,
    'message': '错误描述信息'
}
```

常见错误类型

- 数据库连接错误
- SQL 执行错误
- 用户权限错误
- 数据验证错误

安全考虑

当前实现

- 使用 Flask Session 进行用户认证
- SQL 参数化查询防止注入
- 输入数据验证

需要改进

- 密码加密存储
- CSRF 防护
- API 访问频率限制
- 敏感数据过滤

部署说明

开发环境

```
python app.py
```

生产环境建议

- 使用 Gunicorn 或 uWSGI 作为 WSGI 服务器
- 配置 Nginx 反向代理
- 启用 HTTPS
- 配置数据库连接池

扩展开发指南

添加新功能

1. 在 `app.py` 中添加新的路由
2. 实现相应的业务逻辑
3. 更新数据库结构（如果需要）
4. 测试功能完整性

性能优化建议

- 数据库查询优化
- 添加缓存机制
- 异步处理耗时操作
- 分页查询大数据集

返回README (../README.md)

docs\数据库结构概览md

to top

数据库结构概览

数据库基本信息

- 数据库名称: OJ
- 数据库类型: Microsoft SQL Server
- 认证方式: Windows 身份验证
- 字符编码: UTF-8

主要数据表结构

1. 用户表 (`Users`)

存储系统所有用户信息：

```
CREATE TABLE Users (
    user_id BIGINT IDENTITY(1,1) PRIMARY KEY,
    handle VARCHAR(50) UNIQUE NOT NULL,           -- 用户名（唯一）
    email VARCHAR(100) UNIQUE NOT NULL,           -- 邮箱（唯一）
    password VARCHAR(255) NOT NULL,               -- 密码
    name VARCHAR(100),                            -- 真实姓名
    rating INT DEFAULT 0,                         -- 当前评分
    max_rating INT DEFAULT 0,                     -- 最高评分
    user_rank VARCHAR(50) DEFAULT 'Newbie',       -- 当前等级
    max_rank VARCHAR(50) DEFAULT 'Newbie',        -- 最高等级
    country VARCHAR(100),                         -- 国家
    city VARCHAR(100),                            -- 城市
    organization VARCHAR(200),                    -- 组织
    avatar VARCHAR(500),                          -- 头像URL
    registration_time DATETIME2 DEFAULT GETDATE(), -- 注册时间
    last_online_time DATETIME2 DEFAULT GETDATE(),  -- 最后在线时间
    contribution INT DEFAULT 0,                   -- 贡献值
    is_admin BIT DEFAULT 0,                       -- 管理员标识
    is_active BIT DEFAULT 1                       -- 活跃状态
);
```

索引:

- `idx_user_rating` - 评分索引
- `idx_user_handle` - 用户名索引
- `idx_user_rank` - 等级索引

2. 比赛表 (`CONTEST`)

存储比赛相关信息：

```sql
CREATE TABLE CONTEST (
    contest_id BIGINT IDENTITY(1,1) PRIMARY KEY,
    name VARCHAR(200) NOT NULL,                    -- 比赛名称
    type VARCHAR(10) NOT NULL DEFAULT 'CF'         -- 比赛类型
        CHECK (type IN ('CF', 'IOI', 'ICPC')),
    phase VARCHAR(30) DEFAULT 'BEFORE'             -- 比赛阶段
        CHECK (phase IN ('BEFORE', 'CODING', 'PENDING_SYSTEM_TEST', 'SYSTEM_TEST',
'FINISHED')),
    frozen BIT DEFAULT 0,                          -- 是否冻结
    start_time DATETIME2 NOT NULL,                 -- 开始时间
    duration_seconds INT NOT NULL,                 -- 持续时间（秒）
    website_url VARCHAR(500),                      -- 官方网址
    description TEXT,                              -- 比赛描述
    difficulty INT DEFAULT 0,                      -- 难度评级
    kind VARCHAR(100),                             -- 比赛种类
    icpc_region VARCHAR(100),                      -- ICPC区域
    country VARCHAR(100),                          -- 国家
    city VARCHAR(100),                             -- 城市
    season VARCHAR(50),                            -- 赛季
    created_by BIGINT,                             -- 创建者ID
    FOREIGN KEY (created_by) REFERENCES Users(user_id)
);
```

索引:

- `idx_contest_time` - 开始时间索引
- `idx_contest_type` - 比赛类型索引

3. 题目表 (`PROBLEM`)

存储题目详细信息:

```
CREATE TABLE PROBLEM (
    problem_id VARCHAR(50) PRIMARY KEY,           -- 题目ID（唯一）
    contest_id BIGINT,                            -- 所属比赛ID
    problem_index VARCHAR(10),                    -- 题目索引（A,B,C...）
    title VARCHAR(300) NOT NULL,                  -- 题目标题
    statement TEXT,                               -- 题目描述
    input_specification TEXT,                     -- 输入规范
    output_specification TEXT,                    -- 输出规范
    sample_tests NVARCHAR(MAX),                   -- 样例测试（JSON格式）
    notes TEXT,                                   -- 备注
    time_limit_ms INT NOT NULL DEFAULT 1000,      -- 时间限制（毫秒）
    memory_limit_kb INT NOT NULL DEFAULT 256000,  -- 内存限制（KB）
    difficulty VARCHAR(10),                       -- 难度等级
    accepted_count INT DEFAULT 0,                 -- 通过次数
    submission_count INT DEFAULT 0,               -- 提交次数
    creation_time DATETIME2 DEFAULT GETDATE(),    -- 创建时间
    is_visible BIT DEFAULT 1,                     -- 可见性控制
    FOREIGN KEY (contest_id) REFERENCES CONTEST(contest_id)
);
```

索引:

- `idx_problem_contest` - 比赛ID索引
- `idx_problem_difficulty` - 难度索引

4. 题目标签系统

标签表（`PROBLEM_TAG`）

```
CREATE TABLE PROBLEM_TAG (
    tag_id BIGINT IDENTITY(1,1) PRIMARY KEY,
    name VARCHAR(50) UNIQUE NOT NULL,             -- 标签名称（唯一）
    description TEXT                              -- 标签描述
);
```

标签关系表（`PROBLEM_TAG_RELATION`）

```
CREATE TABLE PROBLEM_TAG_RELATION (
    problem_id VARCHAR(50) NOT NULL,              -- 题目ID
    tag_id BIGINT NOT NULL,                       -- 标签ID
    PRIMARY KEY (problem_id, tag_id),             -- 复合主键
    FOREIGN KEY (problem_id) REFERENCES PROBLEM(problem_id) ON DELETE CASCADE,
    FOREIGN KEY (tag_id) REFERENCES PROBLEM_TAG(tag_id) ON DELETE CASCADE
);
```

5. 测试用例表（`TEST_CASE`）

存储题目的测试数据：

```
CREATE TABLE TEST_CASE (
    test_case_id BIGINT IDENTITY(1,1) PRIMARY KEY,
    problem_id VARCHAR(50) NOT NULL,              -- 题目ID
    input_data TEXT NOT NULL,                     -- 输入数据
    expected_output TEXT NOT NULL,                -- 期望输出
    is_sample BIT DEFAULT 0,                      -- 是否为样例
    test_order INT DEFAULT 0,                     -- 测试顺序
    FOREIGN KEY (problem_id) REFERENCES PROBLEM(problem_id) ON DELETE CASCADE
);
```

索引:

- `idx_test_case_problem` - 题目ID索引

6. 提交表 (`SUBMISSION`)

记录用户代码提交和评测结果:

```
CREATE TABLE SUBMISSION (
    submission_id BIGINT IDENTITY(1,1) PRIMARY KEY,
    user_id BIGINT NOT NULL,                      -- 用户ID
    problem_id VARCHAR(50) NOT NULL,              -- 题目ID
    contest_id BIGINT,                            -- 比赛ID
    programming_language VARCHAR(50) NOT NULL,    -- 编程语言
    source_code TEXT NOT NULL,                    -- 源代码
    source_length INT NOT NULL,                   -- 代码长度
    verdict VARCHAR(50) DEFAULT 'Pending'         -- 评测结果
        CHECK (verdict IN ('Pending', 'Running', 'Accepted', 'Wrong Answer',
                           'Time Limit Exceeded', 'Memory Limit Exceeded',
                           'Runtime Error', 'Compilation Error', 'Presentation Error',
                           'Failed', 'Idleness Limit Exceeded', 'Partial Solution',
                           'Skipped', 'Challenged', 'Rejected')),
    time_consumed_ms INT DEFAULT 0,               -- 运行时间（毫秒）
    memory_consumed_kb INT DEFAULT 0,             -- 内存使用（KB）
    passed_test_count INT DEFAULT 0,              -- 通过测试数
    test_results NVARCHAR(MAX),                   -- 详细测试结果（JSON）
    submission_time DATETIME2 DEFAULT GETDATE(),  -- 提交时间
    relative_time INT,                            -- 相对时间
    points INT,                                   -- 得分
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (problem_id) REFERENCES PROBLEM(problem_id),
    FOREIGN KEY (contest_id) REFERENCES CONTEST(contest_id)
);
```

索引:

- `idx_submission_user_time` - 用户和时间索引
- `idx_submission_contest` - 比赛和题目索引
- `idx_submission_verdict` - 评测结果索引

7. 比赛用户关系表（`CONTEST_USER`）

记录用户参与比赛的情况：

```
CREATE TABLE CONTEST_USER (
    contest_id BIGINT NOT NULL,                    -- 比赛ID
    user_id BIGINT NOT NULL,                       -- 用户ID
    registration_time DATETIME2 DEFAULT GETDATE(), -- 注册时间
    role VARCHAR(30) NOT NULL DEFAULT 'contestant' -- 用户角色
        CHECK (role IN ('author', 'tester', 'contestant', 'out_of_competition',
'virtual')),
    rating_before INT,                             -- 比赛前评分
    rating_after INT,                              -- 比赛后评分
    contest_rank INT,                              -- 比赛排名
    solved_count INT DEFAULT 0,                    -- 解题数量
    total_penalty INT DEFAULT 0,                   -- 总罚分
    scores INT DEFAULT 0,                          -- 得分
    PRIMARY KEY (contest_id, user_id),             -- 复合主键
    FOREIGN KEY (contest_id) REFERENCES CONTEST(contest_id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
);
```

索引：

- `idx_contest_user_role` - 用户角色索引

8. Hack表（`HACK`）

记录Hack行为：

```
CREATE TABLE HACK (
    hack_id BIGINT IDENTITY(1,1) PRIMARY KEY,
    hacker_id BIGINT NOT NULL,                     -- Hack者ID
    defender_id BIGINT NOT NULL,                   -- 被Hack者ID
    problem_id VARCHAR(50) NOT NULL,               -- 题目ID
    contest_id BIGINT NOT NULL,                    -- 比赛ID
    verdict VARCHAR(20) NOT NULL                   -- Hack结果
        CHECK (verdict IN ('SUCCESSFUL', 'UNSUCCESSFUL', 'INVALID')),
    test_case TEXT NOT NULL,                       -- 使用的测试用例
    hack_time DATETIME2 DEFAULT GETDATE(),         -- Hack时间
    hack_result TEXT,                              -- Hack结果详情
    FOREIGN KEY (hacker_id) REFERENCES Users(user_id),
    FOREIGN KEY (defender_id) REFERENCES Users(user_id),
    FOREIGN KEY (problem_id) REFERENCES PROBLEM(problem_id),
    FOREIGN KEY (contest_id) REFERENCES CONTEST(contest_id)
);
```

索引：

- `idx_hack_time` - Hack时间索引

- `idx_hack_verdict` - Hack结果索引

数据库关系图

```
Users (1) ←---→ (N) CONTEST_USER (N) ←---→ (1) CONTEST (1) ←---→ (N) PROBLEM (1) ←---→
(N) TEST_CASE
  |                                                                    |
  |                                                                    |
  ↓                                                                    ↓
SUBMISSION (N) ←---→ (1) PROBLEM (N) ←---→ (N) PROBLEM_TAG_RELATION (N) ←---→ (1)
PROBLEM_TAG
  |
  ↓
HACK
```

数据完整性约束

主键约束

- 所有表都有明确的主键
- 用户表和比赛表使用自增ID
- 题目表使用自定义ID（便于与外部系统集成）

外键约束

- 完整的参照完整性
- 级联删除设置（如标签关系、测试用例）
- 防止孤儿记录

检查约束

- 枚举值验证（比赛类型、阶段、评测结果等）
- 数据范围验证

性能优化

索引策略

- 为常用查询字段创建索引
- 复合索引优化多条件查询
- 定期维护索引统计信息

查询优化

- 使用参数化查询
- 避免全表扫描
- 合理使用连接查询

返回README (../README.md)

READMEmd

to top

数据库原理大作业 - 在线判题系统 (OJ)

> 数据库原理实验：课程考核是分小组完成一个数据库设计大作业，每个小组2人，需要从选题、需求分析开始分阶段完成一个数据库应用程序的设计与实现，课程结束需要进行演示答辩并提交设计报告和源程序。课程设计大作业占总成绩的70%，过程考核（实验作业、考勤和课堂表现）占总成绩的30%。

📋 项目简介

这是一个基于 **Codeforces** 平台功能设计的在线判题系统（OJ），使用 **Microsoft SQL Server** 作为数据库，**Python Flask** 作为后端框架，前端采用原生 HTML/CSS/JavaScript 实现。

🏗 系统架构

技术栈

- 后端: Python Flask + pyodbc
- 数据库: Microsoft SQL Server
- 前端: HTML5 + CSS3 + JavaScript (原生)
- 认证: Windows 身份验证

核心功能

- ☑ 用户注册与登录系统
- ☑ 题目浏览与提交
- ☑ 代码评测（模拟评测）
- ☑ 比赛管理
- ☑ 用户排名系统
- ☑ 管理员功能（题目/比赛创建）

📁 项目结构

```
Database-Principles-Major-Assignment/
├── app.py                      # Flask 主应用
├── codeforces_crawler.py       # Codeforces 数据爬虫
├── config.py                   # 数据库配置
├── requirements.txt            # Python 依赖
├── README.md                   # 项目说明文档
├── database/                   # 数据库相关文件
│   ├── create.sql             # 数据库表结构创建脚本
│   ├── import.sql             # 数据导入脚本
│   ├── view.sql               # 视图创建脚本
│   ├── create_admin.sql       # 管理员创建脚本
│   └── E-R图/                 # 数据库设计图
├── docs/                       # 项目文档
│   ├── 功能实现.md            # 功能说明文档
│   ├── 后端开发手册.md        # 后端开发指南
│   ├── 前端开发手册.md        # 前端开发指南
│   └── 数据库结构概览.md      # 数据库结构说明
├── static/                     # 静态资源文件
└── templates/                 # HTML 模板文件
    ├── index.html             # 首页
    ├── problems.html          # 题目列表页
    ├── problem_detail.html    # 题目详情页
    ├── contests.html          # 比赛列表页
    ├── contest_detail.html    # 比赛详情页
    ├── contest_standings.html # 比赛排名页
    ├── submissions.html       # 提交记录页
    ├── users.html             # 用户列表页
    ├── profile.html           # 用户个人资料页
    ├── create_problem.html    # 题目创建页
    ├── create_contest.html    # 比赛创建页
    └── index_loger.html       # 登录页
```

数据库设计

主要数据表

1. 用户表 (Users)

- 用户ID、用户名、邮箱、密码
- 评分信息：当前评分、最高评分、等级
- 个人信息：国家、城市、组织、头像
- 活跃信息：注册时间、最后在线时间、贡献值
- 权限控制：管理员标识、活跃状态

2. 比赛表 (CONTEST)

- 比赛ID、名称、类型（CF/IOI/ICPC）、阶段
- 时间信息：开始时间、持续时间
- 描述信息：网址、描述、难度、类型、ICPC区域

- 创建者外键关联用户

3. 题目表 (PROBLEM)

- 题目ID（唯一）、所属比赛、题目索引
- 题目内容：标题、描述、输入输出规范、样例
- 限制条件：时间限制、内存限制
- 统计信息：通过数、提交数、难度
- 可见性控制

4. 题目标签系统

- `PROBLEM_TAG`：标签字典表
- `PROBLEM_TAG_RELATION`：题目与标签的多对多关系表

5. 测试用例表 (TEST_CASE)

- 输入数据、期望输出
- 标记是否为样例、测试顺序

6. 提交表 (SUBMISSION)

- 提交ID、用户ID、题目ID、比赛ID
- 编程语言、源代码、代码长度
- 判题结果：状态、耗时、内存使用
- 通过测试数、详细测试结果、提交时间

7. 比赛用户关系表 (CONTEST_USER)

- 用户在比赛中的角色
- 比赛前后的评分变化
- 比赛排名、解题数、罚分、得分

8. Hack表 (HACK)

- Hack者、被Hack者、题目、比赛
- Hack结果、使用的测试用例、Hack时间

E-R 图

```
erDiagram
    Users {
        bigint user_id PK
        varchar handle UK
        varchar email UK
        varchar password
        varchar name
        int rating
        int max_rating
        varchar user_rank
        varchar max_rank
        varchar country
        varchar city
        varchar organization
        varchar avatar
        datetime2 registration_time
        datetime2 last_online_time
        int contribution
        bit is_admin
        bit is_active
    }

    CONTEST {
        bigint contest_id PK
        varchar name
        varchar type
        varchar phase
        bit frozen
        datetime2 start_time
        int duration_seconds
        varchar website_url
        text description
        int difficulty
        varchar kind
        varchar icpc_region
        varchar country
        varchar city
        varchar season
        bigint created_by FK
    }

    PROBLEM {
        varchar problem_id PK
        bigint contest_id FK
        varchar problem_index
        varchar title
        text statement
        text input_specification
        text output_specification
        nvarchar sample_tests
        text notes
        int time_limit_ms
```

```
        int memory_limit_kb
        varchar difficulty
        int accepted_count
        int submission_count
        datetime2 creation_time
        bit is_visible
    }

    PROBLEM_TAG {
        bigint tag_id PK
        varchar name UK
        text description
    }

    PROBLEM_TAG_RELATION {
        varchar problem_id PK,FK
        bigint tag_id PK,FK
    }

    TEST_CASE {
        bigint test_case_id PK
        varchar problem_id FK
        text input_data
        text expected_output
        bit is_sample
        int test_order
    }

    SUBMISSION {
        bigint submission_id PK
        bigint user_id FK
        varchar problem_id FK
        bigint contest_id FK
        varchar programming_language
        text source_code
        int source_length
        varchar verdict
        int time_consumed_ms
        int memory_consumed_kb
        int passed_test_count
        nvarchar test_results
        datetime2 submission_time
        int relative_time
        int points
    }

    CONTEST_USER {
        bigint contest_id PK,FK
        bigint user_id PK,FK
        datetime2 registration_time
        varchar role
        int rating_before
        int rating_after
```

```
        int contest_rank
        int solved_count
        int total_penalty
        int scores
    }

    HACK {
        bigint hack_id PK
        bigint hacker_id FK
        bigint defender_id FK
        varchar problem_id FK
        bigint contest_id FK
        varchar verdict
        text test_case
        datetime2 hack_time
        text hack_result
    }

    Users ||--o{ CONTEST : "created_by"
    Users ||--o{ SUBMISSION : "submits"
    Users ||--o{ HACK : "as_hacker"
    Users ||--o{ HACK : "as_defender"
    Users }o--|| CONTEST_USER : "participates_in"

    CONTEST ||--o{ PROBLEM : "contains"
    CONTEST ||--o{ SUBMISSION : "has_submissions"
    CONTEST ||--o{ HACK : "has_hacks"
    CONTEST }o--|| CONTEST_USER : "has_participants"

    PROBLEM ||--o{ PROBLEM_TAG_RELATION : "has_tags"
    PROBLEM ||--o{ TEST_CASE : "has_test_cases"
    PROBLEM ||--o{ SUBMISSION : "has_submissions"
    PROBLEM ||--o{ HACK : "targeted_in"

    PROBLEM_TAG ||--o{ PROBLEM_TAG_RELATION : "categorizes"
```

🚀 快速开始

环境要求

- Python 3.7+
- Microsoft SQL Server
- Windows 操作系统（支持 Windows 身份验证）

安装步骤

1. 克隆项目

```
git clone <repository-url>
cd Database-Principles-Major-Assignment
```

2. 安装 Python 依赖

```
pip install -r requirements.txt
```

3. 配置数据库

   - 确保 SQL Server 服务正在运行
   - 修改 `config.py` 中的数据库配置：

```
DB_SERVER = 'localhost'  # 或您的 SQL Server 实例名
DB_NAME = 'OJ'
```

4. 创建数据库

   - 在 SQL Server 中创建名为 `OJ` 的数据库
   - 执行 `database/create.sql` 创建表结构
   - 可选：执行 `database/import.sql` 导入示例数据

5. 启动应用

```
python app.py
```

6. 访问系统
   打开浏览器访问： `http://localhost:5000`

默认管理员账号

系统启动后，您需要手动在数据库中创建管理员用户：

```
INSERT INTO Users (handle, email, password, name, is_admin)
VALUES ('admin', 'admin@example.com', 'password', 'Administrator', 1);
```

🔧 功能特性

用户功能

- [x] 用户注册与登录
- [x] 个人信息管理
- [x] 密码修改

- [x] 题目浏览与搜索
- [x] 代码提交与评测
- [x] 提交记录查看
- [x] 比赛参与
- [x] 排名查看

管理员功能

- [x] 题目创建与管理
- [x] 比赛创建与管理
- [x] 用户权限管理
- [x] 系统数据管理

评测系统

- [x] 多语言支持
- [x] 模拟评测（随机结果）
- [x] 详细的评测信息
- [x] 测试用例管理

API 接口

用户相关

- `POST /api/register` - 用户注册
- `POST /api/login` - 用户登录
- `GET /api/logout` - 用户登出
- `GET /api/user/current` - 获取当前用户信息
- `POST /api/user/update` - 更新用户信息
- `POST /api/user/change_password` - 修改密码

题目相关

- `GET /api/problems` - 获取题目列表
- `GET /api/problem/<problem_id>` - 获取题目详情
- `POST /api/submit` - 提交代码
- `POST /api/problems/create` - 创建题目（管理员）

比赛相关

- `GET /api/contests` - 获取比赛列表
- `GET /api/contest/<contest_id>` - 获取比赛详情
- `GET /api/contest/<contest_id>/standings` - 获取比赛排名
- `POST /api/contests/create` - 创建比赛（管理员）

提交记录

- `GET /api/submissions` - 获取提交记录
- 支持按用户、题目、结果筛选

已知限制

1. **评测系统**: 当前使用随机结果模拟评测，不支持真实的代码编译和执行
2. **安全性**: 密码以明文存储，生产环境需要加密处理
3. **性能**: 未进行大规模并发测试和性能优化
4. **功能**: 缺少真实的代码编译、Hack 功能等高级特性

开发说明

数据库连接

系统使用 Windows 身份验证连接 SQL Server，无需配置用户名和密码。

扩展开发

- 添加新的编程语言支持
- 实现真实的代码评测系统
- 添加更多比赛类型和功能
- 优化前端界面和用户体验

贡献指南

1. Fork 本项目
2. 创建功能分支 (`git checkout -b feature/AmazingFeature`)
3. 提交更改 (`git commit -m 'Add some AmazingFeature'`)
4. 推送到分支 (`git push origin feature/AmazingFeature`)
5. 开启 Pull Request

许可证

本项目仅用于数据库原理课程教学目的，为本科生大作业项目。

使用条款：

- 本项目仅供学习和教学使用
- 禁止用于商业用途
- 允许在注明出处的情况下用于学术研究
- 不得用于任何违法或不道德的行为

联系方式

如有问题或建议，请联系项目维护者。

最后更新: 2025年11月

requirementstxt

to top

```
Flask==2.3.3
Flask-SQLAlchemy==3.0.5
Flask-CORS==4.0.0
pymssql==2.2.7
```

templates\contestshtml

to top

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>比赛列表 - 在线判题系统</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; background: #f5f5f5; }
        .navbar { background: #2c3e50; color: white; padding: 1rem; }
        .nav-content { max-width: 1200px; margin: 0 auto; display: flex; justify-
content: space-between; }
        .nav-links a { color: white; text-decoration: none; margin-left: 1rem; }
        .container { max-width: 1200px; margin: 2rem auto; padding: 0 1rem; }
        .card { background: white; padding: 1.5rem; border-radius: 8px; box-shadow: 0
2px 4px rgba(0,0,0,0.1); margin-bottom: 1rem; }
        .header-row { display: flex; justify-content: space-between; align-items:
center; margin-bottom: 1rem; }
        .btn { background: #3498db; color: white; border: none; padding: 0.5rem 1rem;
border-radius: 4px; cursor: pointer; text-decoration: none; display: inline-block; }
        .btn:hover { background: #2980b9; }
        .btn-success { background: #27ae60; }
        .btn-success:hover { background: #219a52; }
        .btn-warning { background: #f39c12; }
        .btn-warning:hover { background: #e67e22; }
        .contests-table { width: 100%; border-collapse: collapse; }
        .contests-table th, .contests-table td { padding: 0.75rem; text-align: left;
border-bottom: 1px solid #ddd; }
        .contests-table th { background: #f8f9fa; font-weight: bold; }
        .contest-link { color: #3498db; text-decoration: none; font-weight: 500; }
        .contest-link:hover { color: #2980b9; text-decoration: underline; }
        .phase-before { color: #e67e22; font-weight: bold; }
        .phase-coding { color: #27ae60; font-weight: bold; }
        .phase-finished { color: #7f8c8d; font-weight: bold; }
        .difficulty-easy { color: #28a745; }
        .difficulty-medium { color: #ffc107; }
        .difficulty-hard { color: #dc3545; }
        .pagination { display: flex; justify-content: center; gap: 0.5rem; margin-top:
1rem; }
        .page-btn { padding: 0.5rem 1rem; border: 1px solid #ddd; background: white;
cursor: pointer; }
        .page-btn.active { background: #3498db; color: white; border-color: #3498db; }
        .page-btn:hover:not(.active) { background: #f8f9fa; }
        .filter-bar { display: flex; gap: 1rem; margin-bottom: 1rem; flex-wrap: wrap; }
        .filter-group { display: flex; flex-direction: column; gap: 0.25rem; }
        .filter-label { font-size: 0.875rem; color: #666; }
        .filter-select { padding: 0.5rem; border: 1px solid #ddd; border-radius: 4px;
min-width: 120px; }
        .search-box { padding: 0.5rem; border: 1px solid #ddd; border-radius: 4px; min-
width: 200px; }
        .empty-state { text-align: center; padding: 2rem; color: #666; }
        .admin-actions { margin-bottom: 1rem; }
```

```
    </style>
</head>
<body>
    <nav class="navbar">
        <div class="nav-content">
            <h1>在线判题系统</h1>
            <div class="nav-links">
                <a href="/">首页</a>
                <a href="/problems">题目</a>
                <a href="/contests">比赛</a>
                <a href="/submissions">提交记录</a>
                <a href="/users">用户</a>
                <a href="/profile" id="profile-link">个人资料</a>
            </div>
        </div>
    </nav>

    <div class="container">
        <div class="card">
            <div class="header-row">
                <h1>比赛列表</h1>
                <div>
                    <button id="create-contest-btn" class="btn btn-success"
style="display: none;">创建比赛</button>
                </div>
            </div>

            <!-- 管理员操作 -->
            <div id="admin-actions" class="admin-actions" style="display: none;">
                <button id="refresh-contests-btn" class="btn btn-warning">刷新比赛列表
</button>
                <small style="color: #666; margin-left: 1rem;">从外部API同步比赛数据
</small>
            </div>

            <!-- 筛选栏 -->
            <div class="filter-bar">
                <div class="filter-group">
                    <span class="filter-label">比赛阶段</span>
                    <select id="phase-filter" class="filter-select">
                        <option value="">全部</option>
                        <option value="BEFORE">未开始</option>
                        <option value="CODING">进行中</option>
                        <option value="FINISHED">已结束</option>
                    </select>
                </div>
                <div class="filter-group">
                    <span class="filter-label">比赛类型</span>
                    <select id="type-filter" class="filter-select">
                        <option value="">全部</option>
                        <option value="CF">Codeforces式</option>
                        <option value="IOI">IOI式</option>
                        <option value="ICPC">ICPC式</option>
```

```
                    </select>
                </div>
                <div class="filter-group">
                    <span class="filter-label">难度</span>
                    <select id="difficulty-filter" class="filter-select">
                        <option value="">全部</option>
                        <option value="简单">简单</option>
                        <option value="中等">中等</option>
                        <option value="困难">困难</option>
                    </select>
                </div>
                <div class="filter-group">
                    <span class="filter-label">搜索</span>
                    <input type="text" id="search-input" class="search-box"
placeholder="搜索比赛名称...">
                </div>
                <div class="filter-group" style="align-self: flex-end;">
                    <button id="apply-filters-btn" class="btn">应用筛选</button>
                    <button id="reset-filters-btn" class="btn" style="background:
#95a5a6;">重置</button>
                </div>
            </div>

            <div id="contests-table-container">
                <table class="contests-table">
                    <thead>
                        <tr>
                            <th width="100">比赛ID</th>
                            <th>比赛名称</th>
                            <th width="120">阶段</th>
                            <th width="120">类型</th>
                            <th width="100">难度</th>
                            <th width="180">开始时间</th>
                            <th width="180">结束时间</th>
                            <th width="100">操作</th>
                        </tr>
                    </thead>
                    <tbody id="contests-table-body">
                        <!-- 比赛数据将通过JavaScript动态加载 -->
                    </tbody>
                </table>
            </div>

            <div id="pagination" class="pagination">
                <!-- 分页按钮将通过JavaScript动态生成 -->
            </div>
        </div>
    </div>

    <!-- 创建比赛模态框 -->
    <div id="create-contest-modal" style="display: none; position: fixed; top: 0; left:
0; width: 100%; height: 100%; background: rgba(0,0,0,0.5); z-index: 1000;">
        <div style="position: absolute; top: 50%; left: 50%; transform: translate(-50%,
```

```
-50%); background: white; padding: 2rem; border-radius: 8px; width: 90%; max-width:
600px;">
            <h2 style="margin-bottom: 1rem;">创建比赛</h2>
            <form id="create-contest-form">
                <div style="display: grid; gap: 1rem;">
                    <div>
                        <label style="display: block; margin-bottom: 0.25rem; font-
weight: bold;">比赛名称</label>
                        <input type="text" id="contest-name" required style="width:
100%; padding: 0.5rem; border: 1px solid #ddd; border-radius: 4px;">
                    </div>
                    <div>
                        <label style="display: block; margin-bottom: 0.25rem; font-
weight: bold;">比赛描述</label>
                        <textarea id="contest-description" style="width: 100%; padding:
0.5rem; border: 1px solid #ddd; border-radius: 4px; height: 100px;"></textarea>
                    </div>
                    <div style="display: grid; grid-template-columns: 1fr 1fr; gap:
1rem;">
                        <div>
                            <label style="display: block; margin-bottom: 0.25rem; font-
weight: bold;">开始时间</label>
                            <input type="datetime-local" id="contest-start-time"
required style="width: 100%; padding: 0.5rem; border: 1px solid #ddd; border-radius:
4px;">
                        </div>
                        <div>
                            <label style="display: block; margin-bottom: 0.25rem; font-
weight: bold;">结束时间</label>
                            <input type="datetime-local" id="contest-end-time" required
style="width: 100%; padding: 0.5rem; border: 1px solid #ddd; border-radius: 4px;">
                        </div>
                    </div>
                    <div style="display: grid; grid-template-columns: 1fr 1fr; gap:
1rem;">
                        <div>
                            <label style="display: block; margin-bottom: 0.25rem; font-
weight: bold;">比赛类型</label>
                            <select id="contest-type" required style="width: 100%;
padding: 0.5rem; border: 1px solid #ddd; border-radius: 4px;">
                                <option value="CF">Codeforces式</option>
                                <option value="IOI">IOI式</option>
                                <option value="ICPC">ICPC式</option>
                            </select>
                        </div>
                        <div>
                            <label style="display: block; margin-bottom: 0.25rem; font-
weight: bold;">难度</label>
                            <select id="contest-difficulty" style="width: 100%;
padding: 0.5rem; border: 1px solid #ddd; border-radius: 4px;">
                                <option value="">无</option>
                                <option value="简单">简单</option>
                                <option value="中等">中等</option>
```

```
                                <option value="困难">困难</option>
                            </select>
                        </div>
                    </div>
                    <div>
                        <label style="display: block; margin-bottom: 0.25rem; font-
weight: bold;">比赛种类</label>
                        <input type="text" id="contest-kind" style="width: 100%;
padding: 0.5rem; border: 1px solid #ddd; border-radius: 4px;">
                    </div>
                </div>
                <div style="display: flex; gap: 0.5rem; margin-top: 1.5rem;">
                    <button type="submit" class="btn btn-success">创建</button>
                    <button type="button" id="cancel-create-btn" class="btn"
style="background: #95a5a6;">取消</button>
                </div>
            </form>
        </div>
    </div>

    <script>
        let contests = [];
        let currentPage = 1;
        const pageSize = 20;
        let filters = {
            phase: '',
            type: '',
            difficulty: '',
            search: ''
        };

        function getPhaseText(phase) {
            const phaseMap = {
                'BEFORE': '未开始',
                'CODING': '进行中',
                'PENDING_SYSTEM_TEST': '等待系统测试',
                'SYSTEM_TEST': '系统测试中',
                'FINISHED': '已结束'
            };
            return phaseMap[phase] || phase;
        }

        function getPhaseClass(phase) {
            const phaseClassMap = {
                'BEFORE': 'phase-before',
                'CODING': 'phase-coding',
                'FINISHED': 'phase-finished'
            };
            return phaseClassMap[phase] || '';
        }

        function getTypeText(type) {
            const typeMap = {
```

```
                'CF': 'Codeforces式',
                'IOI': 'IOI式',
                'ICPC': 'ICPC式'
            };
            return typeMap[type] || type;
        }

        function getDifficultyClass(difficulty) {
            if (!difficulty) return '';
            if (difficulty === '简单' || difficulty === 'Easy') return 'difficulty-
easy';
            if (difficulty === '中等' || difficulty === 'Medium') return 'difficulty-
medium';
            if (difficulty === '困难' || difficulty === 'Hard') return 'difficulty-
hard';
            return '';
        }

        function formatDateTime(datetimeStr) {
            if (!datetimeStr) return '-';
            const date = new Date(datetimeStr);
            return date.toLocaleString('zh-CN');
        }

        async function loadContests() {
            try {
                const params = new URLSearchParams({
                    page: currentPage,
                    page_size: pageSize,
                    ...filters
                });

                const response = await fetch(`/api/contests?${params}`);
                const result = await response.json();

                if (result.success) {
                    contests = result.contests;
                    displayContests(contests);
                    setupPagination(result.total_count);
                } else {
                    console.error('加载比赛列表失败:', result.message);
                    displayContests([]);
                }
            } catch (error) {
                console.error('加载比赛列表时发生错误:', error);
                displayContests([]);
            }
        }

        function displayContests(contests) {
            const tbody = document.getElementById('contests-table-body');

            if (!contests || contests.length === 0) {
```

```
                tbody.innerHTML = `
                    <tr>
                        <td colspan="8" class="empty-state">
                            暂无比赛数据
                        </td>
                    </tr>
                `;
                return;
            }

            tbody.innerHTML = '';
            contests.forEach(contest => {
                const row = document.createElement('tr');
                row.innerHTML = `
                    <td>${contest.contest_id}</td>
                    <td>
                        <a href="/contest/${contest.contest_id}" class="contest-link">
                            ${contest.name}
                        </a>
                    </td>
                    <td
class="${getPhaseClass(contest.phase)}">${getPhaseText(contest.phase)}</td>
                    <td>${getTypeText(contest.type)}</td>
                    <td
class="${getDifficultyClass(contest.difficulty)}">${contest.difficulty || '-'}</td>
                    <td>${formatDateTime(contest.start_time)}</td>
                    <td>${formatDateTime(contest.end_time)}</td>
                    <td>
                        <a href="/contest/${contest.contest_id}" class="btn">查看</a>
                    </td>
                `;
                tbody.appendChild(row);
            });
        }

        function setupPagination(totalCount) {
            const totalPages = Math.ceil(totalCount / pageSize);
            const pagination = document.getElementById('pagination');

            if (totalPages <= 1) {
                pagination.innerHTML = '';
                return;
            }

            let paginationHTML = '';

            // 上一页按钮
            if (currentPage > 1) {
                paginationHTML += `<button class="page-btn"
onclick="changePage(${currentPage - 1})">上一页</button>`;
            }

            // 页码按钮
```

```
            const startPage = Math.max(1, currentPage - 2);
            const endPage = Math.min(totalPages, currentPage + 2);

            if (startPage > 1) {
                paginationHTML += `<button class="page-btn"
onclick="changePage(1)">1</button>`;
                if (startPage > 2) {
                    paginationHTML += `<span style="padding: 0.5rem;">...</span>`;
                }
            }

            for (let i = startPage; i <= endPage; i++) {
                paginationHTML += `<button class="page-btn ${i === currentPage ?
'active' : ''}" onclick="changePage(${i})">${i}</button>`;
            }

            if (endPage < totalPages) {
                if (endPage < totalPages - 1) {
                    paginationHTML += `<span style="padding: 0.5rem;">...</span>`;
                }
                paginationHTML += `<button class="page-btn"
onclick="changePage(${totalPages})">${totalPages}</button>`;
            }

            // 下一页按钮
            if (currentPage < totalPages) {
                paginationHTML += `<button class="page-btn"
onclick="changePage(${currentPage + 1})">下一页</button>`;
            }

            pagination.innerHTML = paginationHTML;
        }

        function changePage(page) {
            currentPage = page;
            loadContests();
            // 滚动到顶部
            window.scrollTo(0, 0);
        }

        function applyFilters() {
            filters = {
                phase: document.getElementById('phase-filter').value,
                type: document.getElementById('type-filter').value,
                difficulty: document.getElementById('difficulty-filter').value,
                search: document.getElementById('search-input').value.trim()
            };
            currentPage = 1;
            loadContests();
        }

        function resetFilters() {
            document.getElementById('phase-filter').value = '';
```

```
            document.getElementById('type-filter').value = '';
            document.getElementById('difficulty-filter').value = '';
            document.getElementById('search-input').value = '';
            applyFilters();
        }

        // 检查管理员权限
        async function checkAdminPermission() {
            try {
                const response = await fetch('/api/user/current');
                const result = await response.json();

                if (result.success && result.user && result.user.is_admin) {
                    document.getElementById('create-contest-btn').style.display =
'inline-block';
                    document.getElementById('admin-actions').style.display = 'block';
                }
            } catch (error) {
                console.error('检查管理员权限失败:', error);
            }
        }

        // 创建比赛
        async function createContest(contestData) {
            try {
                const response = await fetch('/api/contest/create', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    },
                    body: JSON.stringify(contestData)
                });

                const result = await response.json();

                if (result.success) {
                    alert('比赛创建成功! ');
                    hideCreateContestModal();
                    loadContests(); // 刷新列表
                } else {
                    alert('创建比赛失败: ' + result.message);
                }
            } catch (error) {
                console.error('创建比赛失败:', error);
                alert('创建比赛失败，请检查网络连接');
            }
        }

        function showCreateContestModal() {
            document.getElementById('create-contest-modal').style.display = 'block';
        }

        function hideCreateContestModal() {
```

```
            document.getElementById('create-contest-modal').style.display = 'none';
            document.getElementById('create-contest-form').reset();
        }

        // 页面加载时执行
        document.addEventListener('DOMContentLoaded', function() {
            loadContests();
            checkAdminPermission();
        });

        // 事件监听器
        document.getElementById('apply-filters-btn').addEventListener('click',
applyFilters);
        document.getElementById('reset-filters-btn').addEventListener('click',
resetFilters);
        document.getElementById('create-contest-btn').addEventListener('click',
showCreateContestModal);
        document.getElementById('cancel-create-btn').addEventListener('click',
hideCreateContestModal);
        document.getElementById('refresh-contests-btn').addEventListener('click',
function() {
            if (confirm('确定要从外部API同步比赛数据吗？')) {
                refreshContestsFromAPI();
            }
        });

        // 创建比赛表单提交
        document.getElementById('create-contest-form').addEventListener('submit',
function(e) {
            e.preventDefault();

            const contestData = {
                name: document.getElementById('contest-name').value,
                description: document.getElementById('contest-description').value,
                start_time: document.getElementById('contest-start-time').value,
                end_time: document.getElementById('contest-end-time').value,
                type: document.getElementById('contest-type').value,
                difficulty: document.getElementById('contest-difficulty').value ||
null,
                kind: document.getElementById('contest-kind').value || null
            };

            createContest(contestData);
        });

        // 从外部API刷新比赛数据
        async function refreshContestsFromAPI() {
            try {
                const response = await fetch('/api/contests/refresh', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    }
```

```
                });

                const result = await response.json();

                if (result.success) {
                    alert('比赛数据刷新成功！');
                    loadContests(); // 刷新列表
                } else {
                    alert('刷新比赛数据失败: ' + result.message);
                }
            } catch (error) {
                console.error('刷新比赛数据失败:', error);
                alert('刷新比赛数据失败，请检查网络连接');
            }
        }

    </script>
</body>
</html>
```

templates\contest_detailhtml

to top

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>比赛详情 - 在线判题系统</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; background: #f5f5f5; }
        .navbar { background: #2c3e50; color: white; padding: 1rem; }
        .btn-warning { background: #f39c12; }
        .btn-warning:hover { background: #e67e22; }
        .nav-content { max-width: 1200px; margin: 0 auto; display: flex; justify-
content: space-between; }
        .nav-links a { color: white; text-decoration: none; margin-left: 1rem; }
        .container { max-width: 1200px; margin: 2rem auto; padding: 0 1rem; }
        .card { background: white; padding: 1.5rem; border-radius: 8px; box-shadow: 0
2px 4px rgba(0,0,0,0.1); margin-bottom: 1rem; }
        .contest-header { margin-bottom: 2rem; }
        .contest-title { font-size: 2rem; color: #2c3e50; margin-bottom: 0.5rem; }
        .contest-meta { display: grid; grid-template-columns: repeat(auto-fit,
minmax(250px, 1fr)); gap: 1rem; margin-bottom: 2rem; }
        .meta-item { display: flex; flex-direction: column; }
        .meta-label { font-size: 0.875rem; color: #666; margin-bottom: 0.25rem; }
        .meta-value { font-weight: 500; }
        .section-title { border-bottom: 2px solid #3498db; padding-bottom: 0.5rem;
margin-bottom: 1rem; color: #2c3e50; }
        .problems-table { width: 100%; border-collapse: collapse; }
        .problems-table th, .problems-table td { padding: 0.75rem; text-align: left;
border-bottom: 1px solid #ddd; }
        .problems-table th { background: #f8f9fa; font-weight: bold; }
        .problem-link { color: #3498db; text-decoration: none; font-weight: 500; }
        .problem-link:hover { color: #2980b9; text-decoration: underline; }
        .btn { background: #3498db; color: white; border: none; padding: 0.5rem 1rem;
border-radius: 4px; cursor: pointer; text-decoration: none; display: inline-block;
margin-right: 0.5rem; }
        .btn:hover { background: #2980b9; }
        .btn-secondary { background: #95a5a6; }
        .btn-secondary:hover { background: #7f8c8d; }
        .btn-success { background: #27ae60; }
        .btn-success:hover { background: #219a52; }
        .btn-danger { background: #e74c3c; }
        .btn-danger:hover { background: #c0392b; }
        .phase-before { color: #e67e22; font-weight: bold; }
        .phase-coding { color: #27ae60; font-weight: bold; }
        .phase-finished { color: #7f8c8d; font-weight: bold; }
        .difficulty-easy { color: #28a745; }
        .difficulty-medium { color: #ffc107; }
        .difficulty-hard { color: #dc3545; }
        .header-actions { display: flex; gap: 0.5rem; margin-top: 1rem; }
        .countdown { background: #f8f9fa; padding: 1rem; border-radius: 4px; margin-
bottom: 1rem; text-align: center; }
```

```
        .countdown-timer { font-size: 1.5rem; font-weight: bold; color: #e74c3c; }
        .description-content { line-height: 1.6; margin-bottom: 1rem; }
        .empty-state { text-align: center; padding: 2rem; color: #666; }
        .admin-actions { display: flex; gap: 0.5rem; margin-bottom: 1rem; }
        .form-section { background: #f8f9fa; padding: 1rem; border-radius: 4px; margin-
bottom: 1rem; }
    </style>
</head>
<body>
    <nav class="navbar">
        <div class="nav-content">
            <h1>在线判题系统</h1>
            <div class="nav-links">
                <a href="/">首页</a>
                <a href="/problems">题目</a>
                <a href="/contests">比赛</a>
                <a href="/submissions">提交记录</a>
                <a href="/users">用户</a>
                <a href="/profile" id="profile-link">个人资料</a>
            </div>
        </div>
    </nav>

    <div class="container">
        <div id="loading" class="card">
            <p>加载中...</p>
        </div>

        <div id="contest-content" style="display: none;">
            <div class="card">
                <div class="contest-header">
                    <h1 id="contest-title" class="contest-title">比赛标题</h1>
                    <div id="countdown" class="countdown" style="display: none;">
                        <div class="meta-label">剩余时间</div>
                        <div id="countdown-timer" class="countdown-timer">--:--:--
</div>
                    </div>
                    <div class="header-actions">
                        <a href="#" id="standings-link" class="btn">查看排名</a>
                        <a href="/contests" class="btn btn-secondary">返回列表</a>
                        <button id="register-btn" class="btn btn-success"
style="display: none;">报名参赛</button>
                        <button id="enter-btn" class="btn btn-success" style="display:
none;">进入比赛</button>
                    </div>
                </div>

                <div class="contest-meta" id="contest-meta">
                    <!-- 比赛信息将通过JavaScript动态加载 -->
                </div>

                <div id="contest-description">
                    <!-- 比赛描述将通过JavaScript动态加载 -->
```

```
                </div>
            </div>

            <!-- 管理员面板 -->
            <div id="admin-panel" class="card" style="display: none;">
                <h2 class="section-title">题目管理</h2>

                <div class="admin-actions">
                    <button id="manage-problems-btn" class="btn btn-primary">管理题目
</button>
                    <button id="generate-standings-btn" class="btn btn-warning">生成排
名</button>
                    <button id="refresh-problems-btn" class="btn btn-secondary">刷新题
目列表</button>
                </div>

                <!-- 添加题目表单 -->
                <div id="add-problem-form" style="display: none; background: #f8f9fa;
padding: 1rem; border-radius: 4px; margin-bottom: 1rem;">
                    <h3>添加题目到比赛</h3>
                    <div style="display: grid; grid-template-columns: 1fr auto auto;
gap: 0.5rem; align-items: end;">
                        <div>
                            <label style="display: block; margin-bottom: 0.25rem; font-
weight: bold;">选择题目</label>
                            <select id="problem-select" style="width: 100%; padding:
0.5rem; border: 1px solid #ddd; border-radius: 4px;">
                                <option value="">请选择题目</option>
                            </select>
                        </div>
                        <div>
                            <label style="display: block; margin-bottom: 0.25rem; font-
weight: bold;">题目索引</label>
                            <input type="text" id="problem-index" placeholder="A"
                                   style="padding: 0.5rem; border: 1px solid #ddd;
border-radius: 4px; width: 80px;"
                                   maxlength="2" value="A">
                        </div>
                        <div>
                            <button id="add-problem-btn" class="btn btn-success">添加
</button>
                            <button id="cancel-add-btn" class="btn btn-secondary">取消
</button>
                        </div>
                    </div>
                </div>

                <!-- 当前比赛题目列表 -->
                <div id="contest-problems-admin">
                    <h3>当前比赛题目</h3>
                    <table class="problems-table">
                        <thead>
                            <tr>
```

```
                            <th width="80">索引</th>
                            <th width="100">题目ID</th>
                            <th>标题</th>
                            <th width="100">难度</th>
                            <th width="100">操作</th>
                        </tr>
                    </thead>
                    <tbody id="admin-problems-table-body">
                        <!-- 管理员视角的题目列表 -->
                    </tbody>
                </table>
            </div>
        </div>

        <div class="card">
            <h2 class="section-title">比赛题目</h2>
            <div id="problems-list">
                <table class="problems-table">
                    <thead>
                        <tr>
                            <th width="100">题目ID</th>
                            <th>标题</th>
                            <th width="100">难度</th>
                            <th width="120">时间限制</th>
                            <th width="120">内存限制</th>
                        </tr>
                    </thead>
                    <tbody id="problems-table-body">
                        <!-- 题目数据将通过JavaScript动态加载 -->
                    </tbody>
                </table>
            </div>
        </div>
    </div>

    <div id="error-message" class="card" style="display: none; background: #f8d7da;
color: #721c24;">
        <h2>错误</h2>
        <p id="error-text"></p>
        <a href="/contests" class="btn btn-secondary">返回比赛列表</a>
    </div>
</div>

<script>
    // 从URL路径中获取比赛ID
    function getContestId() {
        const pathParts = window.location.pathname.split('/').filter(part => part);
        // URL格式可能是 /contest/123 或 /contest/123/
        for (let i = 0; i < pathParts.length; i++) {
            if (pathParts[i] === 'contest' && i + 1 < pathParts.length) {
                const contestId = pathParts[i + 1];
                // 验证contestId是否为数字
                if (contestId && contestId.match(/^\d+$/)) {
```

```
                        return contestId;
                    }
                }
            }
            return null;
        }

        let contestId = getContestId();
        let contest = null;
        let countdownInterval = null;

        function getPhaseText(phase) {
            const phaseMap = {
                'BEFORE': '未开始',
                'CODING': '进行中',
                'PENDING_SYSTEM_TEST': '等待系统测试',
                'SYSTEM_TEST': '系统测试中',
                'FINISHED': '已结束'
            };
            return phaseMap[phase] || phase;
        }

        function getPhaseClass(phase) {
            const phaseClassMap = {
                'BEFORE': 'phase-before',
                'CODING': 'phase-coding',
                'FINISHED': 'phase-finished'
            };
            return phaseClassMap[phase] || '';
        }

        function getTypeText(type) {
            const typeMap = {
                'CF': 'Codeforces式',
                'IOI': 'IOI式',
                'ICPC': 'ICPC式'
            };
            return typeMap[type] || type;
        }

        function formatDuration(seconds) {
            const hours = Math.floor(seconds / 3600);
            const minutes = Math.floor((seconds % 3600) / 60);
            return `${hours}小时${minutes}分钟`;
        }

        function getDifficultyClass(difficulty) {
            if (!difficulty) return 'difficulty-easy';
            if (difficulty === '简单' || difficulty === 'Easy') return 'difficulty-
easy';
            if (difficulty === '中等' || difficulty === 'Medium') return 'difficulty-
medium';
            if (difficulty === '困难' || difficulty === 'Hard') return 'difficulty-
```

```
hard';
            return 'difficulty-easy';
        }

        function updateCountdown(endTime) {
            const end = new Date(endTime).getTime();
            const now = new Date().getTime();
            const distance = end - now;

            if (distance < 0) {
                document.getElementById('countdown').style.display = 'none';
                if (countdownInterval) {
                    clearInterval(countdownInterval);
                }
                return;
            }

            const hours = Math.floor(distance / (1000 * 60 * 60));
            const minutes = Math.floor((distance % (1000 * 60 * 60)) / (1000 * 60));
            const seconds = Math.floor((distance % (1000 * 60)) / 1000);

            document.getElementById('countdown-timer').textContent =
                `${hours.toString().padStart(2, '0')}:${minutes.toString().padStart(2,
'0')}:${seconds.toString().padStart(2, '0')}`;
        }

        async function loadContestDetail() {
            if (!contestId) {
                showError('比赛ID不存在或格式错误');
                return;
            }

            try {
                const response = await fetch(`/api/contest/${contestId}`);
                const result = await response.json();

                if (result.success) {
                    contest = result.contest;
                    displayContestDetail(contest);
                    loadContestProblems();
                } else {
                    showError(result.message);
                }
            } catch (error) {
                showError('加载比赛详情失败: ' + error.message);
            }
        }

        function displayContestDetail(contest) {
            document.getElementById('loading').style.display = 'none';
            document.getElementById('contest-content').style.display = 'block';

            // 更新标题
```

```
            document.getElementById('contest-title').textContent = contest.name;
            document.title = `${contest.name} - 在线判题系统`;

            // 更新排名链接
            document.getElementById('standings-link').href =
`/contest/${contest.contest_id}/standings`;

            // 更新比赛信息
            const contestMeta = document.getElementById('contest-meta');
            contestMeta.innerHTML = `
                <div class="meta-item">
                    <span class="meta-label">比赛阶段</span>
                    <span class="meta-value
${getPhaseClass(contest.phase)}">${getPhaseText(contest.phase)}</span>
                </div>
                <div class="meta-item">
                    <span class="meta-label">比赛类型</span>
                    <span class="meta-value">${getTypeText(contest.type)}</span>
                </div>
                <div class="meta-item">
                    <span class="meta-label">开始时间</span>
                    <span class="meta-value">${contest.start_time}</span>
                </div>
                <div class="meta-item">
                    <span class="meta-label">结束时间</span>
                    <span class="meta-value">${contest.end_time}</span>
                </div>
                <div class="meta-item">
                    <span class="meta-label">持续时间</span>
                    <span class="meta-value">${formatDuration(contest.duration)}</span>
                </div>
                ${contest.difficulty ? `
                <div class="meta-item">
                    <span class="meta-label">难度评级</span>
                    <span class="meta-value">${contest.difficulty}</span>
                </div>
                ` : ''}
                ${contest.kind ? `
                <div class="meta-item">
                    <span class="meta-label">比赛种类</span>
                    <span class="meta-value">${contest.kind}</span>
                </div>
                ` : ''}
                ${contest.icpc_region ? `
                <div class="meta-item">
                    <span class="meta-label">区域</span>
                    <span class="meta-value">${contest.icpc_region}</span>
                </div>
                ` : ''}
                ${contest.country ? `
                <div class="meta-item">
                    <span class="meta-label">国家</span>
                    <span class="meta-value">${contest.country}</span>
```

```
                </div>
                ` : ''}
                ${contest.city ? `
                <div class="meta-item">
                    <span class="meta-label">城市</span>
                    <span class="meta-value">${contest.city}</span>
                </div>
                ` : ''}
            `;

            // 更新比赛描述
            const descriptionContainer = document.getElementById('contest-
description');
            if (contest.description && contest.description.trim() !== '') {
                descriptionContainer.innerHTML = `
                    <h3 class="section-title">比赛描述</h3>
                    <div class="description-content">${contest.description}</div>
                `;
            } else {
                descriptionContainer.innerHTML = '';
            }

            // 设置按钮状态和倒计时
            if (contest.phase === 'CODING') {
                document.getElementById('countdown').style.display = 'block';
                document.getElementById('enter-btn').style.display = 'inline-block';
                // 立即更新一次倒计时
                updateCountdown(contest.end_time);
                // 设置定时器每秒更新
                countdownInterval = setInterval(() =>
updateCountdown(contest.end_time), 1000);
            } else if (contest.phase === 'BEFORE') {
                document.getElementById('register-btn').style.display = 'inline-block';
            }
        }

        async function loadContestProblems() {
            try {
                const response = await fetch(`/api/contest/${contestId}/problems`);
                const result = await response.json();

                if (result.success) {
                    displayContestProblems(result.problems);
                } else {
                    console.error('加载比赛题目失败:', result.message);
                    displayContestProblems([]);
                }
            } catch (error) {
                console.error('加载比赛题目时发生错误:', error);
                displayContestProblems([]);
            }
        }
```

```
        function displayContestProblems(problems) {
            const tbody = document.getElementById('problems-table-body');

            if (!problems || problems.length === 0) {
                tbody.innerHTML = `
                    <tr>
                        <td colspan="5" class="empty-state">
                            暂无题目数据
                        </td>
                    </tr>
                `;
                return;
            }

            tbody.innerHTML = '';
            problems.forEach(problem => {
                const row = document.createElement('tr');
                row.innerHTML = `
                    <td>${problem.problem_id || 'N/A'}</td>
                    <td>
                        <a href="/problem/${problem.problem_id}" class="problem-link">
                            ${problem.title || '未知题目'}
                        </a>
                    </td>
                    <td class="${getDifficultyClass(problem.difficulty)}">
                        ${problem.difficulty || '简单'}
                    </td>
                    <td>${problem.time_limit || 1000}ms</td>
                    <td>${problem.memory_limit ? Math.floor(problem.memory_limit /
1024) : 256}MB</td>
                `;
                tbody.appendChild(row);
            });
        }

        function showError(message) {
            document.getElementById('loading').style.display = 'none';
            document.getElementById('contest-content').style.display = 'none';
            document.getElementById('error-message').style.display = 'block';
            document.getElementById('error-text').textContent = message;

            // 清理定时器
            if (countdownInterval) {
                clearInterval(countdownInterval);
            }
        }

        // 检查管理员权限并显示管理面板
        async function checkAdminPermission() {
            try {
                const response = await fetch('/api/user/current');
                const result = await response.json();
```

```
                if (result.success && result.user && result.user.is_admin) {
                    document.getElementById('admin-panel').style.display = 'block';
                    loadAvailableProblems();
                }
            } catch (error) {
                console.error('检查管理员权限失败:', error);
            }
        }

        // 加载可用的题目列表
        async function loadAvailableProblems() {
            try {
                const response = await
fetch(`/api/contest/${contestId}/available_problems`);
                const result = await response.json();

                if (result.success) {
                    displayAvailableProblems(result.problems);
                } else {
                    console.error('加载可用题目失败:', result.message);
                displayAvailableProblems([]);
                document.getElementById('admin-problems-table-body').innerHTML = `
                    <tr>
                        <td colspan="5" class="empty-state">
                            加载题目失败: ${result.message}
                        </td>
                    </tr>
                `;
                }
            } catch (error) {
                console.error('加载可用题目时发生错误:', error);
                displayAvailableProblems([]);
                document.getElementById('admin-problems-table-body').innerHTML = `
                    <tr>
                        <td colspan="5" class="empty-state">
                            加载题目时发生错误
                        </td>
                    </tr>
                `;
            }
        }

        // 显示可用的题目列表
        function displayAvailableProblems(problems) {
            const select = document.getElementById('problem-select');
            select.innerHTML = '<option value="">请选择题目</option>';

            problems.forEach(problem => {
                const option = document.createElement('option');
                option.value = problem.problem_id;
                option.textContent = `${problem.problem_id} - ${problem.title}
(${problem.difficulty})`;
                option.disabled = problem.in_contest; // 已在比赛中的题目禁用
```

```
                if (problem.in_contest) {
                    option.textContent += ' [已在比赛中]';
                }
                select.appendChild(option);
            });

            // 更新管理员视角的题目列表
            const adminTbody = document.getElementById('admin-problems-table-body');
            const contestProblems = problems.filter(p => p.in_contest);

            if (contestProblems.length === 0) {
                adminTbody.innerHTML = `
                    <tr>
                        <td colspan="5" class="empty-state">
                            暂无题目，请添加题目到比赛
                        </td>
                    </tr>
                `;
                return;
            }

            adminTbody.innerHTML = '';
            contestProblems.forEach(problem => {
                const row = document.createElement('tr');
                row.innerHTML = `
                    <td>${problem.problem_index || 'A'}</td>
                    <td>${problem.problem_id}</td>
                    <td>
                        <a href="/problem/${problem.problem_id}" class="problem-link">
                            ${problem.title}
                        </a>
                    </td>
                    <td class="${getDifficultyClass(problem.difficulty)}">
                        ${problem.difficulty || '简单'}
                    </td>
                    <td>
                        <button class="btn btn-danger"
onclick="removeProblemFromContest('${problem.problem_id}')">
                            移除
                        </button>
                    </td>
                `;
                adminTbody.appendChild(row);
            });
        }

        // 添加题目到比赛
        async function addProblemToContest() {
            const problemId = document.getElementById('problem-select').value;
            const problemIndex = document.getElementById('problem-index').value.trim();

            if (!problemId) {
                alert('请选择题目');
```

```
                return;
            }

            if (!problemIndex) {
                alert('请输入题目索引');
                return;
            }

            try {
                const response = await fetch(`/api/contest/${contestId}/add_problem`, {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    },
                    body: JSON.stringify({
                        problem_id: problemId,
                        problem_index: problemIndex
                    })
                });

                const result = await response.json();

                if (result.success) {
                    alert('题目添加成功');
                    // 刷新题目列表
                    loadAvailableProblems();
                    loadContestProblems(); // 刷新普通用户看到的题目列表
                    // 隐藏添加表单
                    document.getElementById('add-problem-form').style.display = 'none';
                    // 重置表单
                    document.getElementById('problem-select').value = '';
                    document.getElementById('problem-index').value = 'A';
                } else {
                    alert('添加题目失败: ' + result.message);
                }
            } catch (error) {
                console.error('添加题目失败:', error);
                alert('添加题目失败，请检查网络连接');
            }
        }

        // 从比赛中移除题目
        async function removeProblemFromContest(problemId) {
            if (!confirm('确定要从比赛中移除这个题目吗？')) {
                return;
            }

            try {
                const response = await
fetch(`/api/contest/${contestId}/remove_problem`, {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
```

```
                },
                body: JSON.stringify({
                    problem_id: problemId
                })
            });

            const result = await response.json();

            if (result.success) {
                alert('题目移除成功');
                // 刷新题目列表
                loadAvailableProblems();
                loadContestProblems(); // 刷新普通用户看到的题目列表
            } else {
                alert('移除题目失败: ' + result.message);
            }
        } catch (error) {
            console.error('移除题目失败:', error);
            alert('移除题目失败，请检查网络连接');
        }
    }

    // 报名参赛
    document.getElementById('register-btn').addEventListener('click', function() {
        if (confirm('确定要报名参加这个比赛吗？')) {
            registerForContest();
        }
    });

    // 进入比赛
    document.getElementById('enter-btn').addEventListener('click', function() {
        // 对于进行中的比赛，进入比赛就是刷新页面查看最新状态
        location.reload();
    });

    // 报名参赛的API调用
    async function registerForContest() {
        try {
            // 这里调用报名API
            const response = await fetch(`/api/contest/${contestId}/register`, {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json'
                }
            });

            if (response.ok) {
                const result = await response.json();
                if (result.success) {
                    alert('报名成功！');
                    document.getElementById('register-btn').disabled = true;
                    document.getElementById('register-btn').textContent = '已报名';
                } else {
```

```
                        alert('报名失败: ' + result.message);
                    }
                } else {
                    alert('报名失败，请稍后重试');
                }
            } catch (error) {
                console.error('报名失败:', error);
                alert('报名失败，请检查网络连接');
            }
        }

        // 检查用户登录状态
        async function checkLoginStatus() {
            try {
                const response = await fetch('/api/user/current');
                const result = await response.json();

                if (!result.success) {
                    // 用户未登录，隐藏报名和进入按钮
                    document.getElementById('register-btn').style.display = 'none';
                    document.getElementById('enter-btn').style.display = 'none';
                }
            } catch (error) {
                console.error('检查登录状态失败:', error);
            }
        }

        // 页面加载时执行
        document.addEventListener('DOMContentLoaded', function() {
            checkLoginStatus();
            loadContestDetail();
            checkAdminPermission();
        });

        // 页面卸载时清理定时器
        window.addEventListener('beforeunload', function() {
            if (countdownInterval) {
                clearInterval(countdownInterval);
            }
        });

        // 添加事件监听器
        document.getElementById('manage-problems-btn').addEventListener('click',
function() {
            const form = document.getElementById('add-problem-form');
            form.style.display = form.style.display === 'none' ? 'block' : 'none';
        });

        document.getElementById('refresh-problems-btn').addEventListener('click',
function() {
            loadAvailableProblems();
        });
```

```
        document.getElementById('add-problem-btn').addEventListener('click',
addProblemToContest);

        document.getElementById('cancel-add-btn').addEventListener('click', function()
{
            document.getElementById('add-problem-form').style.display = 'none';
            document.getElementById('problem-select').value = '';
            document.getElementById('problem-index').value = 'A';
        });

        // 生成排名
        document.getElementById('generate-standings-btn').addEventListener('click',
async function() {
            if (!confirm('确定要生成比赛排名吗？这将根据提交记录重新计算排名。')) {
                return;
            }

            try {
                const response = await
fetch(`/api/contest/${contestId}/generate_standings`, {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    }
                });

                const result = await response.json();

                if (result.success) {
                    alert('排名生成成功！');
                    // 刷新排名页面
                    window.location.href = `/contest/${contestId}/standings`;
                } else {
                    alert('生成排名失败: ' + result.message);
                }
            } catch (error) {
                console.error('生成排名失败:', error);
                alert('生成排名失败，请检查网络连接');
            }
        });

    </script>
</body>
</html>
```

templates\contest_standingshtml

to top

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>比赛排名 - 在线判题系统</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; background: #f5f5f5; }
        .navbar { background: #2c3e50; color: white; padding: 1rem; }
        .nav-content { max-width: 1200px; margin: 0 auto; display: flex; justify-
content: space-between; }
        .nav-links a { color: white; text-decoration: none; margin-left: 1rem; }
        .container { max-width: 1200px; margin: 2rem auto; padding: 0 1rem; }
        .card { background: white; padding: 1.5rem; border-radius: 8px; box-shadow: 0
2px 4px rgba(0,0,0,0.1); margin-bottom: 1rem; }
        .header-row { display: flex; justify-content: space-between; align-items:
center; margin-bottom: 1rem; }
        .btn { background: #3498db; color: white; border: none; padding: 0.5rem 1rem;
border-radius: 4px; cursor: pointer; text-decoration: none; display: inline-block; }
        .btn:hover { background: #2980b9; }
        .btn-secondary { background: #95a5a6; }
        .btn-secondary:hover { background: #7f8c8d; }
        .btn-success { background: #27ae60; }
        .btn-success:hover { background: #219a52; }
        .btn-warning { background: #f39c12; }
        .btn-warning:hover { background: #e67e22; }
        .standings-table { width: 100%; border-collapse: collapse; font-size: 0.9rem; }
        .standings-table th, .standings-table td { padding: 0.75rem; text-align:
center; border-bottom: 1px solid #ddd; }
        .standings-table th { background: #f8f9fa; font-weight: bold; position: sticky;
top: 0; }
        .standings-table tr:hover { background: #f8f9fa; }
        .rank-1 { background: #fff9c4 !important; }
        .rank-2 { background: #f5f5f5 !important; }
        .rank-3 { background: #ffecb3 !important; }
        .user-handle { color: #3498db; text-decoration: none; font-weight: 500; }
        .user-handle:hover { color: #2980b9; text-decoration: underline; }
        .rating-high { color: #e74c3c; font-weight: bold; }
        .rating-medium { color: #e67e22; font-weight: bold; }
        .rating-low { color: #27ae60; font-weight: bold; }
        .rating-newbie { color: #95a5a6; }
        .accepted { color: #27ae60; font-weight: bold; }
        .wrong-answer { color: #e74c3c; }
        .rating-positive { color: #27ae60; }
        .rating-negative { color: #e74c3c; }
        .rating-zero { color: #95a5a6; }
        .pagination { display: flex; justify-content: center; gap: 0.5rem; margin-top:
1rem; }
        .page-btn { padding: 0.5rem 1rem; border: 1px solid #ddd; background: white;
cursor: pointer; }
        .page-btn.active { background: #3498db; color: white; border-color: #3498db; }
```

```
        .page-btn:hover:not(.active) { background: #f8f9fa; }
        .contest-phase { padding: 0.25rem 0.5rem; border-radius: 4px; font-size:
0.875rem; font-weight: bold; }
        .phase-before { background: #fff3cd; color: #856404; }
        .phase-coding { background: #d4edda; color: #155724; }
        .phase-finished { background: #e2e3e5; color: #383d41; }
        .admin-actions { margin-bottom: 1rem; }
    </style>
</head>
<body>
    <nav class="navbar">
        <div class="nav-content">
            <h1>在线判题系统</h1>
            <div class="nav-links">
                <a href="/">首页</a>
                <a href="/problems">题目</a>
                <a href="/contests">比赛</a>
                <a href="/submissions">提交记录</a>
                <a href="/users">用户</a>
                <a href="/profile" id="profile-link">个人资料</a>
            </div>
        </div>
    </nav>

    <div class="container">
        <div id="loading" class="card">
            <p>加载排名中...</p>
        </div>

        <div id="standings-content" style="display: none;">
            <div class="card">
                <div class="header-row">
                    <h1 id="contest-title">比赛排名</h1>
                    <div>
                        <a href="#" id="contest-detail-link" class="btn btn-secondary">
返回比赛详情</a>
                        <button id="refresh-standings-btn" class="btn btn-success"
style="margin-left: 0.5rem;">刷新排名</button>
                    </div>
                </div>

                <div class="contest-info" id="contest-info">
                    <!-- 比赛信息将通过JavaScript动态加载 -->
                </div>

                <!-- 管理员操作 -->
                <div id="admin-actions" class="admin-actions" style="display: none;">
                    <button id="generate-standings-btn" class="btn btn-warning">重新生
成排名</button>
                    <small style="color: #666; margin-left: 1rem;">注意：重新生成排名将
根据提交记录重新计算</small>
                </div>
            </div>
```

```
            <div class="card">
                <div id="standings-table-container" style="overflow-x: auto;">
                    <table class="standings-table">
                        <thead>
                            <tr>
                                <th width="60">排名</th>
                                <th>用户</th>
                                <th width="100">评分</th>
                                <th width="80">解题数</th>
                                <th width="100">罚时</th>
                                <th width="100">得分</th>
                                <th width="100">Rating变化</th>
                            </tr>
                        </thead>
                        <tbody id="standings-table-body">
                            <!-- 排名数据将通过JavaScript动态加载 -->
                        </tbody>
                    </table>
                </div>

                <div id="pagination" class="pagination">
                    <!-- 分页按钮将通过JavaScript动态生成 -->
                </div>
            </div>
        </div>

        <div id="error-message" class="card" style="display: none; background: #f8d7da;
color: #721c24;">
            <h2>错误</h2>
            <p id="error-text"></p>
            <a href="/contests" class="btn btn-secondary">返回比赛列表</a>
        </div>
    </div>

    <script>
        // 从URL路径中获取比赛ID
        function getContestId() {
            const pathParts = window.location.pathname.split('/').filter(part => part);
            // URL格式可能是 /contest/123/standings 或 /contest/123/standings/
            for (let i = 0; i < pathParts.length; i++) {
                if (pathParts[i] === 'contest' && i + 1 < pathParts.length) {
                    const contestId = pathParts[i + 1];
                    // 验证contestId是否为数字
                    if (contestId && contestId.match(/^\d+$/)) {
                        return contestId;
                    }
                }
            }
            return null;
        }

        let contestId = getContestId();
```

```
        let contest = null;
        let standings = [];
        let currentPage = 1;
        const pageSize = 50;

        function getPhaseText(phase) {
            const phaseMap = {
                'BEFORE': '未开始',
                'CODING': '进行中',
                'PENDING_SYSTEM_TEST': '等待系统测试',
                'SYSTEM_TEST': '系统测试中',
                'FINISHED': '已结束'
            };
            return phaseMap[phase] || phase;
        }

        function getPhaseClass(phase) {
            const phaseClassMap = {
                'BEFORE': 'phase-before',
                'CODING': 'phase-coding',
                'FINISHED': 'phase-finished'
            };
            return phaseClassMap[phase] || '';
        }

        function getRatingClass(rating) {
            if (!rating) return 'rating-newbie';
            if (rating >= 2400) return 'rating-high';
            if (rating >= 1800) return 'rating-medium';
            if (rating >= 1200) return 'rating-low';
            return 'rating-newbie';
        }

        function formatDuration(seconds) {
            const hours = Math.floor(seconds / 3600);
            const minutes = Math.floor((seconds % 3600) / 60);
            return `${hours}小时${minutes}分钟`;
        }

        function formatPenalty(penalty) {
            if (!penalty) return '0';
            const minutes = Math.floor(penalty / 60);
            return `${minutes}`;
        }

        function formatRatingChange(ratingChange) {
            if (ratingChange === null || ratingChange === undefined) return '-';
            if (ratingChange > 0) return `<span class="rating-
positive">+${ratingChange}</span>`;
            if (ratingChange < 0) return `<span class="rating-negative">${ratingChange}
</span>`;
            return '<span class="rating-zero">0</span>';
        }
```

```
        async function loadContestInfo() {
            if (!contestId) {
                showError('比赛ID不存在或格式错误');
                return;
            }

            try {
                const response = await fetch(`/api/contest/${contestId}`);
                const result = await response.json();

                if (result.success) {
                    contest = result.contest;
                    displayContestInfo(contest);
                } else {
                    showError(result.message);
                }
            } catch (error) {
                showError('加载比赛信息失败: ' + error.message);
            }
        }

        function displayContestInfo(contest) {
            document.getElementById('contest-title').textContent = `${contest.name} -
排名`;
            document.title = `${contest.name} - 排名 - 在线判题系统`;

            // 更新返回链接
            document.getElementById('contest-detail-link').href =
`/contest/${contest.contest_id}`;

            const contestInfo = document.getElementById('contest-info');
            contestInfo.innerHTML = `
                <div style="display: grid; grid-template-columns: repeat(auto-fit,
minmax(200px, 1fr)); gap: 1rem; margin-bottom: 1rem;">
                    <div>
                        <strong>比赛阶段:</strong>
                        <span class="contest-phase
${getPhaseClass(contest.phase)}">${getPhaseText(contest.phase)}</span>
                    </div>
                    <div><strong>开始时间:</strong> ${contest.start_time}</div>
                    <div><strong>结束时间:</strong> ${contest.end_time}</div>
                    <div><strong>持续时间:</strong> ${formatDuration(contest.duration)}
</div>
                </div>
            `;
        }

        async function loadStandings() {
            try {
                const response = await fetch(`/api/contest/${contestId}/standings?
page=${currentPage}&page_size=${pageSize}`);
                const result = await response.json();
```

```
                if (result.success) {
                    standings = result.standings;
                    displayStandings(standings);
                    setupPagination(result.total_count);
                } else {
                    showError(result.message);
                }
            } catch (error) {
                showError('加载排名失败: ' + error.message);
            }
        }

        function displayStandings(standings) {
            const tbody = document.getElementById('standings-table-body');

            if (!standings || standings.length === 0) {
                tbody.innerHTML = `
                    <tr>
                        <td colspan="7" style="text-align: center; padding: 2rem;
color: #666;">
                            暂无排名数据
                        </td>
                    </tr>
                `;
                return;
            }

            tbody.innerHTML = '';
            standings.forEach((row, index) => {
                const rank = (currentPage - 1) * pageSize + index + 1;
                const tr = document.createElement('tr');
                if (rank <= 3) {
                    tr.className = `rank-${rank}`;
                }

                tr.innerHTML = `
                    <td>${rank}</td>
                    <td style="text-align: left;">
                        <a href="/user/${row.user_id}" class="user-handle
${getRatingClass(row.rating)}">
                            ${row.username}
                        </a>
                        ${row.is_me ? '<span style="color: #e74c3c; margin-left:
0.5rem;">(我)</span>' : ''}
                    </td>
                    <td class="${getRatingClass(row.rating)}">${row.rating || '-'}</td>
                    <td>${row.solved_count || 0}</td>
                    <td>${formatPenalty(row.penalty)}</td>
                    <td>${row.total_score || 0}</td>
                    <td>${formatRatingChange(row.rating_change)}</td>
                `;
                tbody.appendChild(tr);
```

```
            });
        }

        function setupPagination(totalCount) {
            const totalPages = Math.ceil(totalCount / pageSize);
            const pagination = document.getElementById('pagination');

            if (totalPages <= 1) {
                pagination.innerHTML = '';
                return;
            }

            let paginationHTML = '';

            // 上一页按钮
            if (currentPage > 1) {
                paginationHTML += `<button class="page-btn"
onclick="changePage(${currentPage - 1})">上一页</button>`;
            }

            // 页码按钮
            const startPage = Math.max(1, currentPage - 2);
            const endPage = Math.min(totalPages, currentPage + 2);

            if (startPage > 1) {
                paginationHTML += `<button class="page-btn"
onclick="changePage(1)">1</button>`;
                if (startPage > 2) {
                    paginationHTML += `<span style="padding: 0.5rem;">...</span>`;
                }
            }

            for (let i = startPage; i <= endPage; i++) {
                paginationHTML += `<button class="page-btn ${i === currentPage ?
'active' : ''}" onclick="changePage(${i})">${i}</button>`;
            }

            if (endPage < totalPages) {
                if (endPage < totalPages - 1) {
                    paginationHTML += `<span style="padding: 0.5rem;">...</span>`;
                }
                paginationHTML += `<button class="page-btn"
onclick="changePage(${totalPages})">${totalPages}</button>`;
            }

            // 下一页按钮
            if (currentPage < totalPages) {
                paginationHTML += `<button class="page-btn"
onclick="changePage(${currentPage + 1})">下一页</button>`;
            }

            pagination.innerHTML = paginationHTML;
        }
```

```
        function changePage(page) {
            currentPage = page;
            loadStandings();
            // 滚动到顶部
            window.scrollTo(0, 0);
        }

        function showError(message) {
            document.getElementById('loading').style.display = 'none';
            document.getElementById('standings-content').style.display = 'none';
            document.getElementById('error-message').style.display = 'block';
            document.getElementById('error-text').textContent = message;
        }

        // 检查管理员权限
        async function checkAdminPermission() {
            try {
                const response = await fetch('/api/user/current');
                const result = await response.json();

                if (result.success && result.user && result.user.is_admin) {
                    document.getElementById('admin-actions').style.display = 'block';
                }
            } catch (error) {
                console.error('检查管理员权限失败:', error);
            }
        }

        // 页面加载时执行
        document.addEventListener('DOMContentLoaded', function() {
            if (!contestId) {
                showError('比赛ID不存在或格式错误');
                return;
            }

            loadContestInfo();
            loadStandings();
            checkAdminPermission();

            // 隐藏加载提示，显示内容
            setTimeout(() => {
                document.getElementById('loading').style.display = 'none';
                document.getElementById('standings-content').style.display = 'block';
            }, 500);
        });

        // 刷新排名
        document.getElementById('refresh-standings-btn').addEventListener('click',
function() {
            loadStandings();
        });
```

```
        // 重新生成排名（管理员功能）
        document.getElementById('generate-standings-btn').addEventListener('click',
async function() {
            if (!confirm('确定要重新生成排名吗？这将根据提交记录重新计算排名。')) {
                return;
            }

            try {
                const response = await
fetch(`/api/contest/${contestId}/generate_standings`, {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    }
                });

                const result = await response.json();

                if (result.success) {
                    alert('排名生成成功！');
                    // 刷新排名
                    loadStandings();
                } else {
                    alert('生成排名失败: ' + result.message);
                }
            } catch (error) {
                console.error('生成排名失败:', error);
                alert('生成排名失败，请检查网络连接');
            }
        });

    </script>
</body>
</html>
```

templates\create_contesthtml

to top

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>创建比赛 - 在线判题系统</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; background: #f5f5f5; }
        .navbar { background: #2c3e50; color: white; padding: 1rem; }
        .nav-content { max-width: 1200px; margin: 0 auto; display: flex; justify-
content: space-between; }
        .nav-links a { color: white; text-decoration: none; margin-left: 1rem; }
        .container { max-width: 800px; margin: 2rem auto; padding: 0 1rem; }
        .card { background: white; padding: 2rem; border-radius: 8px; box-shadow: 0 2px
4px rgba(0,0,0,0.1); }
        .form-group { margin-bottom: 1.5rem; }
        label { display: block; margin-bottom: 0.5rem; font-weight: bold; color:
#2c3e50; }
        input, select, textarea { width: 100%; padding: 0.75rem; border: 1px solid
#ddd; border-radius: 4px; font-size: 1rem; }
        textarea { min-height: 120px; resize: vertical; }
        .btn { background: #3498db; color: white; border: none; padding: 0.75rem
1.5rem; border-radius: 4px; cursor: pointer; font-size: 1rem; }
        .btn:hover { background: #2980b9; }
        .btn-success { background: #27ae60; }
        .btn-success:hover { background: #219a52; }
        .btn-secondary { background: #95a5a6; }
        .btn-secondary:hover { background: #7f8c8d; }
        .form-section { border: 1px solid #e0e0e0; border-radius: 8px; padding: 1.5rem;
margin-bottom: 1.5rem; }
        .form-section h3 { margin-bottom: 1rem; color: #2c3e50; border-bottom: 2px
solid #3498db; padding-bottom: 0.5rem; }
        .problem-item { background: #f8f9fa; padding: 1rem; border-radius: 4px; margin-
bottom: 0.5rem; border: 1px solid #e0e0e0; }
        .problem-header { display: flex; justify-content: between; align-items: center;
margin-bottom: 0.5rem; }
        .problem-actions { display: flex; gap: 0.5rem; }
        .modal { position: fixed; top: 0; left: 0; width: 100%; height: 100%;
background: rgba(0,0,0,0.5); display: flex; justify-content: center; align-items:
center; z-index: 1000; }
        .modal-content { background: white; padding: 2rem; border-radius: 8px; width:
90%; max-width: 800px; max-height: 80vh; overflow-y: auto; }
        .problem-list { max-height: 400px; overflow-y: auto; margin: 1rem 0; }
        .problem-option { padding: 0.75rem; border: 1px solid #ddd; border-radius: 4px;
margin-bottom: 0.5rem; cursor: pointer; }
        .problem-option:hover { background: #f8f9fa; }
        .problem-option.selected { background: #e3f2fd; border-color: #3498db; }
        .search-box { margin-bottom: 1rem; }
        .empty-state { text-align: center; padding: 2rem; color: #666; }
        table { width: 100%; border-collapse: collapse; margin: 1rem 0; }
        th, td { padding: 0.75rem; text-align: left; border-bottom: 1px solid #ddd; }
```

```
        th { background: #f8f9fa; font-weight: bold; }
    </style>
</head>
<body>
    <nav class="navbar">
        <div class="nav-content">
            <h1>在线判题系统</h1>
            <div class="nav-links">
                <a href="/">首页</a>
                <a href="/problems">题目</a>
                <a href="/contests">比赛</a>
                <a href="/submissions">提交记录</a>
                <a href="/users">用户</a>
                <a href="/profile">个人资料</a>
            </div>
        </div>
    </nav>

    <div class="container">
        <div class="card">
            <h2 style="margin-bottom: 1.5rem; color: #2c3e50;">创建新比赛</h2>

            <form id="contest-form">
                <div class="form-section">
                    <h3>基本信息</h3>

                    <div class="form-group">
                        <label for="contest-name">比赛名称 *</label>
                        <input type="text" id="contest-name" name="name" required>
                    </div>

                    <div class="form-group">
                        <label for="contest-type">比赛类型 *</label>
                        <select id="contest-type" name="type" required>
                            <option value="CF">Codeforces式</option>
                            <option value="IOI">IOI式</option>
                            <option value="ICPC">ICPC式</option>
                        </select>
                    </div>

                    <div class="form-group">
                        <label for="contest-phase">比赛阶段</label>
                        <select id="contest-phase" name="phase">
                            <option value="BEFORE">未开始</option>
                            <option value="CODING">进行中</option>
                            <option value="FINISHED">已结束</option>
                        </select>
                    </div>

                    <div class="form-group">
                        <label for="start-time">开始时间 *</label>
                        <input type="datetime-local" id="start-time" name="start_time"
required>
```

```
                    </div>

                    <div class="form-group">
                        <label for="duration">持续时间（秒） *</label>
                        <input type="number" id="duration" name="duration_seconds"
value="7200" min="1800" step="900" required>
                        <small style="color: #666; font-size: 0.875rem;">例如：7200秒 =
2小时</small>
                    </div>
                </div>

                <div class="form-section">
                    <h3>比赛描述</h3>

                    <div class="form-group">
                        <label for="description">描述</label>
                        <textarea id="description" name="description" placeholder="输入
比赛描述..."></textarea>
                    </div>

                    <div class="form-group">
                        <label for="difficulty">难度评级</label>
                        <input type="number" id="difficulty" name="difficulty" min="0"
max="10" step="1">
                    </div>

                    <div class="form-group">
                        <label for="kind">比赛种类</label>
                        <input type="text" id="kind" name="kind" placeholder="例如：常规
比赛、训练赛等">
                    </div>
                </div>

                <div class="form-section">
                    <h3>比赛题目</h3>

                    <div id="contest-problems-selection">
                        <div style="margin-bottom: 1rem;">
                            <button type="button" id="add-problem-btn" class="btn btn-
secondary">选择题目</button>
                            <span style="margin-left: 1rem; color: #666;" id="selected-
count">已选择 0 个题目</span>
                        </div>

                        <div id="selected-problems-list" class="empty-state">
                            尚未选择任何题目
                        </div>
                    </div>
                </div>

                <div class="form-group" style="text-align: center; margin-top: 2rem;">
                    <button type="submit" class="btn btn-success" style="padding: 1rem
2rem;">创建比赛</button>
```

```
                    <a href="/contests" class="btn btn-secondary" style="margin-left:
1rem;">取消</a>
                </div>
            </form>
        </div>
    </div>

    <script>
        let selectedProblems = [];
        let availableProblems = [];

        // 加载可用的题目
        async function loadAvailableProblems() {
            try {
                const response = await fetch('/api/problems');
                const result = await response.json();

                if (result.success) {
                    availableProblems = result.problems;
                } else {
                    alert('加载题目列表失败: ' + result.message);
                }
            } catch (error) {
                alert('加载题目列表时发生错误: ' + error.message);
            }
        }

        // 显示题目选择模态框
        function showProblemSelectionModal() {
            const modal = document.createElement('div');
            modal.className = 'modal';
            modal.innerHTML = `
                <div class="modal-content">
                    <h3>选择题目</h3>
                    <div class="search-box">
                        <input type="text" id="problem-search" placeholder="搜索题目名称
或ID..."
                               style="width: 100%; padding: 0.75rem; border: 1px solid
#ddd; border-radius: 4px;">
                    </div>
                    <div class="problem-list" id="problem-selection-list">
                        <!-- 题目列表将通过JavaScript动态加载 -->
                    </div>
                    <div style="display: flex; justify-content: space-between; align-
items: center; margin-top: 1rem;">
                        <span id="selected-problems-count">已选择 0 个题目</span>
                        <div>
                            <button id="confirm-selection" class="btn btn-success">确认
选择</button>
                            <button id="cancel-selection" class="btn btn-secondary"
style="margin-left: 0.5rem;">取消</button>
                        </div>
                    </div>
```

```
                </div>
            `;

            document.body.appendChild(modal);
            displayProblemSelectionList();

            // 事件监听器
            document.getElementById('problem-search').addEventListener('input',
function(e) {
                filterProblems(e.target.value);
            });

            document.getElementById('confirm-selection').addEventListener('click',
function() {
                document.body.removeChild(modal);
                updateSelectedProblemsDisplay();
            });

            document.getElementById('cancel-selection').addEventListener('click',
function() {
                document.body.removeChild(modal);
            });
        }

        // 显示题目选择列表
        function displayProblemSelectionList(filteredProblems = null) {
            const container = document.getElementById('problem-selection-list');
            const problems = filteredProblems || availableProblems;

            if (problems.length === 0) {
                container.innerHTML = '<div class="empty-state">暂无题目</div>';
                return;
            }

            container.innerHTML = '';
            problems.forEach(problem => {
                const isSelected = selectedProblems.some(p => p.problem_id ===
problem.problem_id);

                const problemDiv = document.createElement('div');
                problemDiv.className = `problem-option ${isSelected ? 'selected' :
''}`;
                problemDiv.innerHTML = `
                    <div style="display: flex; align-items: center; gap: 1rem;">
                        <input type="checkbox" ${isSelected ? 'checked' : ''}

onchange="toggleProblemSelection('${problem.problem_id}', this.checked)"
                               style="width: auto;">
                        <div style="flex: 1;">
                            <div style="font-weight: bold;">${problem.problem_id} -
${problem.title}</div>
                            <div style="font-size: 0.875rem; color: #666;">
                                难度: ${problem.difficulty} | 时间限制:
```

```
${problem.time_limit}ms | 内存限制: ${Math.floor(problem.memory_limit / 1024)}MB
                            </div>
                        </div>
                    </div>
                `;
                container.appendChild(problemDiv);
            });

            updateSelectionCount();
        }

        // 过滤题目
        function filterProblems(searchTerm) {
            if (!availableProblems) return;

            const filtered = availableProblems.filter(problem =>
                problem.title.toLowerCase().includes(searchTerm.toLowerCase()) ||
                problem.problem_id.toLowerCase().includes(searchTerm.toLowerCase())
            );

            displayProblemSelectionList(filtered);
        }

        // 切换题目选择
        function toggleProblemSelection(problemId, isSelected) {
            const problem = availableProblems.find(p => p.problem_id === problemId);

            if (isSelected) {
                if (!selectedProblems.some(p => p.problem_id === problemId)) {
                    selectedProblems.push({
                        ...problem,
                        index: String.fromCharCode(65 + selectedProblems.length) // A,
B, C, ...
                    });
                }
            } else {
                selectedProblems = selectedProblems.filter(p => p.problem_id !==
problemId);
                // 重新分配索引
                selectedProblems.forEach((problem, index) => {
                    problem.index = String.fromCharCode(65 + index);
                });
            }

            displayProblemSelectionList();
            updateSelectionCount();
        }

        // 更新选择计数
        function updateSelectionCount() {
            const countElement = document.getElementById('selected-problems-count');
            if (countElement) {
                countElement.textContent = `已选择 ${selectedProblems.length} 个题目`;
```

```
            }
        }

        // 更新已选择题目的显示
        function updateSelectedProblemsDisplay() {
            const container = document.getElementById('selected-problems-list');
            const countElement = document.getElementById('selected-count');

            countElement.textContent = `已选择 ${selectedProblems.length} 个题目`;

            if (selectedProblems.length === 0) {
                container.innerHTML = '<div class="empty-state">尚未选择任何题目</div>';
                return;
            }

            container.innerHTML = `
                <table>
                    <thead>
                        <tr>
                            <th width="80">索引</th>
                            <th width="120">题目ID</th>
                            <th>标题</th>
                            <th width="100">难度</th>
                            <th width="100">操作</th>
                        </tr>
                    </thead>
                    <tbody>
                        ${selectedProblems.map(problem => `
                            <tr>
                                <td>
                                    <input type="text" value="${problem.index}"

onchange="updateProblemIndex('${problem.problem_id}', this.value)"
                                           style="width: 40px; text-align: center;
padding: 0.25rem; border: 1px solid #ddd; border-radius: 2px;">
                                </td>
                                <td>${problem.problem_id}</td>
                                <td>${problem.title}</td>
                                <td>${problem.difficulty}</td>
                                <td>
                                    <button type="button"
onclick="removeSelectedProblem('${problem.problem_id}')"
                                            class="btn btn-secondary" style="padding:
0.25rem 0.5rem; font-size: 0.875rem;">
                                        移除
                                    </button>
                                </td>
                            </tr>
                        `).join('')}
                    </tbody>
                </table>
            `;
        }
```

```
        // 更新题目索引
        function updateProblemIndex(problemId, newIndex) {
            const problem = selectedProblems.find(p => p.problem_id === problemId);
            if (problem) {
                problem.index = newIndex;
            }
        }

        // 移除已选择的题目
        function removeSelectedProblem(problemId) {
            selectedProblems = selectedProblems.filter(p => p.problem_id !==
problemId);
            // 重新分配索引
            selectedProblems.forEach((problem, index) => {
                problem.index = String.fromCharCode(65 + index);
            });
            updateSelectedProblemsDisplay();
        }

        // 处理表单提交
        document.getElementById('contest-form').addEventListener('submit', async
function(e) {
            e.preventDefault();

            const formData = new FormData(this);
            const contestData = {
                name: formData.get('name'),
                type: formData.get('type'),
                phase: formData.get('phase'),
                start_time: formData.get('start_time'),
                duration_seconds: parseInt(formData.get('duration_seconds')),
                description: formData.get('description'),
                difficulty: formData.get('difficulty') ?
parseInt(formData.get('difficulty')) : 0,
                kind: formData.get('kind')
            };

            // 验证必填字段
            if (!contestData.name || !contestData.start_time ||
!contestData.duration_seconds) {
                alert('请填写所有必填字段');
                return;
            }

            try {
                const response = await fetch('/api/contests/create', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    },
                    body: JSON.stringify(contestData)
                });
```

```
                const result = await response.json();

                if (result.success) {
                    alert('比赛创建成功! ');
                    // 如果有选择的题目，添加到比赛
                    if (selectedProblems.length > 0) {
                        await addProblemsToContest(result.contest_id);
                    }
                    window.location.href = `/contest/${result.contest_id}`;
                } else {
                    alert('创建比赛失败: ' + result.message);
                }
            } catch (error) {
                alert('创建比赛时发生错误: ' + error.message);
            }
        });

        // 添加题目到新创建的比赛
        async function addProblemsToContest(contestId) {
            for (const problem of selectedProblems) {
                try {
                    const response = await
fetch(`/api/contest/${contestId}/add_problem`, {
                        method: 'POST',
                        headers: {
                            'Content-Type': 'application/json'
                        },
                        body: JSON.stringify({
                            problem_id: problem.problem_id,
                            problem_index: problem.index
                        })
                    });

                    const result = await response.json();
                    if (!result.success) {
                        console.error(`添加题目 ${problem.problem_id} 失败:`,
result.message);
                    }
                } catch (error) {
                    console.error(`添加题目 ${problem.problem_id} 时发生错误:`, error);
                }
            }
        }

        // 设置默认开始时间（当前时间+1小时）
        function setDefaultStartTime() {
            const now = new Date();
            now.setHours(now.getHours() + 1);
            now.setMinutes(0);
            now.setSeconds(0);

            const year = now.getFullYear();
```

```
            const month = String(now.getMonth() + 1).padStart(2, '0');
            const day = String(now.getDate()).padStart(2, '0');
            const hours = String(now.getHours()).padStart(2, '0');
            const minutes = String(now.getMinutes()).padStart(2, '0');

            document.getElementById('start-time').value =
`${year}-${month}-${day}T${hours}:${minutes}`;
        }

        // 初始化
        document.addEventListener('DOMContentLoaded', function() {
            loadAvailableProblems();
            setDefaultStartTime();
            document.getElementById('add-problem-btn').addEventListener('click',
showProblemSelectionModal);
        });
    </script>
</body>
</html>
```

templates\create_problemhtml

to top

```
<!-- templates/create_problem.html -->
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>创建题目 - 在线判题系统</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; background: #f5f5f5; }
        .navbar { background: #2c3e50; color: white; padding: 1rem; }
        .nav-content { max-width: 1200px; margin: 0 auto; display: flex; justify-
content: space-between; }
        .nav-links a { color: white; text-decoration: none; margin-left: 1rem; }
        .container { max-width: 1200px; margin: 2rem auto; padding: 0 1rem; }
        .card { background: white; padding: 1.5rem; border-radius: 8px; box-shadow: 0
2px 4px rgba(0,0,0,0.1); margin-bottom: 1rem; }
        .form-group { margin-bottom: 1rem; }
        label { display: block; margin-bottom: 0.5rem; font-weight: bold; }
        input, select, textarea { width: 100%; padding: 0.5rem; border: 1px solid #ddd;
border-radius: 4px; }
        textarea { min-height: 100px; resize: vertical; }
        .btn { background: #3498db; color: white; border: none; padding: 0.5rem 1rem;
border-radius: 4px; cursor: pointer; margin-right: 0.5rem; }
        .btn:hover { background: #2980b9; }
        .btn-secondary { background: #95a5a6; }
        .btn-secondary:hover { background: #7f8c8d; }
        .test-case { border: 1px solid #ddd; padding: 1rem; margin-bottom: 1rem;
border-radius: 4px; }
        .test-case-header { display: flex; justify-content: space-between; align-items:
center; margin-bottom: 0.5rem; }
        .tag-input { display: flex; align-items: center; margin-bottom: 0.5rem; }
        .tag-input input { flex: 1; margin-right: 0.5rem; }
        .tags-container { display: flex; flex-wrap: wrap; gap: 0.5rem; margin-top:
0.5rem; }
        .tag { background: #e1ecf4; color: #39739d; padding: 0.2rem 0.5rem; border-
radius: 3px; font-size: 0.9rem; }
        .remove-tag { margin-left: 0.3rem; cursor: pointer; }
        .error { color: #dc3545; font-size: 0.9rem; margin-top: 0.2rem; }
    </style>
</head>
<body>
    <nav class="navbar">
        <div class="nav-content">
            <h1>在线判题系统</h1>
            <div class="nav-links">
                <a href="/">首页</a>
                <a href="/problems">题目</a>
                <a href="/contests">比赛</a>
                <a href="/submissions">提交记录</a>
                <a href="/users">用户</a>
                <div id="auth-buttons">
```

```
                    <div id="user-info" style="display: none;">
                        <span id="user-handle" style="color: white; margin-right:
1rem;"></span>
                        <a href="/profile" style="color: white; text-decoration: none;
margin-right: 1rem;">个人资料</a>
                        <button onclick="logout()" style="background: #e74c3c; color:
white; border: none; padding: 0.3rem 0.8rem; border-radius: 4px; cursor: pointer;">登出
</button>
                    </div>
                    <div id="guest-buttons">
                        <button onclick="showLogin()">登录</button>
                        <button onclick="showRegister()">注册</button>
                    </div>
                </div>
            </div>
        </div>
    </nav>

    <div class="container">
        <div class="card">
            <h2>创建题目</h2>
            <form id="create-problem-form">
                <div class="form-group">
                    <label for="problem-id">题目ID *</label>
                    <input type="text" id="problem-id" name="problem_id" required>
                    <div class="error" id="problem-id-error"></div>
                </div>

                <div class="form-group">
                    <label for="title">题目标题 *</label>
                    <input type="text" id="title" name="title" required>
                    <div class="error" id="title-error"></div>
                </div>

                <div class="form-group">
                    <label for="statement">题目描述 *</label>
                    <textarea id="statement" name="statement" required></textarea>
                    <div class="error" id="statement-error"></div>
                </div>

                <div class="form-group">
                    <label for="input-specification">输入格式</label>
                    <textarea id="input-specification" name="input_specification">
</textarea>
                </div>

                <div class="form-group">
                    <label for="output-specification">输出格式</label>
                    <textarea id="output-specification" name="output_specification">
</textarea>
                </div>

                <div class="form-group">
```

```
                    <label for="time-limit">时间限制 (ms) *</label>
                    <input type="number" id="time-limit" name="time_limit" value="1000"
min="100" required>
                    <div class="error" id="time-limit-error"></div>
                </div>

                <div class="form-group">
                    <label for="memory-limit">内存限制 (KB) *</label>
                    <input type="number" id="memory-limit" name="memory_limit"
value="256000" min="1000" required>
                    <div class="error" id="memory-limit-error"></div>
                </div>

                <div class="form-group">
                    <label for="difficulty">难度</label>
                    <select id="difficulty" name="difficulty">
                        <option value="简单">简单</option>
                        <option value="中等">中等</option>
                        <option value="困难">困难</option>
                    </select>
                </div>

                <div class="form-group">
                    <label>标签</label>
                    <div class="tag-input">
                        <input type="text" id="tag-input" placeholder="输入标签">
                        <button type="button" class="btn" onclick="addTag()">添加
</button>
                    </div>
                    <div class="tags-container" id="tags-container"></div>
                    <input type="hidden" id="tags" name="tags">
                </div>

                <div class="form-group">
                    <label>测试用例</label>
                    <div id="test-cases">
                        <div class="test-case">
                            <div class="test-case-header">
                                <span>测试用例 #1</span>
                                <button type="button" class="btn btn-secondary"
onclick="removeTestCase(this)">删除</button>
                            </div>
                            <label>输入</label>
                            <textarea name="test_input" placeholder="输入数据">
</textarea>
                            <label>期望输出</label>
                            <textarea name="test_output" placeholder="期望输出">
</textarea>
                            <label>
                                <input type="checkbox" name="is_sample"> 作为样例显示
                            </label>
                        </div>
                    </div>
```

```
                    <button type="button" class="btn" onclick="addTestCase()">添加测试用例</button>
                </div>

                <div class="form-group">
                    <label for="notes">备注</label>
                    <textarea id="notes" name="notes"></textarea>
                </div>

                <div class="form-group">
                    <button type="submit" class="btn">创建题目</button>
                    <button type="button" class="btn btn-secondary" onclick="window.location.href='/problems'">取消</button>
                </div>
            </form>
        </div>
    </div>

    <!-- 登录模态框 -->
    <div id="login-modal" style="display: none; position: fixed; top: 0; left: 0; width: 100%; height: 100%; background: rgba(0,0,0,0.5);">
        <div style="background: white; width: 300px; margin: 100px auto; padding: 2rem; border-radius: 8px;">
            <h3>登录</h3>
            <input type="text" id="login-handle" placeholder="用户名" style="width: 100%; margin: 0.5rem 0; padding: 0.5rem;">
            <input type="password" id="login-password" placeholder="密码" style="width: 100%; margin: 0.5rem 0; padding: 0.5rem;">
            <button onclick="login()" class="btn">登录</button>
            <button onclick="hideLogin()" class="btn" style="background: #95a5a6;">取消</button>
        </div>
    </div>

    <!-- 注册模态框 -->
    <div id="register-modal" style="display: none; position: fixed; top: 0; left: 0; width: 100%; height: 100%; background: rgba(0,0,0,0.5);">
        <div style="background: white; width: 300px; margin: 100px auto; padding: 2rem; border-radius: 8px;">
            <h3>注册</h3>
            <input type="text" id="register-handle" placeholder="用户名" style="width: 100%; margin: 0.5rem 0; padding: 0.5rem;">
            <input type="email" id="register-email" placeholder="邮箱" style="width: 100%; margin: 0.5rem 0; padding: 0.5rem;">
            <input type="text" id="register-name" placeholder="姓名（可选）" style="width: 100%; margin: 0.5rem 0; padding: 0.5rem;">
            <input type="password" id="register-password" placeholder="密码" style="width: 100%; margin: 0.5rem 0; padding: 0.5rem;">
            <button onclick="register()" class="btn">注册</button>
            <button onclick="hideRegister()" class="btn" style="background: #95a5a6;">取消</button>
        </div>
    </div>
```

```
    <script>
        let tags = [];
        let testCaseCount = 1;

        // 检查登录状态
        function checkLoginStatus() {
            const userInfo = document.getElementById('user-info');
            const guestButtons = document.getElementById('guest-buttons');
            const userHandle = document.getElementById('user-handle');

            fetch('/api/user/current')
                .then(response => response.json())
                .then(data => {
                    if (data.success && data.user) {
                        userHandle.textContent = data.user.handle;
                        userInfo.style.display = 'block';
                        guestButtons.style.display = 'none';

                        // 检查是否是管理员
                        if (!data.user.is_admin) {
                            alert('只有管理员可以创建题目');
                            window.location.href = '/problems';
                        }
                    } else {
                        userInfo.style.display = 'none';
                        guestButtons.style.display = 'block';
                        alert('请先登录');
                        window.location.href = '/problems';
                    }
                })
                .catch(() => {
                    userInfo.style.display = 'none';
                    guestButtons.style.display = 'block';
                    alert('请先登录');
                    window.location.href = '/problems';
                });
        }

        // 标签管理
        function addTag() {
            const tagInput = document.getElementById('tag-input');
            const tag = tagInput.value.trim();

            if (tag && !tags.includes(tag)) {
                tags.push(tag);
                updateTagsDisplay();
                tagInput.value = '';
            }
        }

        function removeTag(tag) {
            tags = tags.filter(t => t !== tag);
```

```
            updateTagsDisplay();
        }

        function updateTagsDisplay() {
            const container = document.getElementById('tags-container');
            const hiddenInput = document.getElementById('tags');

            container.innerHTML = '';
            tags.forEach(tag => {
                const tagElement = document.createElement('div');
                tagElement.className = 'tag';
                tagElement.innerHTML = `
                    ${tag}
                    <span class="remove-tag" onclick="removeTag('${tag}')">×</span>
                `;
                container.appendChild(tagElement);
            });

            hiddenInput.value = JSON.stringify(tags);
        }

        // 测试用例管理
        function addTestCase() {
            testCaseCount++;
            const testCasesContainer = document.getElementById('test-cases');
            const newTestCase = document.createElement('div');
            newTestCase.className = 'test-case';
            newTestCase.innerHTML = `
                <div class="test-case-header">
                    <span>测试用例 #${testCaseCount}</span>
                    <button type="button" class="btn btn-secondary"
onclick="removeTestCase(this)">删除</button>
                </div>
                <label>输入</label>
                <textarea name="test_input" placeholder="输入数据"></textarea>
                <label>期望输出</label>
                <textarea name="test_output" placeholder="期望输出"></textarea>
                <label>
                    <input type="checkbox" name="is_sample"> 作为样例显示
                </label>
            `;
            testCasesContainer.appendChild(newTestCase);
        }

        function removeTestCase(button) {
            if (testCaseCount > 1) {
                const testCase = button.closest('.test-case');
                testCase.remove();
                testCaseCount--;
                // 重新编号
                const testCases = document.querySelectorAll('.test-case');
                testCases.forEach((testCase, index) => {
                    testCase.querySelector('.test-case-header span').textContent = `测
```

```
试用例 #${index + 1}`;
                });
            } else {
                alert('至少需要一个测试用例');
            }
        }

        // 表单提交
        document.getElementById('create-problem-form').addEventListener('submit', async
function(e) {
            e.preventDefault();

            const formData = new FormData(this);
            const data = {
                problem_id: formData.get('problem_id'),
                title: formData.get('title'),
                statement: formData.get('statement'),
                input_specification: formData.get('input_specification'),
                output_specification: formData.get('output_specification'),
                time_limit: parseInt(formData.get('time_limit')),
                memory_limit: parseInt(formData.get('memory_limit')),
                difficulty: formData.get('difficulty'),
                notes: formData.get('notes'),
                tags: JSON.parse(formData.get('tags') || '[]')
            };

            // 收集测试用例
            data.test_cases = [];
            const testCases = document.querySelectorAll('.test-case');
            testCases.forEach((testCase, index) => {
                const inputs = testCase.querySelectorAll('textarea');
                const checkbox = testCase.querySelector('input[type="checkbox"]');
                data.test_cases.push({
                    input: inputs[0].value,
                    output: inputs[1].value,
                    is_sample: checkbox.checked,
                    test_order: index + 1
                });
            });

            // 验证必填字段
            const errors = validateForm(data);
            if (Object.keys(errors).length > 0) {
                displayErrors(errors);
                return;
            }

            try {
                const response = await fetch('/api/problems/create', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    },
```

```
                body: JSON.stringify(data)
            });

            const result = await response.json();
            if (result.success) {
                alert('题目创建成功! ');
                window.location.href = `/problem/${data.problem_id}`;
            } else {
                alert('创建失败: ' + result.message);
            }
        } catch (error) {
            alert('创建失败: ' + error.message);
        }
    });

    function validateForm(data) {
        const errors = {};

        if (!data.problem_id.trim()) {
            errors.problem_id = '题目ID不能为空';
        }

        if (!data.title.trim()) {
            errors.title = '题目标题不能为空';
        }

        if (!data.statement.trim()) {
            errors.statement = '题目描述不能为空';
        }

        if (data.time_limit < 100) {
            errors.time_limit = '时间限制不能小于100ms';
        }

        if (data.memory_limit < 1000) {
            errors.memory_limit = '内存限制不能小于1000KB';
        }

        if (data.test_cases.length === 0) {
            alert('至少需要一个测试用例');
        }

        return errors;
    }

    function displayErrors(errors) {
        // 清除所有错误显示
        document.querySelectorAll('.error').forEach(el => el.textContent = '');

        // 显示新的错误
        Object.keys(errors).forEach(field => {
            const errorElement = document.getElementById(`${field}-error`);
            if (errorElement) {
```

```
                errorElement.textContent = errors[field];
            }
        });
    }

    // 登录相关函数（与problems.html相同）
    function showLogin() {
        document.getElementById('login-modal').style.display = 'block';
    }

    function hideLogin() {
        document.getElementById('login-modal').style.display = 'none';
    }

    function showRegister() {
        document.getElementById('register-modal').style.display = 'block';
    }

    function hideRegister() {
        document.getElementById('register-modal').style.display = 'none';
    }

    async function login() {
        const handle = document.getElementById('login-handle').value;
        const password = document.getElementById('login-password').value;

        if (!handle || !password) {
            alert('请填写用户名和密码');
            return;
        }

        try {
            const response = await fetch('/api/login', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ handle, password })
            });

            const result = await response.json();
            alert(result.message);
            if (result.success) {
                hideLogin();
                location.reload();
            }
        } catch (error) {
            alert('登录失败: ' + error.message);
        }
    }

    async function register() {
        const handle = document.getElementById('register-handle').value;
        const email = document.getElementById('register-email').value;
        const name = document.getElementById('register-name').value;
```

```
            const password = document.getElementById('register-password').value;

            if (!handle || !email || !password) {
                alert('请填写必填信息');
                return;
            }

            try {
                const response = await fetch('/api/register', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ handle, email, name, password })
                });

                const result = await response.json();
                alert(result.message);
                if (result.success) {
                    hideRegister();
                    const loginResponse = await fetch('/api/login', {
                        method: 'POST',
                        headers: { 'Content-Type': 'application/json' },
                        body: JSON.stringify({ handle, password })
                    });

                    const loginResult = await loginResponse.json();
                    if (loginResult.success) {
                        location.reload();
                    }
                }
            } catch (error) {
                alert('注册失败: ' + error.message);
            }
        }

        async function logout() {
            try {
                const response = await fetch('/api/logout');
                const result = await response.json();
                alert(result.message);
                if (result.success) {
                    location.reload();
                }
            } catch (error) {
                alert('登出失败: ' + error.message);
            }
        }

        // 页面加载时执行
        document.addEventListener('DOMContentLoaded', function() {
            checkLoginStatus();
        });
    </script>
```

```
</body>
</html>
```

templates\indexhtml

to top

```
<!-- templates/index.html -->
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>在线判题系统</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; background: #f5f5f5; }
        .navbar { background: #2c3e50; color: white; padding: 1rem; }
        .nav-content { max-width: 1200px; margin: 0 auto; display: flex; justify-
content: space-between; }
        .nav-links a { color: white; text-decoration: none; margin-left: 1rem; }
        .container { max-width: 1200px; margin: 2rem auto; padding: 0 1rem; }
        .card { background: white; padding: 1.5rem; border-radius: 8px; box-shadow: 0
2px 4px rgba(0,0,0,0.1); margin-bottom: 1rem; }
        .btn { background: #3498db; color: white; padding: 0.5rem 1rem; border: none;
border-radius: 4px; cursor: pointer; }
        .btn:hover { background: #2980b9; }
    </style>
</head>
<body>
    <nav class="navbar">
        <div class="nav-content">
            <h1>在线判题系统</h1>
            <div class="nav-links">
                <a href="/">首页</a>
                <a href="/problems">题目</a>
                <a href="/contests">比赛</a>
                <a href="/submissions">提交记录</a>
                <a href="/users">用户</a>
                <a href="/profile" id="profile-link">个人资料</a>
                <div id="auth-buttons">
                    <div id="user-info" style="display: none;">
                        <span id="user-handle" style="color: white; margin-right:
1rem;"></span>
                        <button onclick="logout()">登出</button>
                    </div>
                    <div id="guest-buttons">
                        <button onclick="showLogin()">登录</button>
                        <button onclick="showRegister()">注册</button>
                    </div>
                </div>
            </div>
        </div>
    </nav>

    <div class="container">
        <div class="card">
            <h2>欢迎来到在线判题系统</h2>
            <p>这是一个模仿 Codeforces 的在线编程评测平台</p>
```

```
        </div>
    </div>

    <!-- 登录模态框 -->
    <div id="login-modal" style="display: none; position: fixed; top: 0; left: 0;
width: 100%; height: 100%; background: rgba(0,0,0,0.5);">
        <div style="background: white; width: 300px; margin: 100px auto; padding: 2rem;
border-radius: 8px;">
            <h3>登录</h3>
            <input type="text" id="login-handle" placeholder="用户名" style="width:
100%; margin: 0.5rem 0; padding: 0.5rem;">
            <input type="password" id="login-password" placeholder="密码" style="width:
100%; margin: 0.5rem 0; padding: 0.5rem;">
            <button onclick="login()" class="btn">登录</button>
            <button onclick="hideLogin()" class="btn" style="background: #95a5a6;">取消
</button>
        </div>
    </div>

    <!-- 注册模态框 -->
    <div id="register-modal" style="display: none; position: fixed; top: 0; left: 0;
width: 100%; height: 100%; background: rgba(0,0,0,0.5);">
        <div style="background: white; width: 300px; margin: 100px auto; padding: 2rem;
border-radius: 8px;">
            <h3>注册</h3>
            <input type="text" id="register-handle" placeholder="用户名" style="width:
100%; margin: 0.5rem 0; padding: 0.5rem;">
            <input type="email" id="register-email" placeholder="邮箱" style="width:
100%; margin: 0.5rem 0; padding: 0.5rem;">
            <input type="password" id="register-password" placeholder="密码"
style="width: 100%; margin: 0.5rem 0; padding: 0.5rem;">
            <input type="text" id="register-name" placeholder="姓名（可选）"
style="width: 100%; margin: 0.5rem 0; padding: 0.5rem;">
            <button onclick="register()" class="btn">注册</button>
            <button onclick="hideRegister()" class="btn" style="background: #95a5a6;">
取消</button>
        </div>
    </div>

    <script>
        function showLogin() { document.getElementById('login-modal').style.display =
'block'; }
        function hideLogin() { document.getElementById('login-modal').style.display =
'none'; }
        function showRegister() { document.getElementById('register-
modal').style.display = 'block'; }
        function hideRegister() { document.getElementById('register-
modal').style.display = 'none'; }

        async function login() {
            const handle = document.getElementById('login-handle').value;
            const password = document.getElementById('login-password').value;
```

```
        const response = await fetch('/api/login', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ handle, password })
        });

        const result = await response.json();
        alert(result.message);
        if (result.success) {
            hideLogin();
            location.reload();
        }
    }

    async function register() {
        const handle = document.getElementById('register-handle').value;
        const email = document.getElementById('register-email').value;
        const password = document.getElementById('register-password').value;
        const name = document.getElementById('register-name').value;

        const response = await fetch('/api/register', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ handle, email, password, name })
        });

        const result = await response.json();
        alert(result.message);
        if (result.success) {
            hideRegister();
        }
    }

    // 检查登录状态
    function checkLoginStatus() {
        const userInfo = document.getElementById('user-info');
        const guestButtons = document.getElementById('guest-buttons');
        const userHandle = document.getElementById('user-handle');

        // 这里应该从session或localStorage获取用户信息
        // 暂时使用简单的检查方式
        fetch('/api/user/current')
            .then(response => response.json())
            .then(data => {
                if (data.success && data.user) {
                    userHandle.textContent = data.user.handle;
                    userInfo.style.display = 'block';
                    guestButtons.style.display = 'none';
                } else {
                    userInfo.style.display = 'none';
                    guestButtons.style.display = 'block';
                }
            })
```

```
                .catch(() => {
                    userInfo.style.display = 'none';
                    guestButtons.style.display = 'block';
                });
        }

        // 登出功能
        async function logout() {
            const response = await fetch('/api/logout');
            const result = await response.json();
            alert(result.message);
            if (result.success) {
                location.reload();
            }
        }

        // 页面加载时检查登录状态
        document.addEventListener('DOMContentLoaded', checkLoginStatus);
    </script>
</body>
</html>
```

templates\index_logerhtml

to top

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>$Title$</title>
</head>
<body>
$END$
</body>
</html>
```

templates\problemshtml

to top

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>题目列表 - 在线判题系统</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; background: #f5f5f5; }
        .navbar { background: #2c3e50; color: white; padding: 1rem; }
        .nav-content { max-width: 1200px; margin: 0 auto; display: flex; justify-
content: space-between; }
        .nav-links a { color: white; text-decoration: none; margin-left: 1rem; }
        .container { max-width: 1200px; margin: 2rem auto; padding: 0 1rem; }
        .card { background: white; padding: 1.5rem; border-radius: 8px; box-shadow: 0
2px 4px rgba(0,0,0,0.1); margin-bottom: 1rem; }
        table { width: 100%; border-collapse: collapse; }
        th, td { padding: 0.75rem; text-align: left; border-bottom: 1px solid #ddd; }
        th { background: #f8f9fa; font-weight: bold; }
        .difficulty-easy { color: #28a745; }
        .difficulty-medium { color: #ffc107; }
        .difficulty-hard { color: #dc3545; }
        .problem-link { color: #3498db; text-decoration: none; font-weight: 500; }
        .problem-link:hover { color: #2980b9; text-decoration: underline; }
        .header-row { display: flex; justify-content: space-between; align-items:
center; margin-bottom: 1rem; }
        .btn { background: #3498db; color: white; border: none; padding: 0.5rem 1rem;
border-radius: 4px; cursor: pointer; text-decoration: none; display: inline-block; }
        .btn:hover { background: #2980b9; }
        .btn-create { background: #27ae60; }
        .btn-create:hover { background: #219a52; }
        #create-problem-btn { display: none; }
    </style>
</head>
<body>
    <nav class="navbar">
        <div class="nav-content">
            <h1>在线判题系统</h1>
            <div class="nav-links">
                <a href="/">首页</a>
                <a href="/problems">题目</a>
                <a href="/contests">比赛</a>
                <a href="/submissions">提交记录</a>
                <a href="/users">用户</a>
                <a href="/profile" id="profile-link">个人资料</a>
            </div>
        </div>
    </nav>

    <div class="container">
        <div class="card">
            <div class="header-row">
```

```
                <h2>题目列表</h2>
                <a href="/create-problem" class="btn btn-create" id="create-problem-
btn">创建题目</a>
            </div>
            <div id="problems-list">
                <table>
                    <thead>
                        <tr>
                            <th>题目ID</th>
                            <th>标题</th>
                            <th>难度</th>
                            <th>通过率</th>
                            <th>时间限制</th>
                            <th>内存限制</th>
                        </tr>
                    </thead>
                    <tbody id="problems-table-body">
                        <!-- 题目数据将通过JavaScript动态加载 -->
                    </tbody>
                </table>
            </div>
        </div>
    </div>

    <!-- 登录模态框 -->
    <div id="login-modal" style="display: none; position: fixed; top: 0; left: 0;
width: 100%; height: 100%; background: rgba(0,0,0,0.5);">
        <div style="background: white; width: 300px; margin: 100px auto; padding: 2rem;
border-radius: 8px;">
            <h3>登录</h3>
            <input type="text" id="login-handle" placeholder="用户名" style="width:
100%; margin: 0.5rem 0; padding: 0.5rem;">
            <input type="password" id="login-password" placeholder="密码" style="width:
100%; margin: 0.5rem 0; padding: 0.5rem;">
            <button onclick="login()" class="btn">登录</button>
            <button onclick="hideLogin()" class="btn" style="background: #95a5a6;">取消
</button>
        </div>
    </div>

    <!-- 注册模态框 -->
    <div id="register-modal" style="display: none; position: fixed; top: 0; left: 0;
width: 100%; height: 100%; background: rgba(0,0,0,0.5);">
        <div style="background: white; width: 300px; margin: 100px auto; padding: 2rem;
border-radius: 8px;">
            <h3>注册</h3>
            <input type="text" id="register-handle" placeholder="用户名" style="width:
100%; margin: 0.5rem 0; padding: 0.5rem;">
            <input type="email" id="register-email" placeholder="邮箱" style="width:
100%; margin: 0.5rem 0; padding: 0.5rem;">
            <input type="text" id="register-name" placeholder="姓名（可选）"
style="width: 100%; margin: 0.5rem 0; padding: 0.5rem;">
            <input type="password" id="register-password" placeholder="密码"
```

```
style="width: 100%; margin: 0.5rem 0; padding: 0.5rem;">
            <button onclick="register()" class="btn">注册</button>
            <button onclick="hideRegister()" class="btn" style="background: #95a5a6;">
取消</button>
        </div>
    </div>

    <script>
        // 检查登录状态
        function checkLoginStatus() {
            const createProblemBtn = document.getElementById('create-problem-btn');

            fetch('/api/user/current')
                .then(response => response.json())
                .then(data => {
                    if (data.success && data.user && data.user.is_admin) {
                        // 只有管理员才显示创建题目按钮
                        createProblemBtn.style.display = 'inline-block';
                    } else {
                        createProblemBtn.style.display = 'none';
                    }
                })
                .catch(() => {
                    createProblemBtn.style.display = 'none';
                });
        }

        // 登录相关函数
        function showLogin() {
            document.getElementById('login-modal').style.display = 'block';
        }

        function hideLogin() {
            document.getElementById('login-modal').style.display = 'none';
        }

        function showRegister() {
            document.getElementById('register-modal').style.display = 'block';
        }

        function hideRegister() {
            document.getElementById('register-modal').style.display = 'none';
        }

        async function login() {
            const handle = document.getElementById('login-handle').value;
            const password = document.getElementById('login-password').value;

            if (!handle || !password) {
                alert('请填写用户名和密码');
                return;
            }
```

```
        try {
            const response = await fetch('/api/login', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ handle, password })
            });

            const result = await response.json();
            alert(result.message);
            if (result.success) {
                hideLogin();
                location.reload();
            }
        } catch (error) {
            alert('登录失败: ' + error.message);
        }
    }

    async function register() {
        const handle = document.getElementById('register-handle').value;
        const email = document.getElementById('register-email').value;
        const name = document.getElementById('register-name').value;
        const password = document.getElementById('register-password').value;

        if (!handle || !email || !password) {
            alert('请填写必填信息');
            return;
        }

        try {
            const response = await fetch('/api/register', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ handle, email, name, password })
            });

            const result = await response.json();
            alert(result.message);
            if (result.success) {
                hideRegister();
                // 注册成功后自动登录
                const loginResponse = await fetch('/api/login', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ handle, password })
                });

                const loginResult = await loginResponse.json();
                if (loginResult.success) {
                    location.reload();
                }
            }
        } catch (error) {
```

```
                alert('注册失败: ' + error.message);
            }
        }

        // 登出功能
        async function logout() {
            try {
                const response = await fetch('/api/logout');
                const result = await response.json();
                alert(result.message);
                if (result.success) {
                    location.reload();
                }
            } catch (error) {
                alert('登出失败: ' + error.message);
            }
        }

        async function loadProblems() {
            try {
                const response = await fetch('/api/problems');
                const result = await response.json();

                if (result.success) {
                    const tbody = document.getElementById('problems-table-body');
                    tbody.innerHTML = '';

                    result.problems.forEach(problem => {
                        const passRate = problem.submission_count > 0
                            ? ((problem.accepted_count / problem.submission_count) *
100).toFixed(1) + '%'
                            : '0%';

                        const row = document.createElement('tr');
                        row.innerHTML = `
                            <td>${problem.problem_id}</td>
                            <td>
                                <a href="/problem/${problem.problem_id}"
class="problem-link">
                                    ${problem.title}
                                </a>
                            </td>
                            <td
class="difficulty-${getDifficultyClass(problem.difficulty)}">
                                ${problem.difficulty || '简单'}
                            </td>
                            <td>${problem.accepted_count}/${problem.submission_count}
(${passRate})</td>
                            <td>${problem.time_limit}ms</td>
                            <td>${Math.floor(problem.memory_limit / 1024)}MB</td>
                        `;
                        tbody.appendChild(row);
                    });
```

```
                } else {
                    alert('加载题目失败: ' + result.message);
                }
            } catch (error) {
                alert('加载题目时发生错误: ' + error.message);
            }
        }

        // 辅助函数：根据难度返回对应的CSS类名
        function getDifficultyClass(difficulty) {
            if (!difficulty) return 'easy';

            const difficultyMap = {
                '简单': 'easy',
                '中等': 'medium',
                '困难': 'hard',
                'Easy': 'easy',
                'Medium': 'medium',
                'Hard': 'hard'
            };

            return difficultyMap[difficulty] || 'easy';
        }

        // 页面加载时执行
        document.addEventListener('DOMContentLoaded', function() {
            checkLoginStatus();
            loadProblems();
        });
    </script>
</body>
</html>
```

templates\problem_detailhtml

to top

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>题目详情 - 在线评测系统</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css"
rel="stylesheet">
    <link
href="https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.7.0/styles/github.min.css"
rel="stylesheet">
    <style>
        .problem-header { background: #f8f9fa; padding: 20px; border-radius: 5px;
margin-bottom: 20px; }
        .tag { display: inline-block; background: #e9ecef; padding: 2px 8px; margin:
2px; border-radius: 3px; font-size: 0.9em; }
        .sample-test { background: #f8f9fa; padding: 15px; border-radius: 5px; margin:
10px 0; }
        .code-block { background: #f1f3f4; padding: 10px; border-radius: 3px; font-
family: monospace; white-space: pre-wrap; }
        .submit-btn { margin-top: 20px; }
        .loading { text-align: center; padding: 50px; }
    </style>
</head>
<body>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
        <div class="container">
            <a class="navbar-brand" href="/">Online Judge</a>
            <div class="navbar-nav ms-auto">
                <a class="nav-link" href="/">首页</a>
                <a class="nav-link" href="/problems">题目列表</a>
                <a class="nav-link" href="/contests">比赛</a>
                <a class="nav-link" href="/submissions">提交记录</a>
                <a class="nav-link" href="/profile">个人资料</a>
            </div>
        </div>
    </nav>

    <div class="container mt-4" id="problem-container">
        <!-- 加载中状态 -->
        <div class="loading" id="loading">
            <div class="spinner-border" role="status">
                <span class="visually-hidden">加载中...</span>
            </div>
            <p class="mt-2">正在加载题目详情...</p>
        </div>

        <!-- 题目内容（动态加载） -->
        <div id="problem-content" style="display: none;">
            <!-- 题目头部信息 -->
            <div class="problem-header">
```

```
                <h1 id="problem-title">题目标题</h1>
                <div class="problem-meta">
                    <span class="badge" id="difficulty-badge">难度</span>
                    <span>时间限制: <span id="time-limit">0</span>ms</span>
                    <span>内存限制: <span id="memory-limit">0</span>KB</span>
                    <span>通过: <span id="accepted-count">0</span></span>
                    <span>提交: <span id="submission-count">0</span></span>
                </div>
                <div class="tags mt-2" id="tags-container">
                    <!-- 标签动态加载 -->
                </div>
            </div>

            <!-- 题目内容 -->
            <div class="problem-content">
                <!-- 题目描述 -->
                <div class="card mb-3">
                    <div class="card-header">
                        <h5 class="mb-0">题目描述</h5>
                    </div>
                    <div class="card-body">
                        <div id="statement">题目描述加载中...</div>
                    </div>
                </div>

                <!-- 输入格式 -->
                <div class="card mb-3">
                    <div class="card-header">
                        <h5 class="mb-0">输入格式</h5>
                    </div>
                    <div class="card-body">
                        <div id="input-spec">输入格式加载中...</div>
                    </div>
                </div>

                <!-- 输出格式 -->
                <div class="card mb-3">
                    <div class="card-header">
                        <h5 class="mb-0">输出格式</h5>
                    </div>
                    <div class="card-body">
                        <div id="output-spec">输出格式加载中...</div>
                    </div>
                </div>

                <!-- 样例 -->
                <div class="card mb-3" id="samples-card" style="display: none;">
                    <div class="card-header">
                        <h5 class="mb-0">样例</h5>
                    </div>
                    <div class="card-body" id="samples-container">
                        <!-- 样例动态加载 -->
                    </div>
```

```
            </div>
        </div>

        <!-- 提交区域 -->
        <div class="card">
            <div class="card-header">
                <h5 class="mb-0">提交代码</h5>
            </div>
            <div class="card-body">
                <form id="submit-form">
                    <div class="mb-3">
                        <label for="language" class="form-label">编程语言</label>
                        <select class="form-select" id="language" name="language">
                            <option value="C++">C++</option>
                            <option value="Java">Java</option>
                            <option value="Python">Python</option>
                            <option value="C">C</option>
                            <option value="R">R</option>
                            <option value="Brain fuck">Brain fuck</option>
                            <option value="C#">C#</option>
                            <option value="D">D</option>
                            <option value="Go">Go</option>
                            <option value="Haskell">Haskell</option>
                            <option value="Kotlin">Kotlin</option>
                            <option value="OCaml">OCaml</option>
                            <option value="Pascal">Pascal</option>
                            <option value="Perl">Perl</option>
                            <option value="PHP">PHP</option>
                            <option value="Ruby">Ruby</option>
                            <option value="Rust">Rust</option>
                            <option value="Scala">Scala</option>
                            <option value="JavaScript">JavaScript</option>
                            <option value="Node.js">Node.js</option>
                            <option value="TypeScript">TypeScript</option>
                            <option value="F#">F#</option>
                            <option value="Clang">Clang</option>
                            <option value="Clang++">Clang++</option>
                            <option value="GNU C++11">GNU C++11</option>
                            <option value="GNU C++14">GNU C++14</option>
                            <option value="GNU C++17">GNU C++17</option>
                            <option value="GNU C++20">GNU C++20</option>
                            <option value="MS C++">MS C++</option>
                            <option value="MS C#">MS C#</option>
                            <option value="GNU C11">GNU C11</option>
                            <option value="Python 2">Python 2</option>
                            <option value="Python 3">Python 3</option>
                            <option value="PyPy 2">PyPy 2</option>
                            <option value="PyPy 3">PyPy 3</option>
                            <option value="PyPy 3-64">PyPy 3-64</option>
                            <option value="Java 8">Java 8</option>
                            <option value="Java 11">Java 11</option>
                            <option value="Java 17">Java 17</option>
                            <option value="Text">Text</option>
```

```
                            </select>
                        </div>
                        <div class="mb-3">
                            <label for="source-code" class="form-label">源代码</label>
                            <textarea class="form-control" id="source-code"
name="source_code" rows="15"
                                      placeholder="请在此处编写你的代码..." style="font-
family: 'Courier New', monospace;"></textarea>
                        </div>
                        <button type="submit" class="btn btn-primary submit-btn">提交
</button>
                    </form>
                </div>
            </div>
        </div>

        <!-- 错误信息 -->
        <div class="alert alert-danger" id="error-message" style="display: none;">
            <h4>加载失败</h4>
            <p id="error-text"></p>
            <a href="/problems" class="btn btn-secondary">返回题目列表</a>
        </div>
    </div>

    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js">
</script>
    <script
src="https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.7.0/highlight.min.js">
</script>
    <script>
        // 页面加载时获取题目数据
        document.addEventListener('DOMContentLoaded', function() {
            const problemId = window.location.pathname.split('/').pop();
            loadProblemDetail(problemId);

            // 提交表单处理
            document.getElementById('submit-form').addEventListener('submit',
function(e) {
                e.preventDefault();
                submitSolution(problemId);
            });
        });

        function loadProblemDetail(problemId) {
            fetch(`/api/problem/${problemId}`)
                .then(response => response.json())
                .then(data => {
                    if (data.success) {
                        displayProblem(data.problem);
                    } else {
                        showError(data.message || '获取题目详情失败');
                    }
```

```
                })
                .catch(error => {
                    console.error('Error:', error);
                    showError('网络错误，请稍后重试');
                });
        }

        function displayProblem(problem) {
            // 隐藏加载中，显示内容
            document.getElementById('loading').style.display = 'none';
            document.getElementById('problem-content').style.display = 'block';

            // 更新页面标题
            document.title = problem.title + ' - 在线评测系统';

            // 更新题目信息
            document.getElementById('problem-title').textContent = problem.title;
            document.getElementById('statement').innerHTML = problem.statement || '<p>
暂无题目描述</p>';
            document.getElementById('input-spec').innerHTML =
problem.input_specification || '<p>暂无输入格式说明</p>';
            document.getElementById('output-spec').innerHTML =
problem.output_specification || '<p>暂无输出格式说明</p>';
            document.getElementById('time-limit').textContent = problem.time_limit;
            document.getElementById('memory-limit').textContent = problem.memory_limit;
            document.getElementById('accepted-count').textContent =
problem.accepted_count || 0;
            document.getElementById('submission-count').textContent =
problem.submission_count || 0;

            // 设置难度
            const difficultyBadge = document.getElementById('difficulty-badge');
            const colors = {
                '简单': 'bg-success',
                '中等': 'bg-warning',
                '困难': 'bg-danger',
                'Easy': 'bg-success',
                'Medium': 'bg-warning',
                'Hard': 'bg-danger'
            };
            difficultyBadge.textContent = problem.difficulty || '简单';
            difficultyBadge.className = `badge ${colors[problem.difficulty] || 'bg-
secondary'}`;

            // 更新标签
            const tagsContainer = document.getElementById('tags-container');
            if (problem.tags && problem.tags.length > 0) {
                tagsContainer.innerHTML = '';
                problem.tags.forEach(tag => {
                    const tagElement = document.createElement('span');
                    tagElement.className = 'tag';
                    tagElement.textContent = tag;
                    tagsContainer.appendChild(tagElement);
```

```
            });
        } else {
            tagsContainer.innerHTML = '<span class="text-muted">暂无标签</span>';
        }

        // 更新样例
        const samplesContainer = document.getElementById('samples-container');
        const samplesCard = document.getElementById('samples-card');
        if (problem.sample_tests && problem.sample_tests.length > 0) {
            samplesCard.style.display = 'block';
            samplesContainer.innerHTML = '';

            problem.sample_tests.forEach((sample, index) => {
                const sampleElement = document.createElement('div');
                sampleElement.className = 'sample-test';
                sampleElement.innerHTML = `
                    <strong>样例 ${index + 1}:</strong>
                    <div class="mt-2">
                        <strong>输入:</strong>
                        <div class="code-block">${sample.input || ''}</div>
                    </div>
                    <div class="mt-2">
                        <strong>输出:</strong>
                        <div class="code-block">${sample.output || ''}</div>
                    </div>
                `;
                samplesContainer.appendChild(sampleElement);
            });

            // 渲染代码高亮
            document.querySelectorAll('.code-block').forEach(block => {
                hljs.highlightElement(block);
            });
        } else {
            samplesCard.style.display = 'none';
        }
    }

    function showError(message) {
        document.getElementById('loading').style.display = 'none';
        document.getElementById('error-message').style.display = 'block';
        document.getElementById('error-text').textContent = message;
    }

    function submitSolution(problemId) {
        const sourceCode = document.getElementById('source-code').value;
        const language = document.getElementById('language').value;

        if (!sourceCode.trim()) {
            alert('请填写代码');
            return;
        }
```

```
            fetch('/api/submit', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                },
                body: JSON.stringify({
                    problem_id: problemId,
                    source_code: sourceCode,
                    language: language
                })
            })
            .then(response => response.json())
            .then(data => {
                if (data.success) {
                    alert('提交成功! 提交ID: ' + data.submission_id);
                    window.location.href = '/submissions';
                } else {
                    alert('提交失败: ' + data.message);
                }
            })
            .catch(error => {
                console.error('Error:', error);
                alert('提交失败');
            });
        }
    </script>
</body>
</html>
```

templates\profilehtml

to top

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>个人资料 - 在线判题系统</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; background: #f5f5f5; }
        .navbar { background: #2c3e50; color: white; padding: 1rem; }
        .nav-content { max-width: 1200px; margin: 0 auto; display: flex; justify-
content: space-between; }
        .nav-links a { color: white; text-decoration: none; margin-left: 1rem; }
        .container { max-width: 800px; margin: 2rem auto; padding: 0 1rem; }
        .card { background: white; padding: 1.5rem; border-radius: 8px; box-shadow: 0
2px 4px rgba(0,0,0,0.1); margin-bottom: 1rem; }
        .profile-header { display: flex; align-items: center; margin-bottom: 2rem; }
        .avatar { width: 100px; height: 100px; border-radius: 50%; background: #3498db;
color: white; display: flex; align-items: center; justify-content: center; font-size:
2.5rem; font-weight: bold; margin-right: 2rem; }
        .user-info h1 { margin-bottom: 0.5rem; color: #2c3e50; }
        .user-meta { display: grid; grid-template-columns: repeat(auto-fit,
minmax(200px, 1fr)); gap: 1rem; margin-bottom: 2rem; }
        .meta-item { display: flex; flex-direction: column; }
        .meta-label { font-size: 0.875rem; color: #666; margin-bottom: 0.25rem; }
        .meta-value { font-weight: 500; }
        .section-title { border-bottom: 2px solid #3498db; padding-bottom: 0.5rem;
margin-bottom: 1rem; color: #2c3e50; }
        .form-group { margin-bottom: 1rem; }
        .form-label { display: block; margin-bottom: 0.5rem; font-weight: 500; color:
#333; }
        .form-input { width: 100%; padding: 0.75rem; border: 1px solid #ddd; border-
radius: 4px; font-size: 1rem; }
        .form-input:focus { outline: none; border-color: #3498db; }
        .btn { background: #3498db; color: white; border: none; padding: 0.75rem
1.5rem; border-radius: 4px; cursor: pointer; font-size: 1rem; transition: background
0.3s; }
        .btn:hover { background: #2980b9; }
        .btn-secondary { background: #95a5a6; }
        .btn-secondary:hover { background: #7f8c8d; }
        .btn-danger { background: #e74c3c; }
        .btn-danger:hover { background: #c0392b; }
        .tab-container { margin-bottom: 1rem; }
        .tabs { display: flex; border-bottom: 1px solid #ddd; margin-bottom: 1rem; }
        .tab { padding: 0.75rem 1.5rem; cursor: pointer; border-bottom: 3px solid
transparent; }
        .tab.active { border-bottom-color: #3498db; color: #3498db; font-weight: bold;
}
        .tab-content { display: none; }
        .tab-content.active { display: block; }
        .message { padding: 0.75rem; border-radius: 4px; margin-bottom: 1rem; }
        .message.success { background: #d4edda; color: #155724; border: 1px solid
```

```
#c3e6cb; }
        .message.error { background: #f8d7da; color: #721c24; border: 1px solid
#f5c6cb; }
        .rating-high { color: #e74c3c; }
        .rating-medium { color: #e67e22; }
        .rating-low { color: #27ae60; }
    </style>
</head>
<body>
    <nav class="navbar">
        <div class="nav-content">
            <h1>在线判题系统</h1>
            <div class="nav-links">
                <a href="/">首页</a>
                <a href="/problems">题目</a>
                <a href="/contests">比赛</a>
                <a href="/submissions">提交记录</a>
                <a href="/users">用户</a>
                <a href="/profile" id="profile-link">个人资料</a>
            </div>
        </div>
    </nav>

    <div class="container">
        <div id="login-prompt" class="card" style="display: none;">
            <h2>请先登录</h2>
            <p>您需要登录后才能查看和修改个人资料。</p>
            <button onclick="location.href='/'" class="btn">返回首页</button>
        </div>

        <div id="profile-content" style="display: none;">
            <div class="card">
                <div class="profile-header">
                    <div class="avatar" id="user-avatar">U</div>
                    <div class="user-info">
                        <h1 id="user-handle">用户名</h1>
                        <div id="user-rating" class="rating-low">0</div>
                    </div>
                </div>

                <div class="user-meta" id="user-meta">
                    <!-- 用户信息将通过JavaScript动态加载 -->
                </div>
            </div>

            <div class="tab-container">
                <div class="tabs">
                    <div class="tab active" onclick="switchTab('edit-profile')">编辑资
料</div>
                    <div class="tab" onclick="switchTab('change-password')">修改密码
</div>
                </div>
```

```
                <div id="edit-profile" class="tab-content active">
                    <div class="card">
                        <h2 class="section-title">编辑个人资料</h2>
                        <div id="profile-message"></div>
                        <form id="profile-form">
                            <div class="form-group">
                                <label class="form-label">姓名</label>
                                <input type="text" id="name" class="form-input"
placeholder="请输入您的姓名">
                            </div>
                            <div class="form-group">
                                <label class="form-label">国家</label>
                                <input type="text" id="country" class="form-input"
placeholder="请输入您的国家">
                            </div>
                            <div class="form-group">
                                <label class="form-label">城市</label>
                                <input type="text" id="city" class="form-input"
placeholder="请输入您的城市">
                            </div>
                            <div class="form-group">
                                <label class="form-label">组织/学校</label>
                                <input type="text" id="organization" class="form-input"
placeholder="请输入您的组织或学校">
                            </div>
                            <div class="form-group">
                                <label class="form-label">头像URL</label>
                                <input type="text" id="avatar" class="form-input"
placeholder="请输入头像图片的URL">
                            </div>
                            <button type="submit" class="btn">保存更改</button>
                        </form>
                    </div>
                </div>

                <div id="change-password" class="tab-content">
                    <div class="card">
                        <h2 class="section-title">修改密码</h2>
                        <div id="password-message"></div>
                        <form id="password-form">
                            <div class="form-group">
                                <label class="form-label">原密码</label>
                                <input type="password" id="old-password" class="form-
input" placeholder="请输入原密码">
                            </div>
                            <div class="form-group">
                                <label class="form-label">新密码</label>
                                <input type="password" id="new-password" class="form-
input" placeholder="请输入新密码">
                            </div>
                            <div class="form-group">
                                <label class="form-label">确认新密码</label>
                                <input type="password" id="confirm-password"
```

```
class="form-input" placeholder="请再次输入新密码">
                            </div>
                            <button type="submit" class="btn">修改密码</button>
                        </form>
                    </div>
                </div>
            </div>
        </div>

    <script>
        let currentUser = null;

        function getRatingClass(rating) {
            if (rating >= 2400) return 'rating-high';
            if (rating >= 1600) return 'rating-medium';
            return 'rating-low';
        }

        function formatDate(dateString) {
            const date = new Date(dateString);
            return date.toLocaleDateString('zh-CN') + ' ' +
date.toLocaleTimeString('zh-CN', {hour: '2-digit', minute: '2-digit'});
        }

        function showMessage(elementId, message, type) {
            const element = document.getElementById(elementId);
            element.innerHTML = `<div class="message ${type}">${message}</div>`;
            setTimeout(() => {
                element.innerHTML = '';
            }, 5000);
        }

        function switchTab(tabName) {
            // 隐藏所有标签内容
            document.querySelectorAll('.tab-content').forEach(tab => {
                tab.classList.remove('active');
            });
            // 移除所有标签的激活状态
            document.querySelectorAll('.tab').forEach(tab => {
                tab.classList.remove('active');
            });
            // 显示选中的标签内容
            document.getElementById(tabName).classList.add('active');
            // 激活选中的标签
            event.target.classList.add('active');
        }

        async function loadUserProfile() {
            try {
                const response = await fetch('/api/user/current');
                const result = await response.json();
```

```
                if (result.success) {
                    currentUser = result.user;
                    displayUserProfile(currentUser);
                    document.getElementById('profile-content').style.display = 'block';
                } else {
                    document.getElementById('login-prompt').style.display = 'block';
                }
            } catch (error) {
                console.error('加载用户资料失败:', error);
                document.getElementById('login-prompt').style.display = 'block';
            }
        }

        function displayUserProfile(user) {
            // 更新头像
            document.getElementById('user-avatar').textContent =
user.handle.charAt(0).toUpperCase();
            if (user.avatar) {
                document.getElementById('user-avatar').style.backgroundImage =
`url(${user.avatar})`;
                document.getElementById('user-avatar').textContent = '';
            }

            // 更新用户名和评分
            document.getElementById('user-handle').textContent = user.handle;
            const ratingElement = document.getElementById('user-rating');
            ratingElement.textContent = user.rating;
            ratingElement.className = getRatingClass(user.rating);

            // 更新用户信息
            const userMeta = document.getElementById('user-meta');
            userMeta.innerHTML = `
                <div class="meta-item">
                    <span class="meta-label">姓名</span>
                    <span class="meta-value">${user.name || '未设置'}</span>
                </div>
                <div class="meta-item">
                    <span class="meta-label">邮箱</span>
                    <span class="meta-value">${user.email}</span>
                </div>
                <div class="meta-item">
                    <span class="meta-label">等级</span>
                    <span class="meta-value">${user.rank}</span>
                </div>
                <div class="meta-item">
                    <span class="meta-label">最高评分</span>
                    <span class="meta-value">${user.max_rating}</span>
                </div>
                <div class="meta-item">
                    <span class="meta-label">国家</span>
                    <span class="meta-value">${user.country || '未设置'}</span>
                </div>
                <div class="meta-item">
```

```
                    <span class="meta-label">城市</span>
                    <span class="meta-value">${user.city || '未设置'}</span>
                </div>
                <div class="meta-item">
                    <span class="meta-label">组织</span>
                    <span class="meta-value">${user.organization || '未设置'}</span>
                </div>
                <div class="meta-item">
                    <span class="meta-label">注册时间</span>
                    <span class="meta-value">${formatDate(user.registration_time)}
</span>
                </div>
                <div class="meta-item">
                    <span class="meta-label">最后在线</span>
                    <span class="meta-value">${formatDate(user.last_online_time)}
</span>
                </div>
            `;

            // 填充表单
            document.getElementById('name').value = user.name || '';
            document.getElementById('country').value = user.country || '';
            document.getElementById('city').value = user.city || '';
            document.getElementById('organization').value = user.organization || '';
            document.getElementById('avatar').value = user.avatar || '';
        }

        // 处理个人资料表单提交
        document.getElementById('profile-form').addEventListener('submit', async
function(e) {
            e.preventDefault();

            const formData = {
                name: document.getElementById('name').value,
                country: document.getElementById('country').value,
                city: document.getElementById('city').value,
                organization: document.getElementById('organization').value,
                avatar: document.getElementById('avatar').value
            };

            try {
                const response = await fetch('/api/user/update', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    },
                    body: JSON.stringify(formData)
                });

                const result = await response.json();

                if (result.success) {
                    showMessage('profile-message', '个人资料更新成功', 'success');
```

```
                    // 重新加载用户资料
                    loadUserProfile();
                } else {
                    showMessage('profile-message', result.message, 'error');
                }
            } catch (error) {
                showMessage('profile-message', '更新失败: ' + error.message, 'error');
            }
        });

        // 处理密码修改表单提交
        document.getElementById('password-form').addEventListener('submit', async
function(e) {
            e.preventDefault();

            const oldPassword = document.getElementById('old-password').value;
            const newPassword = document.getElementById('new-password').value;
            const confirmPassword = document.getElementById('confirm-password').value;

            if (newPassword !== confirmPassword) {
                showMessage('password-message', '新密码和确认密码不一致', 'error');
                return;
            }

            if (newPassword.length < 6) {
                showMessage('password-message', '新密码长度至少为6位', 'error');
                return;
            }

            try {
                const response = await fetch('/api/user/change_password', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    },
                    body: JSON.stringify({
                        old_password: oldPassword,
                        new_password: newPassword
                    })
                });

                const result = await response.json();

                if (result.success) {
                    showMessage('password-message', '密码修改成功', 'success');
                    document.getElementById('password-form').reset();
                } else {
                    showMessage('password-message', result.message, 'error');
                }
            } catch (error) {
                showMessage('password-message', '密码修改失败: ' + error.message,
'error');
            }
```

```
        });

        // 页面加载时获取用户资料
        document.addEventListener('DOMContentLoaded', loadUserProfile);
    </script>
</body>
</html>
```

templates\submissionshtml

to top

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>提交记录 - 在线判题系统</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; background: #f5f5f5; }
        .navbar { background: #2c3e50; color: white; padding: 1rem; }
        .nav-content { max-width: 1200px; margin: 0 auto; display: flex; justify-
content: space-between; }
        .nav-links a { color: white; text-decoration: none; margin-left: 1rem; }
        .container { max-width: 1200px; margin: 2rem auto; padding: 0 1rem; }
        .card { background: white; padding: 1.5rem; border-radius: 8px; box-shadow: 0
2px 4px rgba(0,0,0,0.1); margin-bottom: 1rem; }
        table { width: 100%; border-collapse: collapse; }
        th, td { padding: 0.75rem; text-align: left; border-bottom: 1px solid #ddd; }
        th { background: #f8f9fa; font-weight: bold; }
        .verdict-accepted { color: #28a745; font-weight: bold; }
        .verdict-wrong { color: #dc3545; }
        .verdict-time-limit { color: #fd7e14; }
        .verdict-memory-limit { color: #6f42c1; }
        .verdict-runtime { color: #e83e8c; }
        .verdict-compilation { color: #6c757d; }
        .verdict-other { color: #17a2b8; }
        .filters { margin-bottom: 1rem; display: flex; gap: 1rem; }
        .filter-item { display: flex; align-items: center; gap: 0.5rem; }
        .filter-item select, .filter-item input { padding: 0.5rem; border: 1px solid
#ddd; border-radius: 4px; }
    </style>
</head>
<body>
    <nav class="navbar">
        <div class="nav-content">
            <h1>在线判题系统</h1>
            <div class="nav-links">
                <a href="/">首页</a>
                <a href="/problems">题目</a>
                <a href="/contests">比赛</a>
                <a href="/submissions">提交记录</a>
                <a href="/users">用户</a>
                <a href="/profile" id="profile-link">个人资料</a>
            </div>
        </div>
    </nav>

    <div class="container">
        <div class="card">
            <h2>提交记录</h2>

            <!-- 筛选器 -->
```

```
            <div class="filters">
                <div class="filter-item">
                    <label for="user-filter">用户:</label>
                    <input type="text" id="user-filter" placeholder="用户名">
                </div>
                <div class="filter-item">
                    <label for="problem-filter">题目:</label>
                    <input type="text" id="problem-filter" placeholder="题目标题">
                </div>
                <div class="filter-item">
                    <label for="verdict-filter">结果:</label>
                    <select id="verdict-filter">
                        <option value="">所有结果</option>
                        <option value="Accepted">Accepted</option>
                        <option value="Wrong Answer">Wrong Answer</option>
                        <option value="Time Limit Exceeded">Time Limit
Exceeded</option>
                        <option value="Memory Limit Exceeded">Memory Limit
Exceeded</option>
                        <option value="Runtime Error">Runtime Error</option>
                        <option value="Compilation Error">Compilation Error</option>
                        <option value="Idleness Limit Exceeded">Idleness Limit
Exceeded</option>
                        <option value="Presentation Error">Presentation Error</option>
                        <option value="Partial Solution">Partial Solution</option>
                    </select>
                </div>
                <button onclick="loadSubmissions()">筛选</button>
            </div>

            <div id="submissions-list">
                <table>
                    <thead>
                        <tr>
                            <th>提交ID</th>
                            <th>用户</th>
                            <th>题目</th>
                            <th>语言</th>
                            <th>结果</th>
                            <th>时间</th>
                            <th>内存</th>
                            <th>通过测试</th>
                            <th>提交时间</th>
                        </tr>
                    </thead>
                    <tbody id="submissions-table-body">
                        <!-- 提交数据将通过JavaScript动态加载 -->
                    </tbody>
                </table>
            </div>
        </div>
    </div>
```

```
    <script>
        function getVerdictClass(verdict) {
            switch(verdict) {
                case 'Accepted':
                    return 'verdict-accepted';
                case 'Wrong Answer':
                case 'Presentation Error':
                    return 'verdict-wrong';
                case 'Time Limit Exceeded':
                case 'Idleness Limit Exceeded':
                    return 'verdict-time-limit';
                case 'Memory Limit Exceeded':
                    return 'verdict-memory-limit';
                case 'Runtime Error':
                    return 'verdict-runtime';
                case 'Compilation Error':
                    return 'verdict-compilation';
                default:
                    return 'verdict-other';
            }
        }

        async function loadSubmissions() {
            try {
                const userFilter = document.getElementById('user-filter').value;
                const problemFilter = document.getElementById('problem-filter').value;
                const verdictFilter = document.getElementById('verdict-filter').value;

                let url = '/api/submissions';
                const params = new URLSearchParams();

                if (userFilter) params.append('user_handle', userFilter);
                if (problemFilter) params.append('problem_title', problemFilter);
                if (verdictFilter) params.append('verdict', verdictFilter);

                if (params.toString()) {
                    url += '?' + params.toString();
                }

                const response = await fetch(url);
                const result = await response.json();

                if (result.success) {
                    const tbody = document.getElementById('submissions-table-body');
                    tbody.innerHTML = '';

                    if (result.submissions.length === 0) {
                        const row = document.createElement('tr');
                        row.innerHTML = `<td colspan="9" style="text-align: center;">暂
无提交记录</td>`;
                        tbody.appendChild(row);
                        return;
                    }
```

```
                    result.submissions.forEach(submission => {
                        const verdictClass = getVerdictClass(submission.verdict);

                        const row = document.createElement('tr');
                        row.innerHTML = `
                            <td>${submission.submission_id}</td>
                            <td>${submission.handle}</td>
                            <td>${submission.problem_title}</td>
                            <td>${submission.language}</td>
                            <td class="${verdictClass}">${submission.verdict}</td>
                            <td>${submission.time_consumed}ms</td>
                            <td>${Math.floor(submission.memory_consumed / 1024)}MB</td>
                            <td>${submission.passed_tests || 0}</td>
                            <td>${submission.submission_time}</td>
                        `;
                        tbody.appendChild(row);
                    });
                } else {
                    alert('加载提交记录失败: ' + result.message);
                }
            } catch (error) {
                alert('加载提交记录时发生错误: ' + error.message);
            }
        }

        // 页面加载时获取提交记录
        document.addEventListener('DOMContentLoaded', loadSubmissions);
    </script>
</body>
</html>
```

templates\usershtml

to top

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>用户列表 - 在线判题系统</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; background: #f5f5f5; }
        .navbar { background: #2c3e50; color: white; padding: 1rem; }
        .nav-content { max-width: 1200px; margin: 0 auto; display: flex; justify-
content: space-between; }
        .nav-links a { color: white; text-decoration: none; margin-left: 1rem; }
        .container { max-width: 1200px; margin: 2rem auto; padding: 0 1rem; }
        .card { background: white; padding: 1.5rem; border-radius: 8px; box-shadow: 0
2px 4px rgba(0,0,0,0.1); margin-bottom: 1rem; }
        .user-list { display: grid; gap: 1rem; }
        .user-item { border: 1px solid #e0e0e0; border-radius: 8px; padding: 1.5rem;
display: flex; align-items: center; transition: all 0.3s ease; }
        .user-item:hover { box-shadow: 0 4px 8px rgba(0,0,0,0.1); transform:
translateY(-2px); }
        .user-avatar { width: 60px; height: 60px; border-radius: 50%; background:
#3498db; color: white; display: flex; align-items: center; justify-content: center;
font-size: 1.5rem; font-weight: bold; margin-right: 1rem; }
        .user-info { flex: 1; }
        .user-handle { font-size: 1.25rem; font-weight: bold; color: #2c3e50; margin: 0
0 0.5rem 0; }
        .user-meta { display: flex; gap: 1rem; flex-wrap: wrap; }
        .meta-item { display: flex; flex-direction: column; }
        .meta-label { font-size: 0.875rem; color: #666; margin-bottom: 0.25rem; }
        .meta-value { font-weight: 500; }
        .user-rating { font-size: 1.1rem; font-weight: bold; text-align: right; min-
width: 100px; }
        .rating-high { color: #e74c3c; }
        .rating-medium { color: #e67e22; }
        .rating-low { color: #27ae60; }
        .user-rank { background: #34495e; color: white; padding: 0.25rem 0.5rem;
border-radius: 4px; font-size: 0.875rem; margin-left: 0.5rem; }
        .search-box { margin-bottom: 1rem; }
        .search-input { width: 100%; padding: 0.75rem; border: 1px solid #ddd; border-
radius: 4px; font-size: 1rem; }
        .filters { display: flex; gap: 1rem; margin-bottom: 1rem; flex-wrap: wrap; }
        .filter-select { padding: 0.5rem; border: 1px solid #ddd; border-radius: 4px;
background: white; }
        .admin-badge { background: #e74c3c; color: white; padding: 0.2rem 0.5rem;
border-radius: 3px; font-size: 0.75rem; margin-left: 0.5rem; }
        .user-actions { margin-top: 0.5rem; display: flex; gap: 0.5rem; }
        .btn { padding: 0.3rem 0.8rem; border: none; border-radius: 4px; cursor:
pointer; font-size: 0.875rem; }
        .btn-admin { background: #3498db; color: white; }
        .btn-remove-admin { background: #e74c3c; color: white; }
        .btn:hover { opacity: 0.8; }
```

```
        .admin-section { display: none; }
    </style>
</head>
<body>
    <nav class="navbar">
        <div class="nav-content">
            <h1>在线判题系统</h1>
            <div class="nav-links">
                <a href="/">首页</a>
                <a href="/problems">题目</a>
                <a href="/contests">比赛</a>
                <a href="/submissions">提交记录</a>
                <a href="/users">用户</a>
                <a href="/profile" id="profile-link">个人资料</a>
            </div>
        </div>
    </nav>

    <div class="container">
        <div class="card">
            <h2>用户列表</h2>

            <div class="search-box">
                <input type="text" id="search-input" class="search-input"
placeholder="搜索用户..." onkeyup="filterUsers()">
            </div>

            <div class="filters">
                <select id="rank-filter" class="filter-select"
onchange="filterUsers()">
                    <option value="">所有等级</option>
                    <option value="Newbie">Newbie</option>
                    <option value="Pupil">Pupil</option>
                    <option value="Specialist">Specialist</option>
                    <option value="Expert">Expert</option>
                    <option value="Candidate Master">Candidate Master</option>
                    <option value="Master">Master</option>
                    <option value="International Master">International Master</option>
                    <option value="Grandmaster">Grandmaster</option>
                </select>

                <select id="rating-filter" class="filter-select"
onchange="filterUsers()">
                    <option value="">所有评分</option>
                    <option value="0-1199">0-1199</option>
                    <option value="1200-1399">1200-1399</option>
                    <option value="1400-1599">1400-1599</option>
                    <option value="1600-1899">1600-1899</option>
                    <option value="1900-2399">1900-2399</option>
                    <option value="2400+">2400+</option>
                </select>

                <select id="sort-filter" class="filter-select" onchange="sortUsers()">
```

```
                    <option value="rating-desc">评分降序</option>
                    <option value="rating-asc">评分升序</option>
                    <option value="registration-desc">注册时间降序</option>
                    <option value="registration-asc">注册时间升序</option>
                </select>

                <select id="admin-filter" class="filter-select"
onchange="filterUsers()">
                    <option value="">所有用户</option>
                    <option value="admin">仅管理员</option>
                    <option value="non-admin">非管理员</option>
                </select>
            </div>

            <div id="users-list" class="user-list">
                <!-- 用户数据将通过JavaScript动态加载 -->
            </div>
        </div>
    </div>

    <script>
        let allUsers = [];
        let currentUser = null;
        let isCurrentUserAdmin = false;

        function getRatingClass(rating) {
            if (rating >= 2400) return 'rating-high';
            if (rating >= 1600) return 'rating-medium';
            return 'rating-low';
        }

        function getAvatarText(handle) {
            return handle.charAt(0).toUpperCase();
        }

        function formatDate(dateString) {
            const date = new Date(dateString);
            return date.toLocaleDateString('zh-CN');
        }

        // 检查当前用户权限
        async function checkCurrentUser() {
            try {
                const response = await fetch('/api/user/current');
                const result = await response.json();

                if (result.success && result.user) {
                    currentUser = result.user;
                    isCurrentUserAdmin = result.user.is_admin;
                }
            } catch (error) {
                console.error('获取当前用户信息失败:', error);
            }
```

```
        }

        async function loadUsers() {
            try {
                const response = await fetch('/api/users');
                const result = await response.json();

                if (result.success) {
                    allUsers = result.users;
                    displayUsers(allUsers);
                } else {
                    alert('加载用户列表失败: ' + result.message);
                }
            } catch (error) {
                alert('加载用户列表时发生错误: ' + error.message);
            }
        }

        function displayUsers(users) {
            const usersList = document.getElementById('users-list');
            usersList.innerHTML = '';

            if (users.length === 0) {
                usersList.innerHTML = '<p>暂无用户数据</p>';
                return;
            }

            users.forEach(user => {
                const userItem = document.createElement('div');
                userItem.className = 'user-item';

                const isAdmin = user.is_admin;
                const isCurrentUser = currentUser && user.user_id ===
currentUser.user_id;

                userItem.innerHTML = `
                    <div class="user-avatar">${getAvatarText(user.handle)}</div>
                    <div class="user-info">
                        <h3 class="user-handle">
                            ${user.handle}
                            ${user.name ? `<span style="color: #666; font-weight:
normal;">(${user.name})</span>` : ''}
                            <span class="user-rank">${user.rank}</span>
                            ${isAdmin ? '<span class="admin-badge">管理员</span>' : ''}
                        </h3>
                        <div class="user-meta">
                            ${user.country ? `
                            <div class="meta-item">
                                <span class="meta-label">国家</span>
                                <span class="meta-value">${user.country}</span>
                            </div>
                            ` : ''}
                            ${user.city ? `
```

```
                            <div class="meta-item">
                                <span class="meta-label">城市</span>
                                <span class="meta-value">${user.city}</span>
                            </div>
                            ` : ''}
                            ${user.organization ? `
                            <div class="meta-item">
                                <span class="meta-label">组织</span>
                                <span class="meta-value">${user.organization}</span>
                            </div>
                            ` : ''}
                            <div class="meta-item">
                                <span class="meta-label">最高评分</span>
                                <span class="meta-value">${user.max_rating}</span>
                            </div>
                            <div class="meta-item">
                                <span class="meta-label">最高等级</span>
                                <span class="meta-value">${user.max_rank}</span>
                            </div>
                            <div class="meta-item">
                                <span class="meta-label">注册时间</span>
                                <span class="meta-
value">${formatDate(user.registration_time)}</span>
                            </div>
                        </div>
                        ${isCurrentUserAdmin && !isCurrentUser ? `
                        <div class="user-actions">
                            ${!isAdmin ?
                                `<button class="btn btn-admin"
onclick="setAdmin(${user.user_id}, true)">设为管理员</button>` :
                                `<button class="btn btn-remove-admin"
onclick="setAdmin(${user.user_id}, false)">取消管理员</button>`
                            }
                        </div>
                        ` : ''}
                    </div>
                    <div class="user-rating ${getRatingClass(user.rating)}">
                        ${user.rating}
                    </div>
                `;

                usersList.appendChild(userItem);
            });
        }

        function filterUsers() {
            const searchTerm = document.getElementById('search-
input').value.toLowerCase();
            const rankFilter = document.getElementById('rank-filter').value;
            const ratingFilter = document.getElementById('rating-filter').value;
            const adminFilter = document.getElementById('admin-filter').value;

            let filteredUsers = allUsers.filter(user => {
```

```
                // 搜索过滤
                const matchesSearch = user.handle.toLowerCase().includes(searchTerm) ||
                                      (user.name &&
user.name.toLowerCase().includes(searchTerm));

                // 等级过滤
                const matchesRank = !rankFilter || user.rank === rankFilter;

                // 评分过滤
                let matchesRating = true;
                if (ratingFilter) {
                    const [min, max] = ratingFilter.split('-').map(val => {
                        if (val.endsWith('+')) return parseInt(val) || 0;
                        return parseInt(val) || 0;
                    });

                    if (ratingFilter.endsWith('+')) {
                        matchesRating = user.rating >= min;
                    } else {
                        matchesRating = user.rating >= min && user.rating <= max;
                    }
                }

                // 管理员过滤
                let matchesAdmin = true;
                if (adminFilter === 'admin') {
                    matchesAdmin = user.is_admin;
                } else if (adminFilter === 'non-admin') {
                    matchesAdmin = !user.is_admin;
                }

                return matchesSearch && matchesRank && matchesRating && matchesAdmin;
            });

            displayUsers(filteredUsers);
        }

        function sortUsers() {
            const sortValue = document.getElementById('sort-filter').value;

            const sortedUsers = [...allUsers].sort((a, b) => {
                switch (sortValue) {
                    case 'rating-desc':
                        return b.rating - a.rating;
                    case 'rating-asc':
                        return a.rating - b.rating;
                    case 'registration-desc':
                        return new Date(b.registration_time) - new
Date(a.registration_time);
                    case 'registration-asc':
                        return new Date(a.registration_time) - new
Date(b.registration_time);
                    default:
```

```
                    return 0;
                }
            });

            displayUsers(sortedUsers);
        }

        // 设置/取消管理员权限
        async function setAdmin(userId, isAdmin) {
            if (!confirm(`确定要${isAdmin ? '设为' : '取消'}管理员权限吗？`)) {
                return;
            }

            try {
                const response = await fetch('/api/user/set_admin', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    },
                    body: JSON.stringify({
                        user_id: userId,
                        is_admin: isAdmin
                    })
                });

                const result = await response.json();
                if (result.success) {
                    alert(result.message);
                    // 重新加载用户列表
                    await loadUsers();
                } else {
                    alert('操作失败: ' + result.message);
                }
            } catch (error) {
                alert('操作失败: ' + error.message);
            }
        }

        // 页面加载时获取用户列表和当前用户信息
        document.addEventListener('DOMContentLoaded', async function() {
            await checkCurrentUser();
            await loadUsers();
        });
    </script>
</body>
</html>
```